

Google Cloud Platform (GCP) Core Services: Complete Beginner to Advanced Guide

Prodapt-AID Practice

August 2026

Contents

About This Guide	6
Chapter 1: Introduction to Google Cloud Platform	7
What Is Google Cloud Platform?	7
Regions and Zones: GCP's Global Footprint	7
How GCP Services Are Organized	7
Why This Guide Focuses on Core Infrastructure	8
A Roadmap of the Chapters Ahead	8
Getting Started with GCP	8
Key Takeaways	9
Chapter 2: The GCP Resource Hierarchy: Organizations, Folders & Projects	10
Why a Resource Hierarchy Exists	10
Projects: The Fundamental Container	10
Folders: Grouping Projects	10
The Organization Node	10
IAM Policy Inheritance in Practice	11
A Complete Runnable Example	11
Troubleshooting and Best Practices	12
Key Takeaways	12
Chapter 3: IAM Fundamentals: Identities, Roles & Policies	13
The Three Core Concepts	13
The Three Types of Roles	13
The Principle of Least Privilege	14
A Complete Runnable Example	14
Conditional IAM Bindings	14
Troubleshooting Permission Denied Errors	15
Best Practices for Managing IAM	15
Key Takeaways	16
Chapter 4: Compute Engine Fundamentals: Creating & Managing VMs	17
When to Choose Compute Engine	17
The Building Blocks of a VM Instance	17

Creating a VM Instance	17
Connecting to a VM	19
Stopping, Starting, and Deleting Instances	19
Troubleshooting Common Issues	20
Best Practices	20
Key Takeaways	20
Chapter 5: Compute Engine Deep Dive: Disks, Images & Machine Types	22
Persistent Disk Types	22
Boot Disks vs. Additional Disks	22
Snapshots for Backup and Recovery	22
Images and Image Families	23
Choosing a Machine Type Family	24
Changing a Running VM's Machine Type	24
Troubleshooting and Best Practices	24
Key Takeaways	25
Chapter 6: Managed Instance Groups & Autoscaling	26
The Problem MIGs Solve	26
Instance Templates: The Blueprint	26
Creating a Managed Instance Group	26
Autohealing	27
Autoscaling	27
Rolling Updates	28
Regional vs. Zonal MIGs	28
Troubleshooting and Best Practices	28
Key Takeaways	28
Chapter 7: VPC Networking Fundamentals	30
What Is a VPC on GCP?	30
Auto-Mode vs. Custom-Mode VPCs	30
Creating a Custom-Mode VPC and Subnet	30
Primary and Secondary IP Ranges	32
Routes: How Traffic Finds Its Way	32
Connecting and Sharing Networks: Peering and Shared VPC	32
Troubleshooting: "Why Can't My Subnets Talk to Each Other?"	32
Best Practices	33
Key Takeaways	33
Chapter 8: Firewall Rules, Cloud NAT & Private Connectivity	34
How GCP Firewalls Work	34
Anatomy of a Firewall Rule	34
A Runnable Example: Allowing Web Traffic	34
Implied Rules	35
Cloud NAT: Outbound Internet Access Without an External IP	35
Private Google Access	36
Troubleshooting	36
Best Practices	36
Key Takeaways	36
Chapter 9: Load Balancing on GCP	37
One Product, Many Front Ends	37
Backend Services and Health Checks: The Shared Building Blocks	37
A Runnable Example: Global HTTP(S) Load Balancer in Front of web-mig	39
How the Load Balancer Picks a Backend	40
Troubleshooting: The UNHEALTHY Backend Trap	40
Best Practices	41

Key Takeaways	41
Chapter 10: Cloud DNS & Hybrid Connectivity	42
What Cloud DNS Is	42
Why Hybrid Connectivity Matters	43
Troubleshooting	44
Best Practices	44
Key Takeaways	44
Chapter 11: Cloud Storage Fundamentals	46
What Cloud Storage Is	46
Buckets, Names, and Location Types	46
Storage Classes	46
Creating a Bucket, Uploading, and Listing Objects	47
Object Versioning	47
Troubleshooting	47
Best Practices	48
Key Takeaways	48
Chapter 12: Cloud Storage Security & Lifecycle Management	49
Access Control Models: Uniform Access vs. Fine-Grained ACLs	49
Signed URLs: Temporary Access Without Making Anything Public	49
Object Lifecycle Management	50
Encryption at Rest	50
Versioning, Retention Policies, and Bucket Lock	51
Troubleshooting	51
Best Practices	51
Key Takeaways	51
Chapter 13: Cloud SQL: Managed Relational Databases	52
Why Not Self-Manage on a Compute Engine VM?	52
Creating a Cloud SQL Instance	52
Connecting Securely: Auth Proxy vs. Authorized Networks	53
High Availability: Regional Instances	53
Read Replicas for Scaling Read Traffic	54
Backups and Point-in-Time Recovery	54
Troubleshooting Common Issues	55
Best Practices	55
Key Takeaways	55
Chapter 14: Cloud Spanner & Firestore: Globally Distributed Databases	56
Cloud Spanner: SQL at Global Scale	56
Firestore: Serverless NoSQL for Application Backends	58
Troubleshooting and Cost Considerations	58
Best Practices	59
Key Takeaways	59
Chapter 15: Memorystore & Caching Strategies	60
What Memorystore Is, and Why Applications Cache Data	60
Creating a Memorystore for Redis Instance	60
Basic Tier vs. Standard Tier	60
Common Caching Patterns	60
Cache Invalidation and TTL	61
Connecting from Within the VPC	61
Troubleshooting Common Issues	61
Best Practices	61
Key Takeaways	61

Chapter 16: Service Accounts & Workload Identity	63
Service Accounts vs. User Accounts	63
The Default Compute Engine Service Account	63
A Complete Example: Dedicated Service Account for a Workload	63
Why Downloaded JSON Keys Are Risky	64
Service Account Impersonation	65
Workload Identity Federation	65
Troubleshooting	66
Best Practices	66
Key Takeaways	66
Chapter 17: Cloud KMS, Secret Manager & Data Encryption	67
Encryption at Rest: The Default, and When You Need More	67
Cloud KMS Core Concepts	67
A Complete Example: CMEK-Protected Cloud Storage Bucket	67
Secret Manager: A Different Problem	68
A Complete Example: Storing and Retrieving a Secret	68
Why Not Environment Variables or Config Files?	69
Troubleshooting	69
Best Practices	69
Key Takeaways	70
Chapter 18: Cloud Monitoring & Cloud Logging	71
The Operations Suite: On By Default	71
Cloud Logging: Structure and Retention	71
Log-Based Metrics	72
Cloud Monitoring: Structure	72
A Complete Example: Alerting on CPU Utilization	72
Monitoring vs. Logging: Two Different Questions	74
Troubleshooting	74
Best Practices	75
Key Takeaways	75
Chapter 19: Cloud Audit Logs & Security Command Center	76
The Three Types of Cloud Audit Logs	76
Security Command Center	77
Best Practices	78
Key Takeaways	78
Chapter 20: Billing, Budgets & Cost Management	79
The Billing Account	79
Finding Current Spend	79
Budgets and Budget Alerts	79
Labels for Cost Attribution	80
Discounts: Sustained Use and Committed Use	80
Quotas: A Related but Distinct Concept	81
Troubleshooting	81
Best Practices	81
Key Takeaways	82
Chapter 21: Cloud Build & CI/CD Fundamentals on GCP	83
What Cloud Build Does	83
Build Triggers	84
Substitutions and Build Variables	84
Viewing Build Logs	84
CI Versus CD, and Where Cloud Build Fits	85
Troubleshooting	85

Best Practices	85
Key Takeaways	85
Chapter 22: High Availability, Disaster Recovery & Production Best Practices	87
Reference Architecture Walkthrough	87
High Availability Patterns	89
Disaster Recovery Strategies	89
Production Checklist	90
Chapter 23: Interview Questions	91
Chapter 24: Commands Cheat Sheet	96
Project & IAM Commands	96
Compute Engine Commands	97
Networking Commands	97
Storage & Database Commands	98
Security Commands	99
Monitoring & Billing Commands	100
Quick Reference: Useful Flags	100
Chapter 25: Summary	102
Where to Go Next	102
Certifications Worth Pursuing	103
Closing Thoughts	103

Compute, Storage, Networking, Databases, IAM, Monitoring & Cost Management

A practical, hands-on guide for Cloud Engineers, DevOps Engineers, Platform Engineers, and Solution Architects

Prepared by: **Prodapt-AID Practice**
Edition: **2026**

About This Guide

This guide takes you from zero Google Cloud experience to designing and operating production-grade infrastructure on GCP's core services. It is written for five audiences at once — complete beginners, Cloud Engineers, DevOps Engineers, Platform Engineers, and Solution Architects — so every concept is first explained simply, then reinforced with a real, runnable example.

This guide focuses on the foundational services almost every GCP project touches: the resource hierarchy and IAM, Compute Engine, VPC networking and load balancing, Cloud Storage, managed databases (Cloud SQL, Spanner, Firestore), security and encryption, Cloud Monitoring and Logging, and billing and cost management. It is designed as a companion to two related guides — *Terraform on GCP* and *Docker, Kubernetes, GKE & GCP Artifact Registry* — so container orchestration and infrastructure-as-code are only touched on lightly here where relevant.

How to use this guide:

TIP

If you are brand new to GCP, read Chapters 1–3 first — they cover the account/project model and IAM before anything else, since every other service depends on those concepts. If you already have a GCP account, jump to the compute, storage, or networking chapters most relevant to your work. If you are preparing for an interview, jump straight to Chapter 23.

Every chapter follows the same pattern: a plain-language explanation, an architecture diagram where useful, a complete and runnable example (gcloud command, configuration, or console walk-through), a clear explanation of what the command or configuration does, and — where applicable — a troubleshooting and best-practices section.

Examples use illustrative values such as project ID `my-gcp-project-123456`, region `us-central1`, zone `us-central1-a`, and VPC network `my-vpc`. Replace these with your own values before running any command.

Chapter 1: Introduction to Google Cloud Platform

What Is Google Cloud Platform?

Google Cloud Platform (GCP) is Google’s public cloud — a collection of computing, storage, networking, database, and management services that you rent by the hour, minute, or gigabyte instead of buying your own hardware. Instead of purchasing servers and wiring switches in a data center you own, you provision virtual machines, databases, storage buckets, and networking constructs through a web console, a command-line tool, or code, and pay only for what you use.

GCP is one of the “big three” public cloud providers, alongside Amazon Web Services (AWS) and Microsoft Azure. All three offer similar categories of services but use different terminology and have different strengths. GCP runs on the infrastructure that powers Google Search, Gmail, and YouTube, and is often noted for data analytics, machine learning, its global fiber network, and Kubernetes, which it originated. This book does not rank the providers — it focuses on GCP’s core services.

NOTE

If you have used AWS or Azure before, many concepts will feel familiar under different names — an AWS “VPC” and a GCP “VPC network” solve the same problem. This book assumes no prior cloud experience.

Regions and Zones: GCP’s Global Footprint

GCP infrastructure is spread across the world in a two-level hierarchy of **regions** and **zones**.

A **region** is an independent geographic area, such as `us-central1` (Iowa), `eu-west1` (Belgium), or `asia-south1` (Mumbai). Regions are spaced far enough apart that a disaster affecting one is unlikely to affect another, making cross-region deployment a common disaster-recovery strategy.

A **zone** is a deployment area inside a region — effectively a data center with independent power, cooling, and networking. A region typically contains three or more zones, named with a letter suffix such as `us-central1-a` and `us-central1-b`, linked by low-latency connections so spreading an application across zones for availability carries little performance penalty. Rule of thumb: pick a region close to your users, and spread redundant copies of your workload across at least two zones.

TIP

Some resources are zonal (a Compute Engine VM), some regional (a managed instance group, a Cloud SQL instance), and some global (a Cloud Storage bucket, a load balancer’s IP). Check a service’s scope before designing around it.

How GCP Services Are Organized

GCP offers well over a hundred services, but nearly all fall into a handful of categories this book is organized around:

- **Compute** — Compute Engine (VMs), GKE and Cloud Run (containers), Cloud Functions (managed functions).
- **Storage** — object storage (Cloud Storage), plus block/file storage attached to VMs.
- **Networking** — VPC networks, firewalls, load balancers, hybrid connectivity.
- **Databases** — Cloud SQL (relational), Cloud Spanner (global relational), Firestore/Bigtable (NoSQL).
- **Security and identity** — IAM, the resource hierarchy, encryption, audit logging.
- **Operations** — Cloud Monitoring/Logging and cost visibility.

Every chapter maps to one of these categories, building from the ground up.

Why This Guide Focuses on Core Infrastructure

This book is the third in a series from Prodapt-AID Practice. The first, *Terraform on GCP*, covers infrastructure-as-code; the second, *Docker, Kubernetes, GKE & GCP Artifact Registry*, covers containerization and orchestration. Both assume working knowledge of the services they provision and orchestrate — projects, IAM, VPC networks, Compute Engine, Cloud Storage. This guide fills that gap, going deep on the managed services themselves — what they are, how they behave, how they are priced — so that when you return to Terraform or GKE, you understand what is being created underneath.

A Roadmap of the Chapters Ahead

The 25 chapters are grouped into eight themes: **Foundations** (1–3: this introduction, resource hierarchy, IAM), **Compute** (4–7), **Networking** (8–12), **Storage** (13–15), **Databases** (16–18: Cloud SQL, Spanner, Firestore/Bigtable), **Security** (19–20), **Operations and cost** (21–22), and **Reference material** (23–25: interview prep, a CLI cheat sheet, closing summary).

Getting Started with GCP

Before the hands-on examples begin, you need an account and the CLI installed.

1. **Create a Google Cloud account** at cloud.google.com. New accounts typically receive free trial credit (commonly \$300 for 90 days) — check the current offer, as terms change.
2. **Install the gcloud CLI** from cloud.google.com/sdk/docs/install.
3. **Authenticate and initialize** with `gcloud init`, or run the commands separately:

```
gcloud auth login
gcloud config set project my-gcp-project-123456
```

- `gcloud auth login` opens a browser window so you can sign in and grant the CLI permission to act on your behalf.
- `gcloud config set project my-gcp-project-123456` sets the active project for subsequent commands, so you don't need `--project` on every invocation.

Inspect your current settings at any time:

```
gcloud config list
```

```
[core]
account = you@example.com
disable_usage_reporting = True
project = my-gcp-project-123456
```

Your active configuration is: [default]

- `account` shows which identity the CLI is authenticated as.
- `project` confirms which project your commands target — every resource belongs to exactly one project.
- Your active configuration is: [default] indicates `gcloud` supports multiple named configurations, useful when switching between projects.

BEST PRACTICE

Run `gcloud config list` right after setting up a new machine or switching contexts — it is the fastest way to confirm which project a command will affect.

With an account, the CLI, and a default project set, you are ready to build. Chapter 2 covers the resource hierarchy every project lives inside; Chapter 3 covers who can do what to it.

Key Takeaways

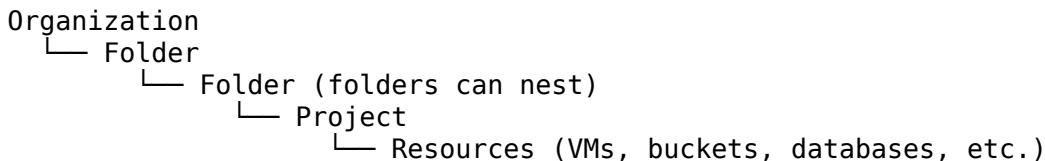
- GCP is one of three major public cloud providers, organized into categories: compute, storage, networking, databases, security/identity, and operations.
- Regions are independent geographic areas; zones are isolated data centers within a region. Spread workloads across zones, and regions, for resilience.
- This guide complements the companion *Terraform on GCP* and *Docker, Kubernetes, GKE & GCP Artifact Registry* guides by going deep on core managed services.
- The `gcloud` CLI, set up with `gcloud init` (or `gcloud auth login` plus `gcloud config set project`), is the primary tool used throughout this book.
- `gcloud config list` is a quick sanity check of which account and project your commands will affect.

Chapter 2: The GCP Resource Hierarchy: Organizations, Folders & Projects

Why a Resource Hierarchy Exists

Every resource you create in GCP — a VM, a storage bucket, a database — has to live somewhere administratively, not just physically. GCP solves this with a **resource hierarchy**: a tree structure determining who is billed for a resource, who can access it, and which policies apply.

The hierarchy has four levels, broadest to narrowest:



The reason this structure matters is **inheritance**. An Identity and Access Management (IAM) role or organization policy applied at a higher level automatically applies to everything beneath it — grant a role at the Organization and every folder, project, and resource inherits it. This lets an administrator manage access and guardrails for hundreds of projects in one place.

Projects: The Fundamental Container

The **Project** is the level at which almost everything happens. Every resource belongs to exactly one project, and a project is the boundary for billing, API enablement, and most quota limits.

A project has three identifying attributes:

- **Project ID** — a globally unique, human-chosen string, such as `my-gcp-project-123456`. Immutable, and cannot be reused after the project is deleted, which is why organizations often append random digits to avoid collisions.
- **Project number** — an automatically assigned numeric identifier (for example, `847213659021`), used internally by some APIs.
- **Project name** — a human-readable label (for example, “Web App Production”) that can be changed anytime.

NOTE

The project ID is what you will type constantly in `gcloud` commands and Terraform configuration, so choose it deliberately — once set, it cannot be changed.

Every project must link to a billing account to create billable resources. A single billing account can link to many projects, centralizing billing while isolating workloads.

Folders: Grouping Projects

Folders sit between the Organization and Projects, existing purely to group projects (or other folders) for policy and access purposes — commonly by environment (Production, Non-Production) or by team. Folders can nest: Organization → Engineering → Production → `payments-prod` (project). They are only available under an Organization node — a standalone project cannot use folders.

The Organization Node

The **Organization** is the root of the hierarchy, tied to a Google Workspace or Cloud Identity domain — for example, `example.com`. GCP automatically creates an Organization node the first time a project is created under a verified domain, and every project created in that domain becomes part of it by default.

This is also why a personal Gmail account gets no Organization node — there is no verified domain

to anchor one to. That limits centralized governance but is fine for learning or individual use.

BEST PRACTICE

Verify a domain and set up Cloud Identity (or Workspace) early, before creating many projects. Migrating existing “orphan” projects into an Organization later is possible but adds friction.

IAM Policy Inheritance in Practice

Because policies flow downward, a role granted higher in the hierarchy silently applies everywhere below it.

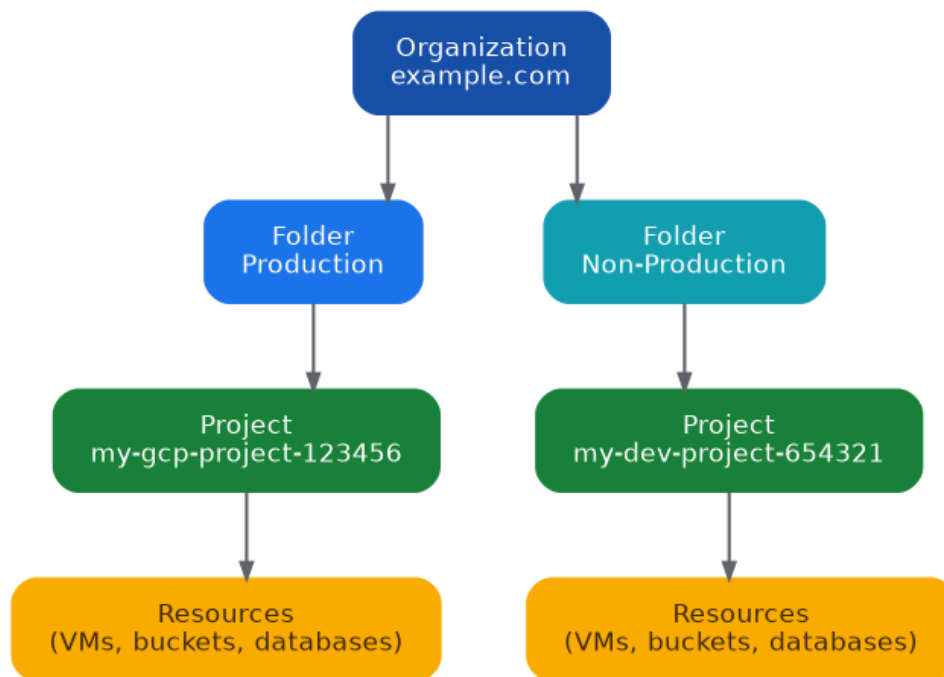


Figure 1: The GCP resource hierarchy: an Organization contains Folders, which contain Projects, which contain the actual resources.

Consider a scenario: folder Production contains two projects, payments-prod and checkout-prod. If a Site Reliability Engineering (SRE) group is granted roles/monitoring.viewer at the Production folder, every member can view Cloud Monitoring dashboards in both projects without individual grants — and automatically gains the same visibility into any new project later created inside that folder. This is the biggest reason to invest in a sensible folder structure early: it turns access management into a one-time, hierarchy-level decision.

A Complete Runnable Example

The following creates a folder under an organization, then a project inside it.

```
gcloud resource-manager folders create \  
  --display-name="Production" \  
  --organization=123456789012
```

- `--display-name="Production"` sets the folder name.
- `--organization=123456789012` is the numeric Organization ID (find it with `gcloud`

organizations list).

```
gcloud projects create my-gcp-project-123456 \
  --name="Web App Production" \
  --folder=987654321098
```

- my-gcp-project-123456 is the immutable project ID.
- --name="Web App Production" sets the display name, changeable later.
- --folder=987654321098 places the project under that folder ID (returned by the folder-create command, or via gcloud resource-manager folders list).

To place the project directly under the Organization, replace --folder= with --organization=123456789012 — a project has exactly one parent, never both.

Finally, link the project to a billing account:

```
gcloud billing projects link my-gcp-project-123456 \
  --billing-account=012345-6789AB-CDEF01
```

- --billing-account=012345-6789AB-CDEF01 is the billing account ID, found with gcloud billing accounts list.

Troubleshooting and Best Practices

A few structural decisions save significant pain later:

- **Do not put everything in one project** — a monolithic project makes IAM overly broad and complicates billing attribution. Separate by environment and team.
- **Use a consistent folder structure** — by environment or by team, sometimes combined.
- **Use labels for lightweight metadata**, such as cost-center: platform-eng, for cost reporting without added complexity.
- **Common errors:** gcloud projects create failing with “project ID already in use” (IDs are globally unique — add a suffix, as in my-gcp-project-123456), or folder creation failing without roles/resourcemanager.folderCreator (or broader) at the Organization or parent-folder level.

WARNING

Deleting a project does not immediately free its project ID for reuse, and deletion has a 30-day recovery window before resources are purged. Do not treat it as instantaneous or reversible after that window closes.

With the hierarchy and a project in place, the next chapter covers how IAM grants access within it — the roles, principals, and policy bindings deciding who can do what.

Key Takeaways

- The resource hierarchy — Organization, Folder, Project, Resource — centralizes billing, access control, and policy management.
- A Project is the fundamental container for resources and billing, identified by an immutable Project ID, a numeric Project Number, and a mutable Project Name.
- Folders group projects for policy and access purposes; the Organization node is the root, tied to a verified domain such as example.com, and does not exist automatically for personal accounts.
- IAM roles and organization policies granted higher in the hierarchy are inherited by everything beneath them, including resources created later.
- Avoid monolithic projects; separate by environment or team, and use labels for lightweight metadata instead of added hierarchy depth.

Chapter 3: IAM Fundamentals: Identities, Roles & Policies

The Three Core Concepts

Identity and Access Management (IAM) is the system that controls who can do what to which resource in GCP. Every access decision — reading a Cloud Storage bucket, writing to a database, deploying code via a pipeline — is governed by IAM. Its three core building blocks make every other security topic in this book easier to reason about.

Principals are the “who”: a human user (user:ana@example.com), a group (group:data-team@example.com), a service account (serviceAccount:app-sa@my-gcp-project-123456.iam.gserviceaccount.com), or one of two special identifiers: allUsers (anyone on the internet) and allAuthenticatedUsers (anyone with a Google account). The last two are powerful and rarely appropriate outside intentionally public resources, such as static assets on a public website.

Roles are the “what.” A role is a named collection of permissions — a fine-grained capability such as storage.objects.get (read an object) or compute.instances.start (start a VM). You almost never grant individual permissions directly — you grant roles, which bundle related permissions together.

Policy bindings connect the two. An IAM policy attached to a resource is a list of bindings, where each pairs one role with one or more principals on one resource (which, per Chapter 2’s hierarchy, also affects everything beneath it). The general shape is always: *this principal has this role on this resource*.

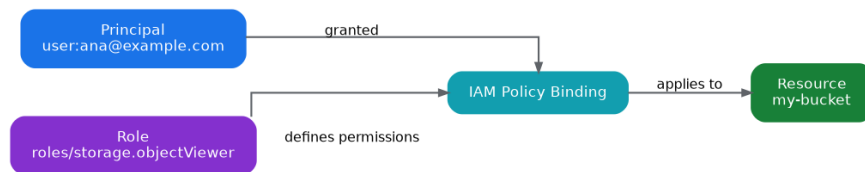


Figure 2: An IAM policy binding ties a principal and a role together and applies that combination to a resource.

The Three Types of Roles

GCP offers three categories of roles, and choosing the right one is a consequential security decision you will make repeatedly.

Basic (primitive) roles — roles/owner, roles/editor, and roles/viewer — predate GCP’s fine-grained IAM system and grant extremely broad access across a project. roles/editor can modify almost any resource; roles/owner also manages IAM policy and billing. They suit sandbox experiments but are discouraged in production, since they violate least privilege — a user who only needs to restart a VM has no business deleting a production database.

Predefined roles — such as roles/storage.objectViewer (read objects, no delete/modify), roles/compute.instanceAdmin.v1 (full instance control, not networking or IAM), and roles/cloudsql.client (connect to Cloud SQL, no admin rights) — are curated, service-specific bundles Google maintains as services evolve. These cover most real-world needs and should be your default.

Custom roles let you assemble your own permission set when nothing predefined fits — for example, starting/stopping instances but not deleting them. They need more upkeep, since Google does not add permissions to them automatically, so reserve them for unusual requirements.

BEST PRACTICE

Default to predefined roles. Reach for basic roles only in sandbox projects, and for custom roles only after confirming, via the predefined roles reference, that nothing pre-built fits.

The Principle of Least Privilege

Least privilege means granting a principal only the permissions it needs, and nothing more. This limits the blast radius of a mistake or compromised credential — a service account restricted to one bucket cannot let a leaked key delete unrelated production databases — and makes audits tractable, since narrow grants produce an actionable list instead of blanket “everything” permissions.

Least privilege is a discipline, not a single setting: regularly ask whether each binding still needs that exact role, and remove access once it is no longer required — a project ends, a person changes teams, a job is retired.

A Complete Runnable Example

To grant a user read-only access to objects in Cloud Storage across a project:

```
gcloud projects add-iam-policy-binding my-gcp-project-123456 \
  --member="user:ana@example.com" \
  --role="roles/storage.objectViewer"
```

- my-gcp-project-123456 is the project whose policy is modified — the binding applies here and, by inheritance, everywhere below it.
- --member="user:ana@example.com" identifies the principal; the user: prefix distinguishes an individual account from group: or serviceAccount:.
- --role="roles/storage.objectViewer" grants read access to objects, without modify, delete, or bucket-config permissions.

To grant a service account permission to manage Compute Engine instances (common for CI/CD):

```
gcloud projects add-iam-policy-binding my-gcp-project-123456 \
  --member="serviceAccount:app-sa@my-gcp-project-123456.iam.gserviceaccount.com" \
  --role="roles/compute.instanceAdmin.v1"
```

- The serviceAccount: prefix identifies the principal as a service account, representing an application or automated process rather than a human.
- roles/compute.instanceAdmin.v1 grants control over instances (create, start, stop, delete, resize) without broader access such as IAM administration.

To review the full IAM policy attached to a project:

```
gcloud projects get-iam-policy my-gcp-project-123456 --format=json
```

- --format=json renders the policy as structured JSON, easier to diff or feed into a policy-as-code pipeline.

The output lists each binding as a role paired with an array of members, plus an etag (a version marker preventing concurrent overwrites).

The Cloud Console’s IAM page shows the same information visually, listing every principal with access alongside their roles — often the fastest way to spot an overly broad grant.

Conditional IAM Bindings

For advanced scenarios, GCP supports **conditional IAM bindings** — a binding that only takes effect when a Common Expression Language (CEL) condition evaluates to true. Two common uses are time-based access (a role expiring on a specific date) and resource-based access (roles/storage.objectViewer limited to objects matching a name prefix). A time-bound example attaches a condition such as request.time < timestamp("2026-12-31T00:00:00Z"), after which the grant stops applying automatically. This adds real power for temporary access, but also complexity to auditing, so use it deliberately.

The screenshot shows the Google Cloud IAM Admin console interface. At the top, the breadcrumb navigation reads "IAM & Admin > IAM". Below this is a table listing the principal-role bindings for a project. The table has three columns: "PRINCIPAL", "ROLE", and "TYPE".

PRINCIPAL	ROLE	TYPE
ana@example.com	Storage Object Viewer	User
app-sa@my-gcp-project-123456.iam.gserv	Cloud SQL Client	Service Acct.
group:platform-team@example.com	Compute Admin	Group

Figure 3: Illustrative an IAM permissions page showing principal-role bindings for a project.

Troubleshooting Permission Denied Errors

“Permission denied” is the most common error new GCP users hit, usually tracing back to one of a few causes:

- **Propagation delay.** IAM changes are eventually consistent and can take up to several minutes (up to 7 in typical cases). If a command fails right after a grant, wait and retry before assuming the grant is wrong.
- **Wrong resource level.** A role granted on one project does nothing for a different project; bucket-level and project-level policies are independent (though project grants flow down by inheritance).
- **Missing API enablement.** The right role is not enough if the underlying API was never enabled — roles/compute.instanceAdmin.v1 does nothing until gcloud services enable compute.googleapis.com is run.
- **Checking the wrong identity.** Confirm the active account with `gcloud auth list` before troubleshooting further.

WARNING

Basic roles can mask permission problems during testing — if roles/editor “just works” while a narrower role fails, that signals a missing permission, not a reason to keep the broad role in production.

Best Practices for Managing IAM

- **Prefer groups over individual bindings** — granting roles to a Google Group (group:data-team@example.com) means membership changes happen in the group, not across dozens of policies.
- **Prefer predefined roles over basic roles**, and reserve custom roles for unmet needs.
- **Audit regularly with gcloud projects get-iam-policy**, watching for basic roles, allUsers/allAuthenticatedUsers on non-public resources, and over-privileged service accounts.
- **Rotate and review access on a schedule** — access that made sense at project start often goes stale as teams evolve.

IAM is the security foundation every later chapter builds on: the same principals, roles, and bindings

introduced here recur throughout the guide.

Key Takeaways

- IAM decisions rest on three concepts: principals (who), roles (what), and policy bindings (which principal has which role on which resource).
- Basic roles (Owner, Editor, Viewer) are broad and generally unsuitable for production; predefined roles are the default choice; custom roles fill genuine gaps only.
- Least privilege — granting only needed access, removing it when no longer needed — limits both accidental damage and compromised-credential impact.
- Conditional IAM bindings add time-based or resource-based restrictions on top of a role grant.
- Most “permission denied” errors trace back to propagation delay, the wrong resource level, or a disabled API.

Chapter 4: Compute Engine Fundamentals: Creating & Managing VMs

Compute Engine is Google Cloud’s Infrastructure-as-a-Service (IaaS) offering for virtual machines. “IaaS” means Google manages the physical hardware, hypervisor, and data center facilities, while you manage everything above that: the operating system, the runtime, and the application. A Compute Engine instance behaves like a server you would rack yourself, except it can be created in seconds, resized on demand, and billed by the second.

This chapter covers what makes up a VM instance, how to create and connect to one, how to stop and delete instances safely, and how to troubleshoot the problems almost every new GCP user hits in their first week.

When to Choose Compute Engine

GCP offers several ways to run workloads. If your workload is a container that can scale to zero, Cloud Run or Google Kubernetes Engine (GKE) are usually a better fit — covered in the companion guide *Docker, Kubernetes, GKE & GCP Artifact Registry*. If it is a short-lived function triggered by an event, Cloud Functions is lighter weight. Compute Engine is the right choice when you need full control over the operating system, are running software that expects a persistent, addressable machine (databases, legacy applications, licensing-bound software), or are lifting-and-shifting an on-premises workload with minimal changes.

NOTE

This chapter focuses on VMs you manage individually. Chapter 6 introduces Managed Instance Groups (MIGs), which run a fleet of identical VMs that scale and heal automatically — the pattern most production workloads should actually use.

The Building Blocks of a VM Instance

Every Compute Engine instance is defined by a small set of properties:

- **Machine type** — the number of virtual CPUs (vCPUs) and memory available, for example e2-medium (2 vCPUs, 4 GB). Machine types are grouped into families optimized for different workloads, covered in Chapter 5.
- **Boot disk and image** — the persistent disk the OS is installed on, created from a source image or an image family, which always resolves to the latest non-deprecated image in that family.
- **Network interface(s)** — connects the VM to a VPC (Virtual Private Cloud) network and subnet, optionally with an external IP.
- **Attached service account** — an identity the VM uses to call other Google Cloud APIs without embedding credentials on the disk.
- **Labels and network tags** — labels are key-value metadata for organization and billing; network tags target firewall rules at specific instances.
- **Startup script** — a shell script that runs automatically on every boot, commonly used to install packages or pull configuration.

Creating a VM Instance

```
gcloud compute instances create web-server-1 \
  --zone=us-central1-a \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --tags=http-server \
  --service-account=app-sa@my-gcp-project-123456.iam.gserviceaccount.com \
```

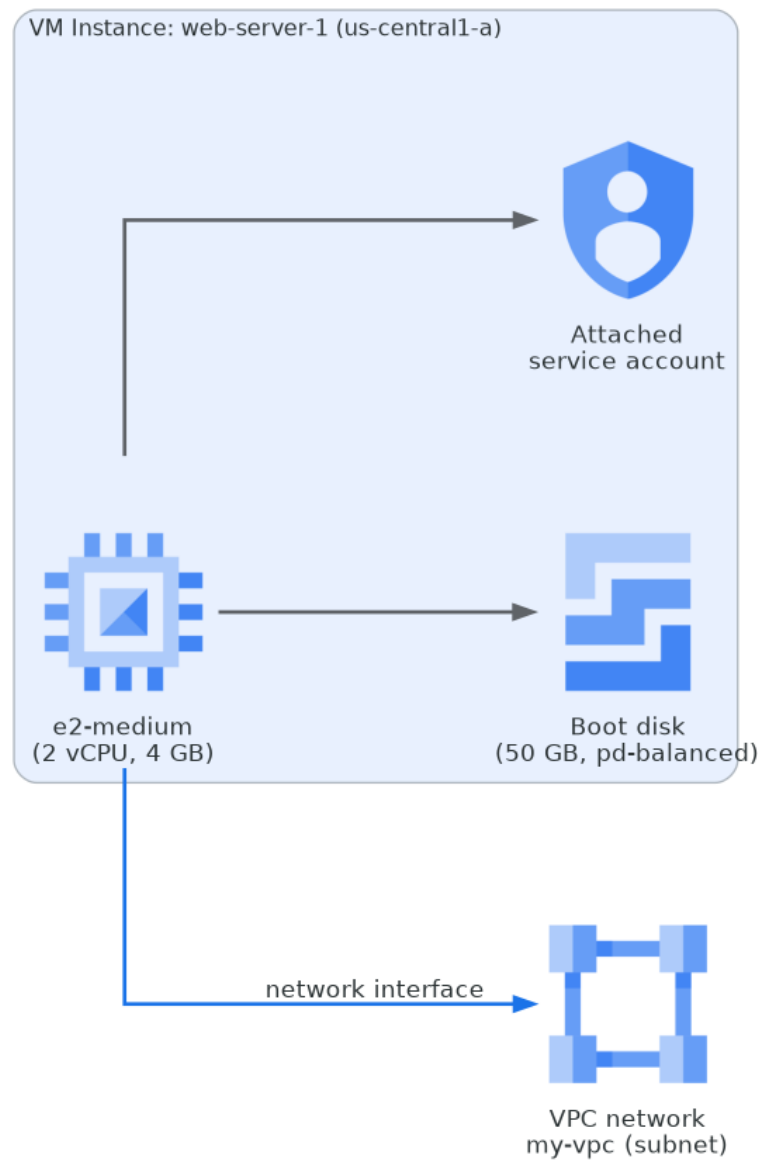


Figure 4: The anatomy of a Compute Engine VM instance: a boot disk, an attached service account, and a network interface into a VPC.

```
--scopes=cloud-platform
```

- `web-server-1` — the instance name; must be unique within the project and follow DNS-label naming rules.
- `--zone=us-central1-a` — VMs are zonal resources, so the zone must be specified explicitly.
- `--machine-type=e2-medium` — a general-purpose E2 machine type, a sensible default for a small web server.
- `--image-family=debian-12` with `--image-project=debian-cloud` — resolves to the latest Debian 12 public image, rather than pinning a specific image that will eventually be deprecated.
- `--tags=http-server` — a network tag used to target this instance with firewall rules.
- `--service-account` / `--scopes=cloud-platform` — attaches `app-sa` and allows it to call Google Cloud APIs, with actual permissions controlled by the service account's IAM role bindings rather than the scope itself.

`gcloud compute instances list` afterward shows the instance's zone, machine type, IPs, and status (RUNNING).

BEST PRACTICE

Tag instances that share a purpose (`http-server`, `backend`, `db`) rather than writing firewall rules against individual instance names, so you can add or remove instances from a traffic pattern without editing rules.

Connecting to a VM

```
gcloud compute ssh web-server-1 --zone=us-central1-a
```

- `gcloud compute ssh` wraps standard `ssh` with Google Cloud's authentication model, so you do not manage key files manually.
- `--zone=us-central1-a` — instance names are only unique within a zone, so the CLI (command-line interface) needs to know where to look.

By default, `gcloud` generates an SSH key pair on your workstation and propagates the public key to the VM's instance metadata; the guest agent inside the VM reads that metadata and adds the key to the appropriate user's `~/.ssh/authorized_keys`, usually within seconds. If OS Login is enabled, authentication is tied to your Google identity and IAM role instead, which centralizes access control and audit logging. Either way, the first connection can take a few extra seconds while metadata propagates — a normal delay, not a failure.

Stopping, Starting, and Deleting Instances

VMs are billed by the second while running, so stopping an unused instance is a simple cost control:

```
gcloud compute instances stop web-server-1 --zone=us-central1-a
gcloud compute instances start web-server-1 --zone=us-central1-a
gcloud compute instances delete web-server-1 --zone=us-central1-a
```

- `stop` shuts down the guest OS and releases compute (vCPU/memory) resources — no further compute billing.
- `start` boots the instance again from the same boot disk, retaining all data.
- `delete` permanently removes the instance and, unless created with `--no-boot-disk-auto-delete`, its boot disk too.

WARNING

A stopped instance still incurs charges for persistent disk storage and any reserved static external IP. Stopping a VM cuts costs substantially but is not free — delete resources you no longer need

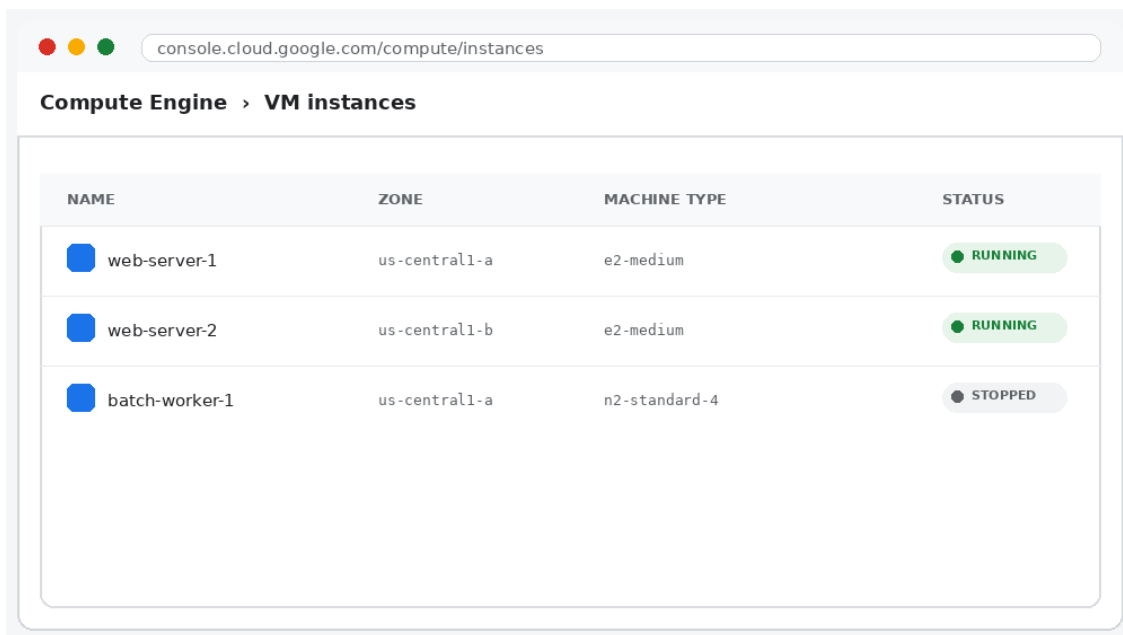


Figure 5: Illustrative the Compute Engine VM instances console showing running instances.

to stop paying entirely.

Troubleshooting Common Issues

- **SSH connection times out.** Usually a missing firewall rule allowing inbound TCP port 22. Networks have an implied deny-all-ingress rule; auto-mode networks include a default-allow-ssh rule, but custom-mode VPCs (Chapter 8) need one created explicitly.
- **Quota exceeded or ZONE_RESOURCE_POOL_EXHAUSTED errors.** Either the zone has temporarily run out of capacity for that machine type (try another zone), or your project has hit a vCPU/IP/disk quota — check IAM & Admin > Quotas in the console.
- **Image family typos.** `--image-family=debian12` (missing the hyphen) or a wrong `--image-project` produces a “resource not found” error. Public image families live in projects like `debian-cloud`, `ubuntu-os-cloud`, and `windows-cloud`, not your own project.

Best Practices

- Use instance templates (Chapter 6) instead of hand-crafted create commands once you need more than one identical VM.
- Tag instances consistently for firewall targeting rather than relying on names or IP ranges.
- Never store plaintext secrets in startup scripts or instance metadata; both are readable by anyone with instance access, and metadata is also readable from inside the VM. Use Secret Manager (Chapter 15) and fetch secrets at boot using the attached service account’s identity.

Key Takeaways

- Compute Engine is Google’s IaaS VM offering; choose it for full OS-level control or migrating existing workloads, and prefer GKE, Cloud Run, or Cloud Functions otherwise.
- A VM is defined by its machine type, boot disk/image, network interface, service account, labels/tags, and optional startup script.
- `gcloud compute ssh` relies on SSH key propagation through metadata (or OS Login) and a guest agent; a short delay on first connect is normal.
- Stopping an instance halts compute billing but persistent disks and reserved static IPs continue to incur charges until deleted.

- Tag instances for firewall targeting, avoid plaintext secrets in startup scripts, and move to instance templates once you need more than a single VM.

Chapter 5: Compute Engine Deep Dive: Disks, Images & Machine Types

Chapter 4 introduced the basic shape of a Compute Engine instance. This chapter goes deeper into three things that determine the performance and cost of your VMs: disks, images, and machine type families.

Persistent Disk Types

Compute Engine's default storage is a **persistent disk (PD)** — network-attached block storage that is durable and independent of the VM's lifecycle, meaning the disk can survive even if the instance is deleted (unless auto-delete is configured). There are four persistent disk types:

- **pd-standard** — rotational hard disk drives (HDD). Lowest cost per gigabyte, but the lowest input/output operations per second (IOPS). Best for sequential-access workloads like batch log processing.
- **pd-balanced** — SSD-backed, a general-purpose middle ground and the default choice for most workloads, including boot disks.
- **pd-ssd** — SSD-backed, tuned for consistently higher IOPS and lower latency than pd-balanced, at a higher price. Appropriate for databases and latency-sensitive workloads.
- **pd-extreme** — the highest-performance tier, with independently provisionable IOPS. For the most demanding transactional databases, at the highest cost.

BEST PRACTICE

Start with pd-balanced unless you have a measured reason to choose otherwise. It is priced close to pd-standard while performing much closer to pd-ssd for most general workloads.

Boot Disks vs. Additional Disks

Every VM has exactly one **boot disk**, created from a source image, containing the OS the instance boots from. You can also attach additional (data) disks — useful for separating application data from the OS or moving a disk between instances.

Resizing a disk while the VM keeps running is routine for increasing capacity (persistent disks cannot be shrunk):

```
gcloud compute disks resize web-server-1 \
  --zone=us-central1-a \
  --size=100GB
```

- web-server-1 — the disk name, which by default matches the instance name for a boot disk created alongside the VM.
- --size=100GB — the new total size; must be larger than the current size.

NOTE

Resizing only changes the block device's size. Inside the guest OS, you still need to grow the filesystem (resize2fs for ext4, xfs_growfs for XFS) before the extra space is usable.

Snapshots for Backup and Recovery

A **snapshot** is an incremental, point-in-time backup of a persistent disk, stored independently of the disk. Only changed blocks are stored after the first snapshot, keeping ongoing storage costs low.

```
gcloud compute disks snapshot web-server-1 \
  --zone=us-central1-a \
  --snapshot-names=web-server-1-snap-20260821
```

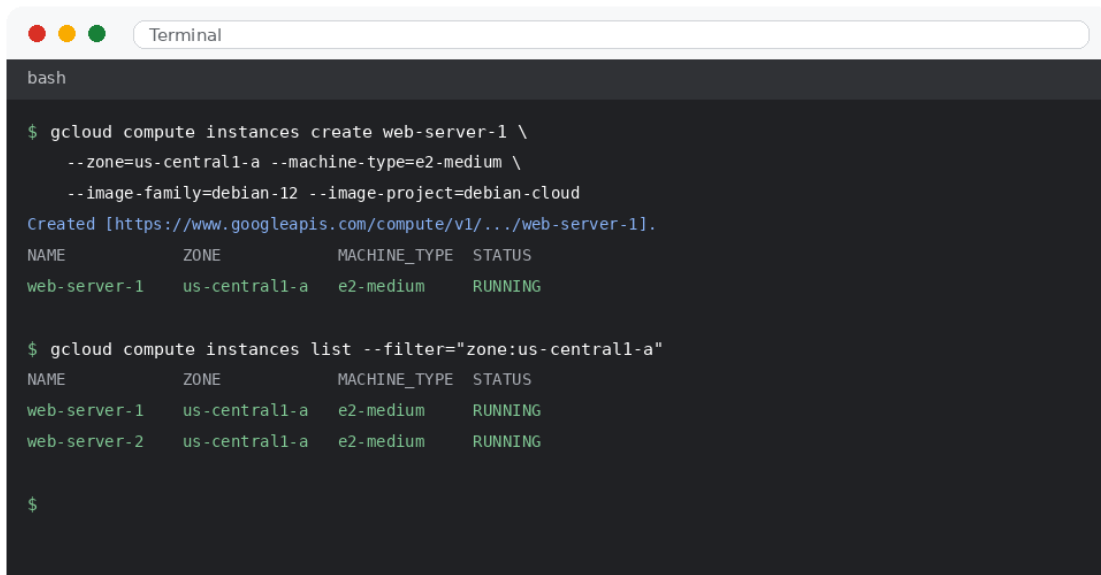
- `disks snapshot` — creates a snapshot without requiring you to stop the instance, though briefly quiescing the application first produces a more consistent backup.
- `--snapshot-names` — an explicit name; including a date makes manual cleanup and identification straightforward.

To restore, create a new disk from the snapshot, then attach it to a new or existing instance:

```
gcloud compute disks create web-server-1-restored \
  --zone=us-central1-a \
  --source-snapshot=web-server-1-snap-20260821

gcloud compute instances create web-server-2 \
  --zone=us-central1-a \
  --machine-type=e2-medium \
  --disk=name=web-server-1-restored,boot=yes
```

- `--source-snapshot` — initializes the new disk's contents from the snapshot rather than an empty disk or public image.
- `--disk=name=...,boot=yes` — attaches the pre-existing disk to the new instance as its boot disk.



```
bash

$ gcloud compute instances create web-server-1 \
  --zone=us-central1-a --machine-type=e2-medium \
  --image-family=debian-12 --image-project=debian-cloud
Created [https://www.googleapis.com/compute/v1/.../web-server-1].
NAME              ZONE             MACHINE_TYPE  STATUS
web-server-1      us-central1-a    e2-medium     RUNNING

$ gcloud compute instances list --filter="zone:us-central1-a"
NAME              ZONE             MACHINE_TYPE  STATUS
web-server-1      us-central1-a    e2-medium     RUNNING
web-server-2      us-central1-a    e2-medium     RUNNING

$
```

Figure 6: Illustrative `gcloud compute instances create` and `list` terminal output.

Images and Image Families

An **image** is a template used to create a disk, most commonly a boot disk. Google provides **public images** for common operating systems — `debian-12`, Ubuntu LTS releases, Windows Server editions — maintained and patched by Google or the OS vendor.

When a VM needs pre-installed software beyond what a startup script reasonably handles each boot, create a **custom image** from a configured VM's disk:

```
gcloud compute images create web-app-image-v1 \
  --source-disk=web-server-1 \
  --source-disk-zone=us-central1-a
```

- `--source-disk` / `--source-disk-zone` — identify the disk to capture; stopping the source VM first avoids capturing a disk mid-write, though it is not required.

An **image family** groups related images and always resolves to the latest non-deprecated image when referenced by `--image-family` instead of `--image` — why Chapter 4 used `--image-family=debian-12`, automatically picking up patches without requiring script updates.

TIP

When building your own custom images, publish new versions into your own image family (for example, `web-app-family`). Instance templates that reference the family pick up the newest image on the next VM creation, while running VMs are unaffected.

Choosing a Machine Type Family

- **General-purpose (E2, N2, N2D)** — balanced vCPU-to-memory ratios suited to most workloads: web servers, small-to-medium databases, development environments. E2 is the most cost-efficient default; N2 and N2D offer higher performance ceilings for workloads that outgrow E2.
- **Compute-optimized (C2)** — high, consistent per-core performance, ideal for CPU-bound work like batch analytics or media transcoding, where single-thread throughput matters more than cost efficiency.
- **Memory-optimized (M2, M3)** — very high memory-to-vCPU ratios, built for in-memory databases (SAP HANA, large Redis deployments) holding very large datasets in RAM.

As a rule of thumb: a small web server fits comfortably on E2; a nightly batch job benefits from C2's dedicated cores; an in-memory database needs M2 or M3's memory headroom.

Changing a Running VM's Machine Type

The instance must be stopped first, since reallocating vCPU and memory requires the hypervisor to reassign underlying physical resources:

```
gcloud compute instances stop web-server-1 --zone=us-central1-a
```

```
gcloud compute instances set-machine-type web-server-1 \
  --zone=us-central1-a \
  --machine-type=e2-standard-4
```

```
gcloud compute instances start web-server-1 --zone=us-central1-a
```

- `set-machine-type` — changes the recorded machine type; it takes effect the next time the instance starts.
- `e2-standard-4` — a larger E2 type (4 vCPUs) than the original `e2-medium`, appropriate if monitoring shows the instance is consistently CPU-constrained.

Troubleshooting and Best Practices

- **Right-size before scaling up.** Check Cloud Monitoring CPU and memory utilization (Chapter 20) before assuming a VM needs a bigger machine type; over-provisioning is one of the most common sources of unnecessary GCP spend.
- **Automate backups with snapshot schedules** rather than manual snapshot commands:

```
gcloud compute resource-policies create snapshot-schedule daily-backup-policy \
  --region=us-central1 \
  --daily-schedule \
  --start-time=03:00 \
  --max-retention-days=14
```

```
gcloud compute disks add-resource-policies web-server-1 \
  --zone=us-central1-a \
```

```
--resource-policies=daily-backup-policy
```

- `--daily-schedule / --start-time=03:00` — runs an automatic snapshot daily at 3:00 AM UTC.
- `--max-retention-days=14` — automatically deletes snapshots older than 14 days, keeping storage costs bounded.
- `add-resource-policies` — attaches the schedule to a disk so it starts taking automated snapshots.
- **Default to pd-balanced** unless a measured workload demonstrates you need pd-ssd's higher IOPS or pd-extreme's provisioned performance; jumping to the most expensive tier "just in case" is a common and avoidable cost mistake.

WARNING

Deleting a disk does not delete its snapshots. Snapshots are stored independently of their source disk specifically so that disk deletion never destroys your backups.

Key Takeaways

- Persistent disk types trade off cost and performance: pd-standard is cheapest for sequential workloads, pd-balanced is the sensible default, pd-ssd suits latency-sensitive databases, and pd-extreme is for the most demanding transactional workloads.
- Disks can be resized larger online, but the guest OS filesystem must be grown separately; machine type changes require the instance to be stopped first.
- Snapshots are incremental, disk-independent backups; restoring means creating a new disk from the snapshot and attaching it to an instance.
- Image families always resolve to the latest non-deprecated image, which is why templates should reference families rather than specific image versions.
- Match machine type family to workload shape — E2/N2/N2D for general-purpose, C2 for CPU-bound batch work, M2/M3 for large in-memory datasets — and automate backups with snapshot schedules.

Chapter 6: Managed Instance Groups & Autoscaling

The previous two chapters worked with individual VMs — creating them one at a time and reasoning about their disks and machine types directly. That works for a single development server, but breaks down once you need a fleet of identical VMs behind a load balancer. Manually creating each instance, keeping configuration in sync, and adding capacity during a traffic spike is operationally painful at scale. **Managed Instance Groups (MIGs)** solve exactly this problem.

The Problem MIGs Solve

Imagine running a web application on three hand-created VMs, as in Chapter 4. If one crashes, someone has to notice and recreate it. If traffic doubles, someone has to manually create more VMs and register them with the load balancer. If you roll out a new version, someone has to update each VM one at a time, hoping not to take the whole fleet down at once. None of this scales past a handful of VMs.

A Managed Instance Group automates all of it: it creates and maintains a specified number of identical instances from a single template, replaces instances that fail health checks, and grows or shrinks the fleet based on real-time load.

Instance Templates: The Blueprint

An **instance template** is an immutable, versioned configuration describing what a VM in the group should look like — machine type, boot image, disks, network settings, service account, startup script. “Immutable” means it cannot be edited after creation; instead, you create a new one, typically with a version suffix, and point the MIG at it.

```
gcloud compute instance-templates create web-template-v1 \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --tags=http-server \
  --service-account=app-sa@my-gcp-project-123456.iam.gserviceaccount.com \
  --scopes=cloud-platform
```

- web-template-v1 — the -v1 suffix is a naming convention (not required syntax) so future revisions become web-template-v2, web-template-v3, making it obvious which version a MIG runs.
- The remaining flags mirror `gcloud compute instances create` exactly — a template captures the same configuration as a single instance, without creating one immediately.

Creating a Managed Instance Group

```
gcloud compute instance-groups managed create web-mig \
  --template=web-template-v1 \
  --size=2 \
  --zone=us-central1-a
```

- web-mig — the name of the managed instance group.
- --template=web-template-v1 — the blueprint every instance is created from.
- --size=2 — the target instance count; Compute Engine immediately creates two VMs with autogenerated name suffixes.
- --zone=us-central1-a — creates a **zonal** MIG. Regional MIGs, covered later, are usually the better choice for production.

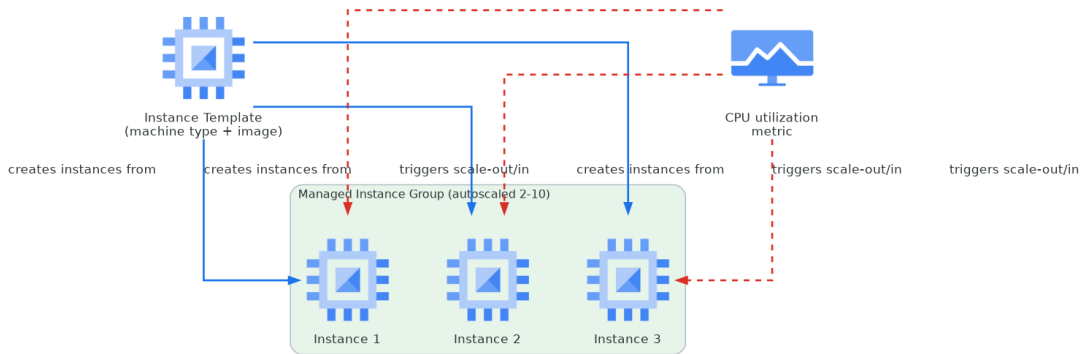


Figure 7: An instance template defines the blueprint; the managed instance group creates instances from it, and the autoscaler adds or removes them based on load.

Autohealing

A MIG can use a health check to continuously verify each instance is actually working, not just powered on. If an instance fails its health check for a sustained period, the MIG deletes and recreates it from the same template — this is **autohealing**. A VM can report as **RUNNING** while the application inside has crashed or hung, and only an application-level check (an HTTP request to a specific path) catches that.

```
gcloud compute health-checks create http web-health-check \
  --port=80 \
  --request-path=/healthz

gcloud compute instance-groups managed update web-mig \
  --zone=us-central1-a \
  --health-check=web-health-check \
  --initial-delay=300
```

- `--request-path=/healthz` — the path the health check calls; your application should return 200 OK here only when genuinely ready.
- `--initial-delay=300` — gives a new instance 300 seconds to boot before autohealing starts evaluating it, avoiding killing slow-starting instances prematurely.

Autoscaling

Autoscaling lets the MIG grow or shrink automatically in response to real load rather than a fixed size:

```
gcloud compute instance-groups managed set-autoscaling web-mig \
  --zone=us-central1-a \
  --max-num-replicas=10 \
  --min-num-replicas=2 \
  --target-cpu-utilization=0.6
```

- `--min-num-replicas=2` / `--max-num-replicas=10` — the floor and ceiling of the group's size; the autoscaler never goes below 2 (baseline availability) or above 10 (cost and blast-radius ceiling).
- `--target-cpu-utilization=0.6` — the autoscaler compares average CPU utilization across the group against this 60 percent target and adds or removes instances to move toward it.

The autoscaler runs a feedback loop: it samples CPU utilization (or another metric, such as load

balancing capacity or a custom Cloud Monitoring metric) every few seconds, adding instances when the average drifts above target (subject to the max ceiling and a cooldown that prevents thrashing) and removing them when it drops well below target (subject to the min floor). Scaling down is intentionally more conservative than scaling up, to avoid cutting capacity right before another spike.

TIP

Target 60-70 percent CPU utilization rather than 90-plus. Headroom gives the autoscaler time to react before existing instances become overloaded, since new instances take time to boot and register healthy.

Rolling Updates

When you publish a new template (for example, `web-template-v2` with a patched OS image), you do not need to delete and recreate every instance manually. A **rolling update** replaces instances gradually:

```
gcloud compute instance-groups managed rolling-action start-update web-mig \
  --zone=us-central1-a \
  --version=template=web-template-v2 \
  --max-unavailable=1 \
  --max-surge=1
```

- `--version=template=web-template-v2` — the new template to roll out.
- `--max-unavailable=1` / `--max-surge=1` — control pace and safety: at most one instance is taken down at a time, and at most one extra instance beyond target size is created temporarily to keep capacity steady.

Regional vs. Zonal MIGs

A **zonal MIG**, like `web-mig` above, places every instance in a single zone. A **regional MIG** spreads instances across multiple zones within a region automatically, so a zone-wide outage takes down only a fraction of your fleet.

BEST PRACTICE

Use regional MIGs for stateless production workloads. The added resilience costs essentially nothing extra — specify `--region` instead of `--zone`, and Compute Engine distributes instances across zones for you.

Troubleshooting and Best Practices

- **Autoscaler will not scale below the minimum.** By design — `--min-num-replicas` is a hard floor. For predictable low-traffic windows, use a scheduled autoscaling policy that lowers the minimum instead of fighting the floor.
- **Instances constantly cycle (created, unhealthy, deleted, recreated).** Almost always a health check misconfiguration — wrong path, wrong port, or too short an initial delay.
- **Rolling update appears stuck.** A low `--max-unavailable` combined with slow-to-pass health checks can look frozen; check `gcloud compute instance-groups managed list-instances web-mig --zone=us-central1-a` for per-instance state.
- Keep templates immutable and versioned by name — this makes rollbacks trivial by pointing the MIG back at the previous version.

Key Takeaways

- Managed Instance Groups automate creation, healing, and scaling of a fleet of identical VMs built from a single instance template.
- Instance templates are immutable and should be versioned by name rather than edited in place.

- Autohealing uses application-level health checks to replace instances that are running but not serving traffic correctly.
- Autoscaling policies, such as targeting 60 percent CPU utilization, let the group grow and shrink within a defined min/max replica range.
- Prefer regional MIGs over zonal MIGs for production workloads for higher availability at negligible extra effort.

Chapter 7: VPC Networking Fundamentals

What Is a VPC on GCP?

A **Virtual Private Cloud (VPC)** is a private, software-defined network that isolates your resources from everyone else's on GCP. It provides the same basic building blocks you would expect from a physical network — IP address ranges, routing, and firewalls — without racking a single switch or router. Every Compute Engine virtual machine, every Google Kubernetes Engine (GKE) cluster, and many other GCP resources attach to a VPC so they can talk to each other and, if allowed, to the internet.

If you have used AWS, one difference is worth understanding early: **a GCP VPC is global, and its subnets are regional — not zonal**. An AWS VPC is confined to a single region, typically requiring a separate VPC per region, connected by peering or a transit gateway. A GCP VPC can contain subnets in every region at once, with no peering needed for resources in different regions of the same VPC to reach each other over Google's private backbone — making multi-region designs noticeably simpler to wire up.

NOTE

"Regional" means a subnet's IP range is reserved within one region (for example, `us-central1`), and any VM in any zone of that region can use an address from it — a subnet is not tied to a single zone.

Auto-Mode vs. Custom-Mode VPCs

When you create a VPC, you choose one of two modes:

- **Auto-mode** networks create one subnet per GCP region automatically, using predefined non-overlapping IP ranges — convenient for learning, since you get a working network with zero planning.
- **Custom-mode** networks start with no subnets. You create exactly the subnets you need, in the regions you need, with IP ranges you choose deliberately.

Auto-mode has a real cost: you inherit whatever ranges Google picked, in every region whether you use it or not, and those ranges may later collide with one you want elsewhere — an on-premises network or a peered VPC, for example. For any project beyond a quick test, custom-mode is recommended, since it forces you to plan addressing up front rather than discovering a conflict later.

BEST PRACTICE

Default to custom-mode VPCs in every real project, even a small one — the extra step of creating a subnet explicitly is minor next to untangling an IP range collision months later.

Creating a Custom-Mode VPC and Subnet

The following two commands create a custom-mode VPC and a single regional subnet inside it — the foundation nearly every other chapter in this book builds on.

```
gcloud compute networks create my-vpc \
  --subnet-mode=custom

gcloud compute networks subnets create my-subnet-central \
  --network=my-vpc \
  --region=us-central1 \
  --range=10.0.0.0/24
```

- `gcloud compute networks create my-vpc` creates the VPC itself, a global container for subnets,

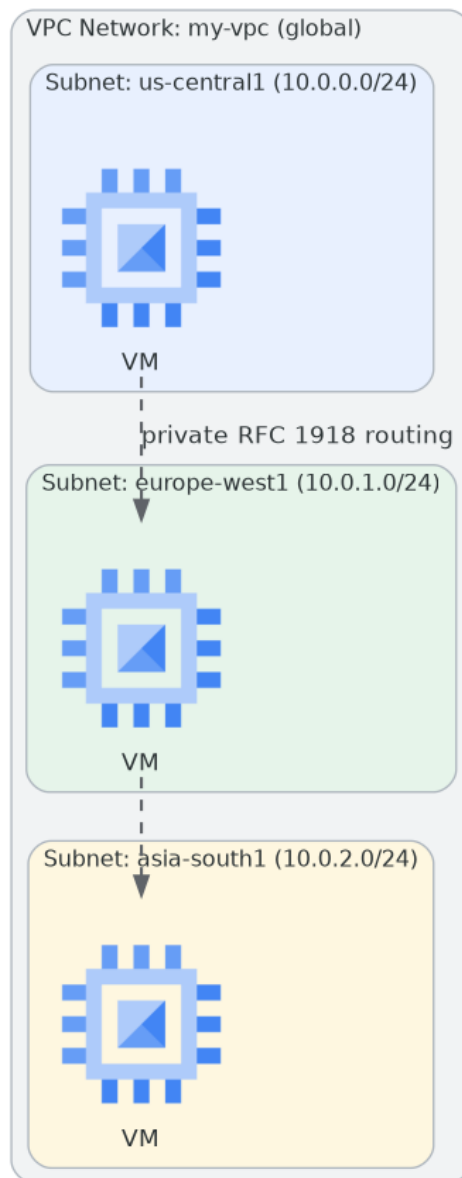


Figure 8: A single GCP VPC network spans every region; each region gets its own regional subnet with a private IP range.

routes, and firewall rules.

- `--subnet-mode=custom` tells GCP not to auto-generate subnets — you create them yourself.
- `gcloud compute networks subnets create my-subnet-central` creates a subnet named `my-subnet-central` inside `my-vpc`.
- `--region=us-central1` scopes the subnet to a single region; VMs in any zone within it (`us-central1-a`, `us-central1-b`, and so on) can use its addresses.
- `--range=10.0.0.0/24` reserves the primary IP range — 256 addresses, a handful reserved automatically for internal use.

You can add more subnets to the same VPC at any time, for example a second one in `eu-west1`, using the same command with a different `--region` and a non-overlapping `--range`.

Primary and Secondary IP Ranges

Every subnet has one **primary IP range**, used for the normal internal addresses assigned to VM network interfaces, as shown above. A subnet can also be given one or more **secondary IP ranges** — additional, separately tracked address pools attached to the same subnet.

Secondary ranges are most commonly used for **alias IP ranges**, which let a VM — or, more importantly, a GKE node — claim a block of extra addresses beyond its own primary one. GKE uses this to give every Pod its own routable IP without consuming a full subnet's worth of primary-range space per node. This is covered in depth in the companion guide, *Docker, Kubernetes, GKE & GCP Artifact Registry* — here, it is enough to know secondary ranges exist and matter when planning CIDR space for a subnet that will host a GKE cluster.

Routes: How Traffic Finds Its Way

A **route** tells the network where to send packets destined for a given IP range. GCP creates two kinds automatically, so you rarely need to add routes by hand for basic connectivity:

- A **system-generated subnet route** for every subnet, so any resource in the VPC knows how to reach any subnet in the same VPC, regardless of region.
- An **implied default route** (`0.0.0.0/0`) sending unmatched traffic toward the internet gateway, letting a VM with an external IP reach the public internet.

Custom static routes can be added later — for example, to send traffic through a virtual appliance or VPN tunnel — but for a straightforward VPC, the automatic routes are all you need.

Connecting and Sharing Networks: Peering and Shared VPC

Two mechanisms let separate VPCs, or separate projects, work together without merging into a single network.

VPC Peering connects two distinct VPCs — possibly belonging to different projects or organizations — so resources in each can reach the other using internal IP addresses, as if on the same network. Each side keeps its own firewall rules and routes, and peering does not transitively extend (if VPC A peers with B, and B with C, A cannot reach C without its own direct peering connection).

Shared VPC takes the opposite approach: one project (the “host project”) owns the VPC and its subnets, and other projects (the “service projects”) attach to it so their resources use those subnets directly. This suits larger organizations that want centralized network administration while application teams keep their own projects for billing and IAM.

Troubleshooting: “Why Can’t My Subnets Talk to Each Other?”

A common misconception among engineers new to GCP is that traffic between two subnets in the same VPC needs an explicit route configured, as in some on-premises designs. It does not. GCP creates a subnet route for every subnet automatically, so **any two VMs in the same VPC can reach each other by default, across subnets and regions, using internal IP addresses** — only a firewall rule can block that traffic. If a connection between two VMs in the same VPC is failing,

check firewall rules (the next chapter's topic) before looking for a missing route.

TIP

`gcloud compute routes list` shows every route in a VPC, including automatically generated ones — a fast way to confirm the routes you expect actually exist before looking elsewhere for the problem.

Best Practices

- **Plan your CIDR ranges before creating any subnet.** A primary range can sometimes be expanded later, but it cannot be shrunk — poor initial planning is a common source of costly rework.
- **Use custom-mode VPCs for anything beyond a quick experiment,** so you control which regions have subnets and which ranges they use.
- **Reserve non-overlapping ranges across VPCs you might ever want to peer,** even with no peering plans today — retrofitting addressing onto a production VPC is far more disruptive than planning it up front.

Key Takeaways

- A GCP VPC is global by default; its subnets are regional, not zonal, letting a single VPC span every region without peering or a transit gateway — a key difference from AWS.
- Auto-mode VPCs suit learning; custom-mode VPCs give deliberate control over CIDR ranges and are the right default for any real project.
- `gcloud compute networks create --subnet-mode=custom` followed by `gcloud compute networks subnets create` is the standard pattern for building a VPC and its subnets.
- Secondary IP ranges enable alias IPs, most notably for GKE Pod addressing, covered further in the companion Kubernetes guide.
- VMs in the same VPC reach each other across subnets and regions by default via automatic routes; only firewall rules can block that traffic.

Chapter 8: Firewall Rules, Cloud NAT & Private Connectivity

How GCP Firewalls Work

A **firewall rule** in GCP controls which network traffic is allowed to or from resources in a VPC. Three properties matter most, since they differ from what many engineers expect from on-premises networking or other clouds:

- **Rules apply at the VPC level, not the subnet level.** You attach a rule to a network (my-vpc), then scope it down to specific instances using targets (covered below).
- **Rules are stateful.** If a rule allows an inbound connection, return traffic for that same connection is automatically allowed back out, without a matching egress rule.
- **The default posture is deny-by-default for ingress, allow-by-default for egress,** via implied rules described below.

Anatomy of a Firewall Rule

Every firewall rule is built from the same handful of properties:

- **Direction** — INGRESS (incoming) or EGRESS (outgoing).
- **Priority** — 0 to 65535; lower numbers take precedence when multiple rules could match the same traffic.
- **Target** — which resources the rule applies to, by **network tag** (a label attached to instances, such as http-server) or **service account**. Targeting by tag or service account is strongly preferred over individual instance names, since it scales naturally as instances come and go.
- **Source/destination ranges** — which IP ranges the rule matches (source for ingress, destination for egress).
- **Protocol and port** — for example tcp:80, tcp:443, or icmp.
- **Action** — ALLOW or DENY.

A Runnable Example: Allowing Web Traffic

Suppose web-server-1 and web-server-2 need to accept HTTP and HTTPS traffic from anywhere. Tag the instances with http-server when creating them (or afterward with `gcloud compute instances add-tags`), then create a firewall rule targeting that tag:

```
gcloud compute firewall-rules create allow-http \
  --network=my-vpc \
  --direction=INGRESS \
  --action=ALLOW \
  --rules=tcp:80,tcp:443 \
  --target-tags=http-server \
  --source-ranges=0.0.0.0/0
```

- `--network=my-vpc` attaches this rule to the VPC created in Chapter 7.
- `--direction=INGRESS` means this rule governs incoming traffic.
- `--rules=tcp:80,tcp:443` allows plain HTTP (port 80) and HTTPS (port 443).
- `--target-tags=http-server` scopes the rule to instances carrying the http-server tag, not every instance in the VPC.
- `--source-ranges=0.0.0.0/0` allows traffic from any IP — appropriate for a public website, far too permissive for anything that shouldn't be internet-facing.

Compare that with SSH access, which should almost never be open to the entire internet:

```
gcloud compute firewall-rules create allow-ssh-from-office \
  --network=my-vpc \
  --direction=INGRESS \
  --action=ALLOW \
  --rules=tcp:22 \
```

```
--target-tags=ssh-access \  
--source-ranges=203.0.113.0/24
```

- `--source-ranges=203.0.113.0/24` restricts SSH (port 22) to a single known range — for example, an office's public IP block — instead of `0.0.0.0/0`. Anyone outside that range cannot even attempt to connect.

WARNING

It is easy to copy-paste `--source-ranges=0.0.0.0/0` out of habit. Before applying a rule, ask whether traffic genuinely needs to come from anywhere on the internet, or from a much smaller, known range. SSH, RDP, and database ports are the most common accidental over-exposures.

Implied Rules

Every VPC comes with two rules you did not create and cannot delete, at the lowest possible priority (65535): an **implied deny-all-ingress rule** (unmatched incoming traffic is dropped), and an **implied allow-all-egress rule** (outgoing traffic is allowed unless an explicit DENY egress rule restricts it). This is why you must explicitly allow inbound traffic like HTTP, HTTPS, or SSH, while outbound internet access generally works without extra configuration, as long as the VM has a way to reach the internet in the first place.

Cloud NAT: Outbound Internet Access Without an External IP

Giving every VM a public (external) IP address increases its attack surface for no good reason if it only ever needs outbound connections — OS updates, dependency downloads, calling an external API. **Cloud NAT** (Network Address Translation) solves this: it lets VMs with no external IP initiate outbound internet connections, translating their internal IP to a shared external IP behind the scenes, while still blocking unsolicited inbound connections.



Figure 9: A private VM with no external IP reaches the internet outbound only, through an allow firewall rule and a Cloud NAT gateway.

Cloud NAT is configured through two resources: a **Cloud Router**, handling the dynamic route advertisement Cloud NAT relies on, and the **NAT gateway** itself.

```
gcloud compute routers create my-router \  
  --network=my-vpc \  
  --region=us-central1  
  
gcloud compute routers nats create my-nat \  
  --router=my-router \  
  --region=us-central1 \  
  --auto-allocate-nat-external-ips \  
  --nat-all-subnet-ip-ranges
```

- `gcloud compute routers create my-router` creates a Cloud Router in `us-central1`, scoped to `my-vpc`. A Cloud Router is regional and required infrastructure for Cloud NAT, even though you won't manage BGP routes directly here.
- `gcloud compute routers nats create my-nat` creates the actual NAT gateway, attached to `my-router`.
- `--auto-allocate-nat-external-ips` lets Google manage the pool of external IPs used for

translation, rather than you reserving static IPs yourself.

- `--nat-all-subnet-ip-ranges` applies NAT to every subnet's IP ranges in this region, so any VM without its own external IP can reach the internet through this gateway.

With this in place, `web-server-1` and `web-server-2` can be created with no external IP at all, reducing exposure, while still running `apt-get update` or reaching an external package registry.

BEST PRACTICE

Prefer Cloud NAT over assigning external IPs to every VM — it gives the same outbound connectivity with a smaller attack surface, and centralizes outbound IP management.

Private Google Access

A related concept: **Private Google Access** lets VMs with no external IP still reach Google APIs — Cloud Storage, BigQuery, and so on — over Google's internal network path rather than the public internet. It is enabled per-subnet with a single flag (`--enable-private-ip-google-access` on `gcloud compute networks subnets update`), and is commonly turned on alongside Cloud NAT so a VM can call Google's APIs directly while reaching the broader internet through the NAT gateway.

Troubleshooting

- **SSH connection times out.** Usually no firewall rule allows TCP port 22 from your current IP — the implied deny-all-ingress rule blocks it by default. Check with `gcloud compute firewall-rules list --filter="network=my-vpc"` and confirm a rule targets the right tag or service account from a source range including your actual IP.
- **"It worked from my other VM, but not this one."** Rules apply across the entire VPC but only to instances matching their target. If two VMs behave differently, check whether they carry the same network tags, not whether the rule set differs.
- **VM has no external IP and cannot reach the internet at all.** Confirm a Cloud NAT gateway exists in the VM's region and covers its subnet.

Best Practices

- Restrict `--source-ranges` to the smallest range that meets the actual need, rather than defaulting to `0.0.0.0/0`.
- Use network tags or service accounts as firewall targets instead of individual instance names, so rules automatically apply to new instances sharing the same role.
- Prefer Cloud NAT for outbound-only connectivity over assigning external IPs to every VM.

Key Takeaways

- GCP firewall rules are stateful, apply at the VPC level, and are deny-by-default for ingress while allowing egress by default, via implied rules at the lowest priority.
- A firewall rule's target should almost always be a network tag or service account, not an instance name, so it scales with your fleet.
- Cloud NAT, built on a Cloud Router, gives VMs with no external IP outbound internet connections without exposing them to inbound traffic.
- Private Google Access lets VMs without external IPs reach Google APIs directly, and is commonly enabled alongside Cloud NAT.
- The most common GCP networking issue is a missing ingress rule for the port and source expected — check firewall rules before assuming a routing problem.

Chapter 9: Load Balancing on GCP

One Product, Many Front Ends

GCP's load balancing is often described as “one product, many front ends,” because underneath a handful of different-looking configurations sits a shared set of building blocks — backend services, health checks, and Google's global network. What changes is where traffic enters, what protocol layer it operates at, and how far it reaches. Rather than one “load balancer” product, you choose the front end that matches your traffic:

Load balancer type	Layer	Scope	Typical use case
Global external Application Load Balancer	Layer 7 (HTTP/HTTPS)	Global, one anycast IP, routes to the nearest healthy region	Public-facing websites and APIs
Global external proxy Network Load Balancer	Layer 4 (TCP/SSL)	Global, one anycast IP	Non-HTTP TCP/SSL traffic that still needs a global entry point
Regional external Network Load Balancer	Layer 4, pass-through	Single region	Simple TCP/UDP load balancing where you don't need Google's proxying or global reach
Internal Load Balancer	Layer 4 or Layer 7	Private, internal-only	Traffic that should never leave your VPC, such as a backend microservice tier

“Layer 7” and “Layer 4” refer to the OSI networking model: Layer 7 (application layer) understands HTTP semantics like URL paths and headers, while Layer 4 (transport layer) only understands TCP or UDP connections without looking inside them. An **anycast IP** is a single IP address simultaneously advertised from many locations worldwide; Google's network routes a request to whichever advertised location is closest, letting one IP for a global load balancer still deliver low-latency connections everywhere.

For most public-facing HTTP or HTTPS services, the global external Application Load Balancer is the right starting point, and the one this chapter builds end to end.

TIP

If you are unsure which type to reach for, ask two questions: does the traffic speak HTTP(S), and does it need to be reachable from the public internet? “Yes” to both almost always points to the global external Application Load Balancer — the other types exist for the exceptions.

Backend Services and Health Checks: The Shared Building Blocks

Two concepts sit behind every Application Load Balancer configuration:

- A **health check** periodically probes each backend instance (for example, an HTTP request to a specific port and path) to decide whether it is healthy enough to receive traffic. An instance failing enough consecutive checks is marked unhealthy and temporarily removed from rotation automatically.
- A **backend service** ties one or more backends (typically managed instance groups) together with a health check, defines how traffic is balanced across them, and optionally enables features like Cloud CDN (Content Delivery Network) for caching.

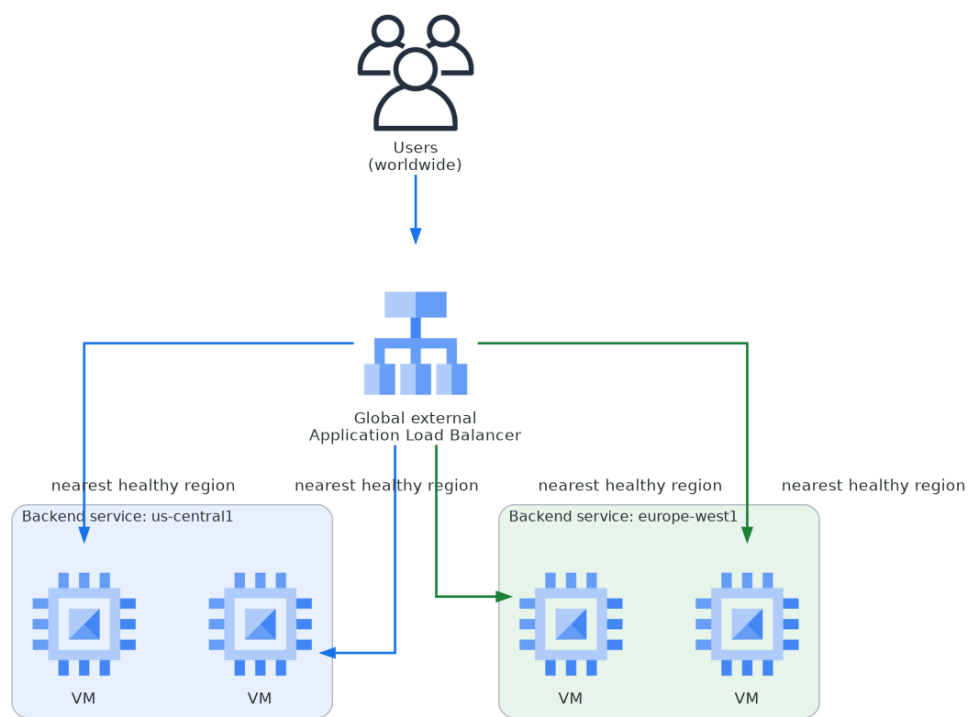


Figure 10: A global external Application Load Balancer routes users to the nearest healthy backend service across multiple regions.

A Runnable Example: Global HTTP(S) Load Balancer in Front of web-mig

This example builds a complete global external Application Load Balancer in front of the web-mig managed instance group introduced in Chapter 6, running in zone us-central1-a.

Step 1 — Create a health check:

```
gcloud compute health-checks create http web-health-check \
  --port=80
```

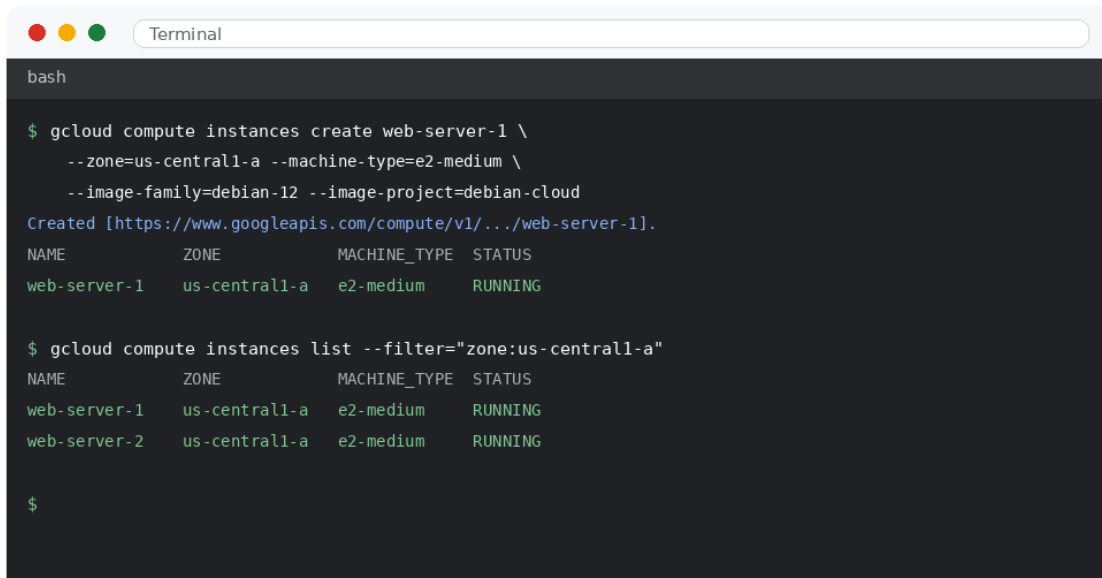
- `gcloud compute health-checks create http` creates an HTTP-based health check named `web-health-check`.
- `--port=80` sends probe requests to port 80, the same port web-mig's instances serve HTTP traffic on.

Step 2 — Create a backend service and attach the managed instance group:

```
gcloud compute backend-services create web-backend \
  --protocol=HTTP \
  --health-checks=web-health-check \
  --global

gcloud compute backend-services add-backend web-backend \
  --instance-group=web-mig \
  --instance-group-zone=us-central1-a \
  --global
```

- `gcloud compute backend-services create web-backend --global` creates a global backend service tied to `web-health-check`, since a global Application Load Balancer requires a global backend service.
- `gcloud compute backend-services add-backend` attaches `web-mig` as a backend, so traffic sent to `web-backend` is distributed across every healthy instance in that managed instance group.
- `--instance-group-zone=us-central1-a` identifies where the zonal managed instance group lives (a regional MIG would use `--instance-group-region` instead).



```
Terminal

bash

$ gcloud compute instances create web-server-1 \
  --zone=us-central1-a --machine-type=e2-medium \
  --image-family=debian-12 --image-project=debian-cloud
Created [https://www.googleapis.com/compute/v1/.../web-server-1].
NAME          ZONE          MACHINE_TYPE  STATUS
web-server-1  us-central1-a e2-medium     RUNNING

$ gcloud compute instances list --filter="zone:us-central1-a"
NAME          ZONE          MACHINE_TYPE  STATUS
web-server-1  us-central1-a e2-medium     RUNNING
web-server-2  us-central1-a e2-medium     RUNNING

$
```

Figure 11: Illustrative `gcloud describe` output for a backend service and its health status.

Running `gcloud compute backend-services get-health web-backend --global` shows the health state Google Cloud has determined for each instance in `web-mig` — `HEALTHY` or `UNHEALTHY` — the

fastest way to confirm the load balancer can actually reach your instances before sending real traffic.

Step 3 — Create the URL map, target HTTP proxy, and forwarding rule:

```
gcloud compute url-maps create web-url-map \
  --default-service=web-backend

gcloud compute target-http-proxies create web-http-proxy \
  --url-map=web-url-map

gcloud compute forwarding-rules create web-forwarding-rule \
  --global \
  --target-http-proxy=web-http-proxy \
  --ports=80
```

- `gcloud compute url-maps create` creates a **URL map**, deciding which backend service handles a request based on host and path — here every request goes to `web-backend` as the default service, but a URL map can route `/api/*` to one backend and `/images/*` to another.
- `gcloud compute target-http-proxies create` creates a **target HTTP proxy**, which takes the URL map and terminates incoming HTTP connections at Google's edge (an HTTPS deployment would use a target HTTPS proxy with an SSL certificate instead).
- `gcloud compute forwarding-rules create --global` creates the **global forwarding rule**, provisioning the actual anycast IP and port users connect to, and handing matching traffic to the target proxy.

Once all four pieces exist, requests arriving at the load balancer's external IP are routed by the URL map to `web-backend`, which forwards them to whichever healthy instance in `web-mig` is best positioned to serve the request, factoring in both proximity and each backend's current load.

How the Load Balancer Picks a Backend

Google's network first routes the request to the point of presence closest to the user, then the load balancer selects a healthy backend from the nearest region that has one available, factoring in each backend's current utilization so load spreads evenly rather than always favoring the single closest instance. If every backend in the nearest region is unhealthy or overloaded, the load balancer fails over to the next-closest region automatically — one of the main reasons to use a global load balancer for a service running in more than one region.

Troubleshooting: The UNHEALTHY Backend Trap

WARNING

By far the most common mistake when setting up a GCP load balancer is forgetting that Google's health-check probes come from specific IP ranges — `130.211.0.0/22` and `35.191.0.0/16` — and a firewall rule allowing office or user traffic does **not** automatically allow those ranges through. If a backend service shows every instance as UNHEALTHY even though the application works fine directly, this is almost always the cause.

The fix is a dedicated firewall rule permitting traffic from those ranges to the health-check port:

```
gcloud compute firewall-rules create allow-health-checks \
  --network=my-vpc \
  --direction=INGRESS \
  --action=ALLOW \
  --rules=tcp:80 \
  --source-ranges=130.211.0.0/22,35.191.0.0/16 \
  --target-tags=http-server
```

- `--source-ranges=130.211.0.0/22,35.191.0.0/16` explicitly allows both ranges Google uses to originate health-check probes.
- `--target-tags=http-server` scopes this rule to the same instances the `allow-http` rule from Chapter 8 already targets, so both user and health-check traffic are permitted.

Beyond the firewall issue, two other checks help early: confirm the application actually listens on the probed port (a port mismatch produces the same symptom), and confirm the instance group's instances are **RUNNING** — an instance stuck in a crash loop will never pass a health check regardless of the firewall.

Best Practices

- Always put a managed instance group behind a backend service rather than individual VMs, so autohealing (Chapter 6) can replace unhealthy instances without manual intervention or load balancer reconfiguration.
- Enable Cloud CDN on the backend service for cacheable content (static assets, images) to reduce latency and backend load.
- Default to the global external Application Load Balancer for internet-facing HTTP(S) services — its single anycast IP, nearest-region routing, and native Cloud CDN and Cloud Armor integration make it the strongest general-purpose choice.

Key Takeaways

- GCP load balancing is one system with several front ends — pick based on protocol layer (Layer 4 vs. Layer 7) and scope (global vs. regional, external vs. internal).
- Backend services and health checks are the shared building blocks every Application Load Balancer is built from.
- A full HTTP(S) load balancer setup requires a health check, a backend service (with the MIG attached as a backend), a URL map, a target HTTP proxy, and a global forwarding rule.
- The single most common GCP load-balancing mistake is forgetting to allow Google's health-check IP ranges (130.211.0.0/22 and 35.191.0.0/16) through the firewall, showing up as every backend marked **UNHEALTHY**.
- Prefer managed instance groups as backends, enable Cloud CDN where cacheable, and default to the global Application Load Balancer for public HTTP(S) services.

Chapter 10: Cloud DNS & Hybrid Connectivity

Every application eventually needs a reliable way for names to resolve to addresses, and a reliable way for your cloud network to talk to whatever still lives outside it — an on-premises data center or another office. Cloud DNS handles the first; Cloud VPN and Interconnect handle the second.

What Cloud DNS Is

Cloud DNS is Google's managed Domain Name System (DNS) service. DNS is the internet's directory lookup: it translates human-readable names like `www.example.com` into the IP (Internet Protocol) addresses computers actually use to route traffic. Instead of running and patching your own DNS servers, Cloud DNS does it for you, backed by Google's global network with a published availability SLA (Service Level Agreement).

Records live inside **managed zones**, which are one of two types:

- **Public zones** are resolvable from anywhere on the public internet — used when you own a domain and want the world to resolve it.
- **Private zones** are resolvable only from the VPC (Virtual Private Cloud) networks you explicitly associate with them. They are the standard way to do internal service discovery — giving services friendly names like `db.internal` instead of hardcoding IP addresses.

Creating a Public Zone and an A Record

```
gcloud dns managed-zones create example-zone \
  --dns-name=example.com. \
  --description="Public zone for example.com"

gcloud dns record-sets create www.example.com. \
  --zone=example-zone \
  --type=A \
  --ttl=300 \
  --rrdatas=203.0.113.10
```

- `example-zone` — the zone's name within your GCP project; a management label, not part of the DNS hierarchy itself.
- `--dns-name=example.com.` — the domain this zone is authoritative for; the trailing dot denotes the DNS root, standard notation for a fully qualified domain name.
- `--type=A` and `--rrdatas=203.0.113.10` — an A record maps a name to an IPv4 address; `rrdatas` is the actual resource record data.
- `--ttl=300` — how many seconds other resolvers may cache this answer. Lower means faster propagation of future changes, at the cost of more queries.

Creating the zone does not make `example.com` resolvable publicly by itself — you must also delegate the domain to Cloud DNS's name servers at your registrar (visible via `gcloud dns managed-zones describe example-zone`).

A Private Zone for Internal Service Discovery

```
gcloud dns managed-zones create internal-zone \
  --dns-name=internal. \
  --description="Private zone for internal service discovery" \
  --visibility=private \
  --networks=my-vpc

gcloud dns record-sets create db.internal. \
  --zone=internal-zone \
  --type=A \
```

```
--ttl=300 \  
--rrdatas=10.128.0.5
```

- `--visibility=private` — marks this a private zone, never published to the public DNS hierarchy.
- `--networks=my-vpc` — the crucial piece: a private zone only resolves for VPC networks explicitly listed here.

TIP

Use private zones for internal hosts instead of hardcoding IP addresses in application configuration. IP addresses change when resources are recreated; a DNS name does not have to.

Why Hybrid Connectivity Matters

Most organizations keep some workloads on-premises while migrating others to GCP, and both sides need to reach each other over private IP addresses rather than the public internet. This is **hybrid connectivity**. GCP offers two main paths.

Cloud VPN creates an encrypted IPsec (Internet Protocol Security) tunnel between your on-premises VPN device and a gateway in your VPC, carried over the public internet. Because it rides on existing internet connectivity, a tunnel can be established in hours, supports respectable bandwidth (multiple gigabits per second across tunnels), and encrypts every packet in transit.

Dedicated Interconnect and **Partner Interconnect** provide a direct, private physical circuit into Google's network — Dedicated runs straight into a Google colocation facility, Partner runs through a supported service provider if you cannot reach one directly. Both offer higher bandwidth (10 Gbps or 100 Gbps circuits), lower and more consistent latency, and traffic never touches the public internet. The tradeoff is lead time: provisioning a physical circuit typically takes weeks to months.

Pick Cloud VPN for quick setup, moderate bandwidth needs, or as a cost-effective backup path. Pick Interconnect for sustained, high-throughput hybrid workloads where bandwidth and latency justify the longer lead time and cost. Many organizations run Interconnect as primary with Cloud VPN as automatic failover.

A Conceptual Cloud VPN Tunnel

A full deployment needs a VPN gateway, a peer gateway definition, and a tunnel resource tying them together. The tunnel command looks conceptually like this (the on-premises peer device's own configuration depends entirely on its vendor):

```
gcloud compute vpn-tunnels create tunnel-to-onprem \  
--peer-address=198.51.100.20 \  
--shared-secret=REPLACE_WITH_STRONG_SECRET \  
--target-vpn-gateway=my-vpn-gateway \  
--local-traffic-selector=10.0.0.0/16 \  
--remote-traffic-selector=192.168.0.0/16
```

- `--peer-address` — the public IP of the on-premises VPN device.
- `--shared-secret` — the pre-shared key both ends authenticate with; both sides must use the identical value.
- `--local-traffic-selector` / `--remote-traffic-selector` — the IP ranges considered “inside” each side of the tunnel, which must match the on-premises device's own routing configuration.

WARNING

Both ends of a VPN tunnel must agree exactly on the shared secret and routing. A mismatch does not always produce an obvious error — it often shows as a tunnel reporting “up” while silently

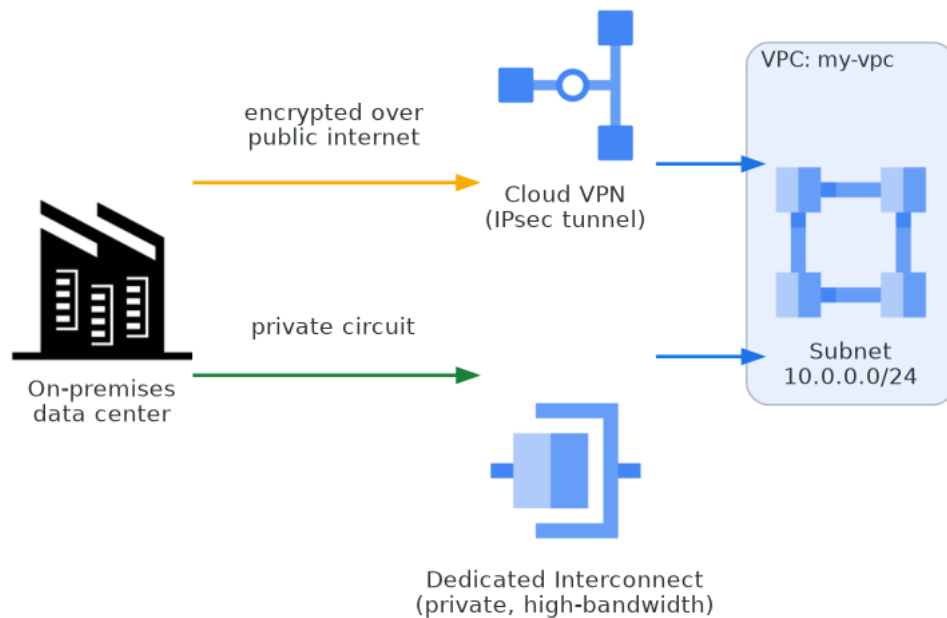


Figure 12: An on-premises data center reaches a VPC either over an encrypted Cloud VPN tunnel or a private Dedicated Interconnect circuit.

dropping traffic.

Troubleshooting

- **DNS changes seem to take forever.** Almost always a TTL issue: resolvers that already cached the old answer keep using it until the previous TTL expires. Lower the TTL well before a planned change.
- **A private zone doesn't resolve from a VM in the VPC.** Check that the zone's --networks list actually includes that VPC. A zone with no (or the wrong) network association simply fails to resolve, with no explicit error.
- **A tunnel shows "up" but no traffic flows.** Verify routing on both sides directs the relevant IP ranges into the tunnel, and that firewall rules permit the expected traffic.

Best Practices

- Run Cloud VPN as a redundant backup path even when Interconnect is primary — a physical circuit can still fail.
- Keep TTLs low during any migration window so rollbacks also propagate quickly.
- Use private DNS zones for internal service names rather than hardcoding IP addresses.

Key Takeaways

- Cloud DNS manages public zones (internet-resolvable) and private zones (resolvable only inside associated VPCs); a private zone must be explicitly associated with the right networks.
- Cloud VPN is a fast, encrypted IPsec tunnel over the public internet, suited to moderate bandwidth or as a backup path.
- Dedicated and Partner Interconnect provide private, high-bandwidth, low-latency circuits at the

- cost of longer lead times.
- A common hybrid pattern runs Interconnect as primary with Cloud VPN as automatic failover.
 - Both ends of a VPN tunnel must match on shared secrets and routing, or it can appear healthy while silently failing.

Chapter 11: Cloud Storage Fundamentals

What Cloud Storage Is

Cloud Storage is Google Cloud’s object storage service — a place to durably store any kind of file (an “object”) at massive scale, accessed over HTTP (Hypertext Transfer Protocol) rather than mounted as a drive. It is worth being precise about the difference between object storage and a traditional filesystem, because it changes how you design around it. A filesystem, like the one attached to a Compute Engine VM (virtual machine), organizes data in a tree of directories and files that support in-place partial edits and POSIX-style permissions. Object storage has no real directory tree — every object sits flatly inside a bucket, identified by a key (its name, which can contain slashes that make it look hierarchical in a console), and objects are replaced wholesale rather than edited in place.

That tradeoff is exactly what makes object storage cheap, durable, and able to scale without you managing any servers. Typical use cases include hosting static website assets (images, CSS, JavaScript), storing backups, staging raw data for a data lake, and receiving application file uploads such as user avatars or exported reports.

Buckets, Names, and Location Types

A **bucket** is the top-level container you create objects inside. Bucket names are globally unique across all of Google Cloud, not just within your project, because a bucket name can form part of a public URL — similar to how domain names must be globally unique. This is the single most common surprise for newcomers.

When you create a bucket, you choose a **location type**, which has direct consequences for latency, availability, and cost:

- **Region** — data lives in a single region, such as `us-central1`. Lowest cost and lowest latency for applications running in that region, but no automatic cross-region replication.
- **Dual-region** — data is stored redundantly across two specific regions, giving higher availability and low-latency access from either, at higher cost.
- **Multi-region** — data is replicated across a broad geographic area (such as “US”), giving the highest availability and serving requests from whichever location is closest, at the highest cost.

NOTE

A bucket’s location is a one-time decision — it cannot be changed afterward. Picking wrong means creating a new bucket in the right location and copying the data over.

Storage Classes

Independent of location, every object also has a **storage class**, which trades storage cost against access cost based on how frequently the data is read:

- **Standard** — the default, best for data accessed frequently or unpredictably. Highest storage cost, no retrieval fee, no minimum storage duration.
- **Nearline** — for data accessed less than once a month, such as recent backups. Lower storage cost, a small per-gigabyte retrieval fee, 30-day minimum storage duration.
- **Coldline** — for data accessed less than once a quarter. Lower cost still, higher retrieval fee, 90-day minimum duration.
- **Archive** — the cheapest class, for data accessed less than once a year, such as regulatory archives. Highest retrieval fee and latency, 365-day minimum duration.

The minimum storage duration matters because Cloud Storage charges an **early deletion fee** if you delete or overwrite an object before its class’s minimum has elapsed — you are billed as though it had been stored for the full minimum period regardless.

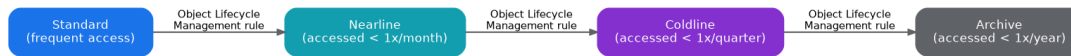


Figure 13: Object Lifecycle Management rules automatically move objects through progressively cheaper, less-frequently-accessed storage classes.

Creating a Bucket, Uploading, and Listing Objects

GCP provides two tools for managing Cloud Storage: `gsutil`, the original tool, and `gcloud storage`, the newer unified command set Google now recommends for new scripts, since it shares configuration with the rest of the `gcloud` CLI (command-line interface) and is generally faster for large transfers. Both remain supported.

Creating a bucket with `gsutil`, and the newer equivalent:

```
gsutil mb -l us-central1 -c standard gs://my-app-assets
```

```
gcloud storage buckets create gs://my-app-assets \
  --location=us-central1 \
  --default-storage-class=standard
```

- `mb / buckets create` — creates a new bucket.
- `-l / --location=us-central1` — sets the bucket’s location.
- `-c / --default-storage-class=standard` — the default class new objects receive if none is specified at upload time.
- `gs://my-app-assets` — the bucket’s address; `my-app-assets` must not already be taken by any Cloud Storage user worldwide.

Uploading a file and listing the bucket’s contents:

```
gcloud storage cp ./logo.png gs://my-app-assets/
```

```
gcloud storage ls gs://my-app-assets
```

- `cp ./logo.png gs://my-app-assets/` — copies the local file into the bucket under the same name; add `-r` to upload whole directories recursively.
- `ls gs://my-app-assets` — lists the bucket’s objects, similar in spirit to a directory listing even though no real directory structure exists underneath.

Object Versioning

Cloud Storage supports **object versioning** at the bucket level. When enabled, overwriting or deleting an object retains the previous data as a noncurrent version you can list, restore, or remove later — a valuable safety net against accidental overwrites, particularly for buckets holding configuration files or anything touched by automated pipelines. Noncurrent versions do continue to incur storage costs until cleaned up, typically via a lifecycle rule (covered in the next chapter).

Troubleshooting

- **“The requested bucket name is not available.”** Bucket names are globally unique across every GCP project. Someone has almost certainly already taken a common name like `assets`. Add a distinguishing prefix, such as your project ID.
- **403 Forbidden when reading or writing objects.** Usually a missing IAM (Identity and Access Management) role, such as `roles/storage.objectViewer` for read access. Check the bucket’s policy with `gcloud storage buckets get-iam-policy gs://my-app-assets`.
- **Uploads succeed but the object isn’t where expected.** Check the destination path — a trailing slash changes whether `cp` treats the destination as a prefix or an exact object name.

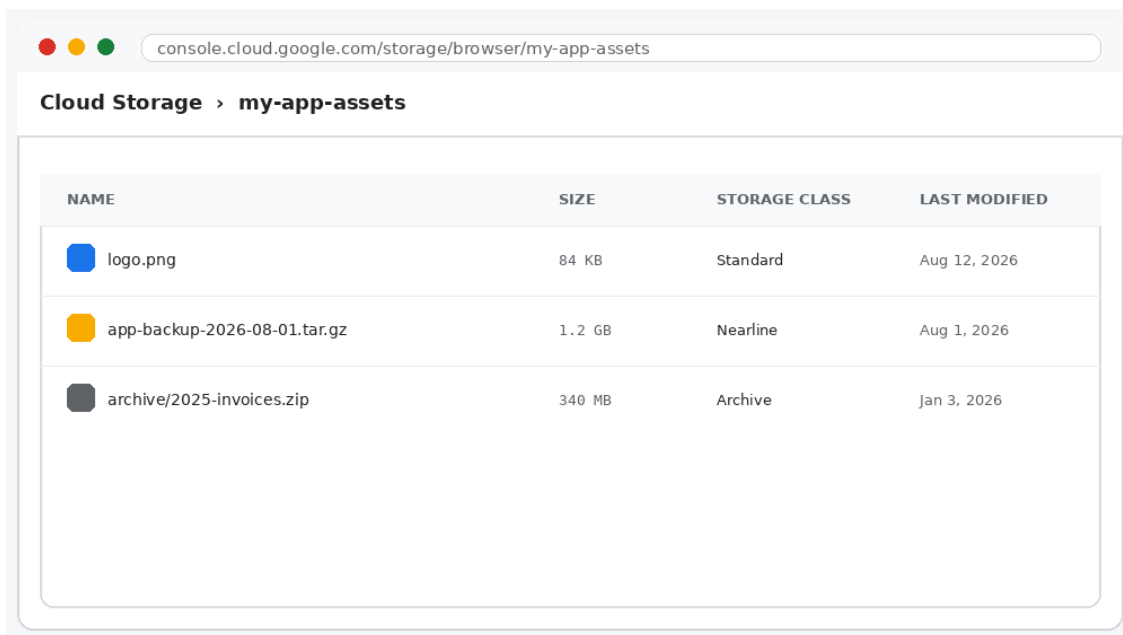


Figure 14: Illustrative a Cloud Storage bucket browser showing objects and storage classes.

Best Practices

- Use the Standard class for anything actively served to users; only move data to Nearline, Coldline, or Archive once you understand its real access pattern.
- Use lifecycle rules to automatically transition older objects to cheaper classes instead of managing this by hand — Chapter 12 covers this in full.
- Avoid making buckets public unless intentionally hosting public content; Chapter 12 covers how to avoid the common mistakes that cause accidental exposure.

Key Takeaways

- Cloud Storage is object storage, not a filesystem — objects sit flatly in globally-unique-named buckets and are replaced wholesale rather than edited in place.
- A bucket's location type (region, dual-region, multi-region) is fixed at creation and trades off latency, availability, and cost.
- The four storage classes trade lower storage cost for higher retrieval cost and longer minimum durations — choose based on real access frequency.
- `gcloud storage` is the newer, recommended unified CLI for Cloud Storage; `gsutil` remains widely used and supported.
- Object versioning protects against accidental overwrites but adds ongoing storage cost until old versions are cleaned up.

Chapter 12: Cloud Storage Security & Lifecycle Management

Chapter 11 covered buckets, storage classes, and basic object mechanics. This chapter goes deeper on what every production bucket needs: controlling who can access what, and automatically managing storage costs and retention instead of doing either by hand.

Access Control Models: Uniform Access vs. Fine-Grained ACLs

Cloud Storage supports two access models. **Fine-grained access** is the legacy model, layering Access Control Lists (ACLs) — permissions on individual objects, separate from IAM (Identity and Access Management) — on top of bucket-level IAM policies. It allows granular control, but at a cost: it is hard to audit and creates two overlapping permission systems that can disagree.

Uniform bucket-level access removes ACLs entirely and controls everything through IAM policies applied at the bucket level, consistently across every object inside. This is the modern recommendation, and new console-created buckets default to it, since one consistent permission system is far easier to audit. If different objects need different permissions, use separate buckets rather than fighting with per-object ACLs.

BEST PRACTICE

Enable uniform bucket-level access on every new bucket unless you have a specific reason to use fine-grained ACLs. Set it at creation with `--uniform-bucket-level-access` on `gcloud storage buckets create`, or enable it later if no object-level ACLs are still relied upon.

Granting Bucket-Level Access

With uniform access enabled, granting a person read access to every object in a bucket is a single IAM binding:

```
gcloud storage buckets add-iam-policy-binding gs://my-app-assets \
  --member="user:ana@example.com" \
  --role="roles/storage.objectViewer"
```

- `add-iam-policy-binding` — adds an entry to the bucket's IAM policy without disturbing existing bindings.
- `--member` — who the permission applies to; also accepts `serviceAccount:...`, `group:...`, or `domain:...`.
- `--role="roles/storage.objectViewer"` — read-only access. Other common roles: `roles/storage.objectCreator` (write-only) and `roles/storage.objectAdmin` (full read/write/delete).

Signed URLs: Temporary Access Without Making Anything Public

Sometimes you need to give someone temporary access to a single private object — a generated invoice, say — without an IAM role or making the object public. A **signed URL** solves this: a URL embedding a cryptographic signature proving authorization, plus an expiration after which it stops working.

```
gcloud storage sign-url gs://my-app-assets/invoice-2026-08.pdf \
  --private-key-file=service-account-key.json \
  --duration=15m
```

- `sign-url` — generates a time-limited, signed HTTP URL for the object.
- `--private-key-file` — a service account private key used to sign the URL; the object's own permissions remain unchanged.
- `--duration=15m` — how long the URL stays valid; afterward, even the original requester is rejected.

Anyone holding a valid signed URL can access the object directly, with no Google account required — the authorization is embedded in the URL itself.

TIP

Prefer signed URLs over making an object public whenever access is meant to be temporary or limited to a specific recipient. A public object is public forever, to everyone, until you notice and revoke it.

Object Lifecycle Management

Object Lifecycle Management lets you define rules that automatically act on objects meeting age or condition criteria — most commonly transitioning them to a cheaper storage class, or deleting them. This is how Chapter 11's storage-class tradeoffs get applied in practice.

A lifecycle policy is JSON (JavaScript Object Notation) applied to a bucket. The following moves objects to Nearline after 30 days, Coldline after 90 days, and deletes them after 365 days:

```
{
  "rule": [
    {
      "action": { "type": "SetStorageClass", "storageClass": "NEARLINE" },
      "condition": { "age": 30 }
    },
    {
      "action": { "type": "SetStorageClass", "storageClass": "COLDLINE" },
      "condition": { "age": 90 }
    },
    {
      "action": { "type": "Delete" },
      "condition": { "age": 365 }
    }
  ]
}
```

Save this as `lifecycle.json` and apply it:

```
gcloud storage buckets update gs://my-app-assets \
  --lifecycle-file=lifecycle.json
```

- Each rule entry pairs an action (what to do) with a condition (when to do it).
- "age": 30 matches any object whose current version has existed 30 days or more; conditions can also match on current storage class or noncurrent-version status.
- `--lifecycle-file` replaces the bucket's entire lifecycle configuration. Rules are evaluated roughly once per day, so actions are not instantaneous.

NOTE

Lifecycle rules interact with the minimum storage duration and early-deletion fees from Chapter 11. The Coldline transition at 90 days and delete at 365 days above are safely spaced beyond Coldline's own 90-day minimum, so objects are never deleted before satisfying their class's minimum duration.

Encryption at Rest

Every object is encrypted at rest by default with keys Google generates and manages — no configuration required. For stricter compliance needs, Cloud Storage also supports **customer-managed encryption keys (CMEK)** through Cloud KMS (Key Management Service), letting you rotate or revoke the key independently of Google's own management. Cloud KMS and CMEK are covered in full

in Chapter 17.

Versioning, Retention Policies, and Bucket Lock

For compliance use cases — financial records, audit logs, legal holds — Cloud Storage offers **retention policies**, which block deletion or overwriting of an object until a minimum period has elapsed, and **Bucket Lock**, which permanently locks that policy so not even a project owner can shorten or remove it. Once locked, the policy can only become stricter, never weaker.

Troubleshooting

- **A bucket is publicly accessible and nobody remembers granting that.** Almost always a role granted to `allUsers` (anyone on the internet) or `allAuthenticatedUsers` (anyone with any Google account). Check with `gcloud storage buckets get-iam-policy gs://my-app-assets`.

WARNING

Granting a role to `allUsers` makes every matching object readable (or writable) by anyone on the internet, no authentication required. This is one of the most common causes of real-world cloud data breaches, and it happens easily — for instance, reusing a command meant for a public static-website bucket against one holding private data. Always double-check the `--member` value before running `add-iam-policy-binding`.

- **Lifecycle rule hasn't run yet.** Actions run roughly once per day, so a just-crossed age threshold may take up to a day to take effect.
- **Signed URL returns 403 Forbidden immediately.** Check that `--duration` has not expired, and that the signing service account still has permission on the object.

Best Practices

- Enable uniform bucket-level access on every new bucket rather than relying on legacy per-object ACLs.
- Never grant a role to `allUsers` or `allAuthenticatedUsers` unless deliberately building public content.
- Use lifecycle rules to manage storage costs automatically instead of manually moving objects between classes.
- Prefer signed URLs over making objects public whenever access is temporary or scoped to a specific recipient.

Key Takeaways

- Uniform bucket-level access, controlled through IAM, is the modern recommended default over legacy fine-grained ACLs.
- Signed URLs grant temporary, scoped access to a private object without making it public.
- Object Lifecycle Management rules automatically transition or delete objects based on age, applying Chapter 11's storage-class tradeoffs without manual effort.
- Objects are encrypted at rest by default; customer-managed keys via Cloud KMS (Chapter 17) are available for stricter control.
- An accidental `allUsers` IAM binding is one of the most common causes of real-world cloud data exposure — always verify the member before granting access.

Chapter 13: Cloud SQL: Managed Relational Databases

Almost every application needs a place to store structured, relational data — customer records, orders, inventory counts. On Google Cloud, the default answer is Cloud SQL: a fully managed database service that runs MySQL, PostgreSQL, or SQL Server for you. “Fully managed” means Google handles patching, backups, and replication, while you design the schema, write queries, and tune indexes.

Why Not Self-Manage on a Compute Engine VM?

You can install PostgreSQL on a Compute Engine virtual machine (VM) yourself, and for unusual configurations Cloud SQL doesn’t support, that may be the right call. For most applications, though, self-managing gives up a lot for little benefit:

- **Patching** becomes your responsibility, and a missed security patch on a database engine is a serious risk.
- **Backups** must be scripted, scheduled, and tested — easy to get wrong, and untested backups are not real backups.
- **High availability (HA)** — keeping the database up even if the host machine fails — requires building failover logic yourself.
- **Replication** for read scaling or disaster recovery is a nontrivial distributed systems problem to build by hand.

Cloud SQL provides all of this as configuration rather than code, at the cost of reduced low-level control over the underlying operating system — a trade worth making for most teams.

Creating a Cloud SQL Instance

```
gcloud sql instances create my-sql-instance \
  --database-version=POSTGRES_15 \
  --tier=db-custom-2-8192 \
  --region=us-central1 \
  --availability-type=REGIONAL
```

- `my-sql-instance` is the instance name, used in every subsequent `gcloud sql` command.
- `--database-version=POSTGRES_15` selects the engine and version (MYSQL_8_0 and SQLSERVER_* versions are also available).
- `--tier=db-custom-2-8192` requests a custom shape with 2 virtual CPUs and 8192 MB of memory. Predefined tiers like `db-f1-micro` suit lightweight dev workloads.
- `--region=us-central1` places the instance regionally; the specific zone (and, with HA, a second zone for the standby) is chosen automatically.
- `--availability-type=REGIONAL` enables high availability, covered below.

Instance creation takes several minutes. Once it exists, create a database and a user:

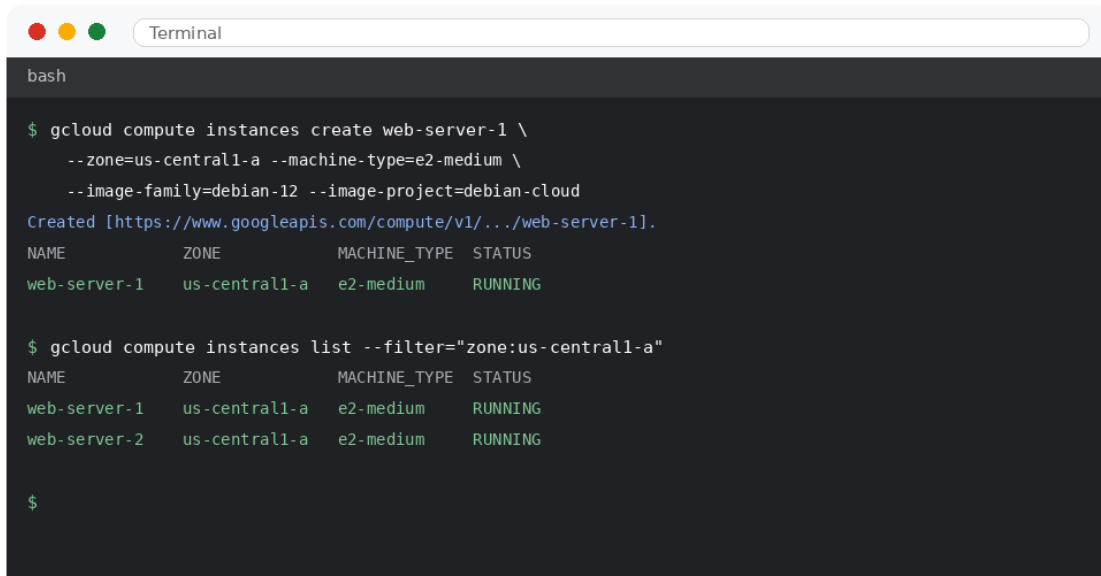
```
gcloud sql databases create appdb --instance=my-sql-instance
```

```
gcloud sql users create appuser \
  --instance=my-sql-instance \
  --password=a-strong-randomly-generated-password
```

- `gcloud sql databases create appdb` creates a logical database inside the instance; one instance can host several databases.
- `gcloud sql users create appuser` creates a database-level user, distinct from IAM users — Cloud SQL also supports IAM database authentication for engines that offer it.

WARNING

Never pass a real password as plain text in a shared terminal, script, or CI/CD log. Use a secret manager or interactive prompt instead.



```
bash

$ gcloud compute instances create web-server-1 \
  --zone=us-central1-a --machine-type=e2-medium \
  --image-family=debian-12 --image-project=debian-cloud
Created [https://www.googleapis.com/compute/v1/.../web-server-1].
NAME          ZONE          MACHINE_TYPE  STATUS
web-server-1  us-central1-a e2-medium     RUNNING

$ gcloud compute instances list --filter="zone:us-central1-a"
NAME          ZONE          MACHINE_TYPE  STATUS
web-server-1  us-central1-a e2-medium     RUNNING
web-server-2  us-central1-a e2-medium     RUNNING

$
```

Figure 15: Illustrative `gcloud sql instances describe` output showing instance state and IP address.

Running `gcloud sql instances describe my-sql-instance` returns the instance's state (RUNNABLE once ready), assigned IP addresses, and configuration — a useful first stop when diagnosing connectivity problems.

Connecting Securely: Auth Proxy vs. Authorized Networks

Authorized networks allowlists specific IP ranges that may attempt to connect (still subject to valid credentials and, ideally, TLS encryption). It's simple, but IP allowlists are brittle in dynamic cloud environments, and hand-configuring TLS certificates is easy to get wrong.

The Cloud SQL Auth Proxy is Google's recommended method for most workloads. It runs alongside your application and authenticates using IAM credentials, establishing an encrypted tunnel automatically — no IP allowlist or manual certificate management required:

```
./cloud-sql-proxy my-gcp-project-123456:us-central1:my-sql-instance
```

- The argument is the instance's connection name, `PROJECT_ID:REGION:INSTANCE_NAME`.
- The proxy listens on a local port and forwards traffic through an encrypted tunnel, so your application connects to `localhost` as if the database were local.

BEST PRACTICE

Prefer the Auth Proxy, or Private IP over your VPC, over exposing a public IP with authorized networks. Both avoid maintaining IP allowlists and reduce the instance's exposure to the public internet.

High Availability: Regional Instances

`--availability-type=REGIONAL` provisions a primary and a standby in two different zones, kept in sync with synchronous replication — a write isn't acknowledged until it's durably recorded on both. If the primary's zone fails, Cloud SQL automatically fails over to the standby, typically within a minute

or two.

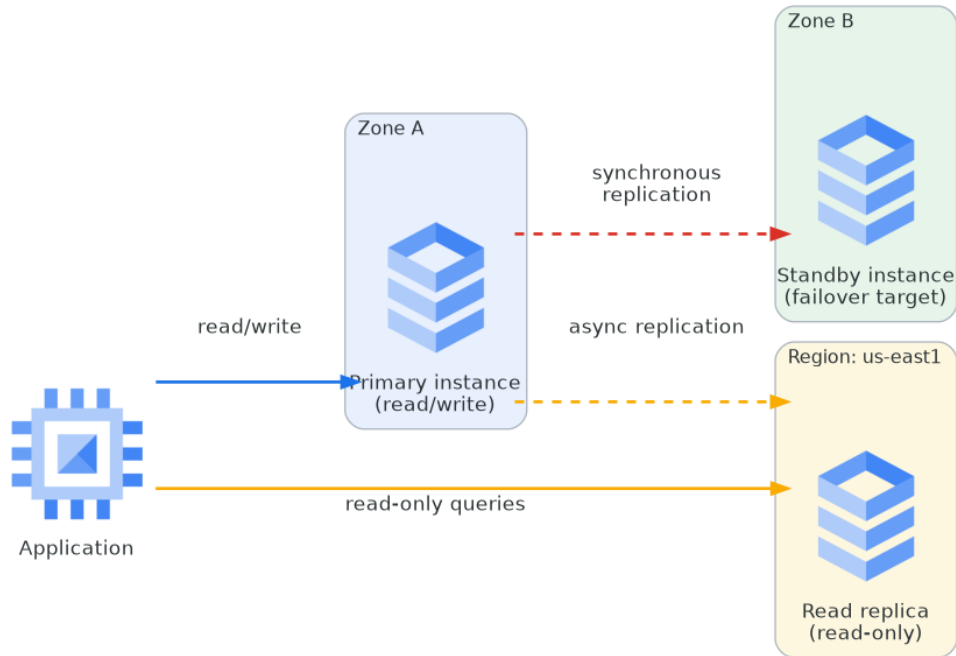


Figure 16: A REGIONAL Cloud SQL instance keeps a synchronously replicated standby in a second zone for automatic failover, plus an asynchronous read replica for scaling reads.

--availability-type=ZONAL runs a single instance with no standby: cheaper, but any zone outage means downtime.

Read Replicas for Scaling Read Traffic

A read replica copies your primary asynchronously (a small, typically sub-second lag) and serves read-only queries independently. Because replication is asynchronous, replicas can live in a different region than the primary.

```
gcloud sql instances create my-sql-instance-replica \
  --master-instance-name=my-sql-instance \
  --region=us-east1
```

- my-sql-instance-replica names the new replica.
- --master-instance-name=my-sql-instance designates the source instance being copied.
- --region=us-east1 places the replica in a different region than the primary, showing replicas aren't restricted to the primary's region.

Direct read-only traffic to the replica's own endpoint and keep all writes on the primary — Cloud SQL does not route traffic between them automatically.

Backups and Point-in-Time Recovery

Cloud SQL takes automated daily backups by default and retains transaction logs enabling point-in-time recovery (PITR) — restoring to an exact moment within the retention window, not just to the last daily backup. This is invaluable after an accidental DROP TABLE or a bad deployment. Both are enabled by default for new instances, but worth explicitly verifying with `gcloud sql instances describe`.

Troubleshooting Common Issues

“Connection refused” or timeouts usually mean the client’s IP was never added to authorized networks, or the Auth Proxy was never started and the app tried connecting directly. Confirm which method you intend to use and verify it end to end.

“Instance is out of disk space” usually traces back to automatic storage increases being disabled. Check with `gcloud sql instances describe my-sql-instance` and confirm `autoresize` is enabled for any production instance.

NOTE

Storage increases are one-directional — Cloud SQL storage can grow automatically but cannot shrink without exporting the data into a new, smaller instance.

Best Practices

- **Use REGIONAL availability type for any production instance** — the cost difference versus ZONAL is modest compared to the cost of an outage.
- **Prefer the Auth Proxy or Private IP over public IP with authorized networks** to avoid allowlist maintenance and reduce exposed attack surface.
- **Right-size the tier based on measured load, not guesswork.** Start modest, monitor CPU and memory in Cloud Monitoring, and scale up only when data supports it.
- **Enable automatic storage increases** unless you have a specific reason to cap growth.

Key Takeaways

- Cloud SQL is a fully managed relational database service for MySQL, PostgreSQL, and SQL Server, handling patching, backups, and replication.
- The Cloud SQL Auth Proxy is the recommended connection method, using IAM-based authentication and encrypted tunnels instead of manual IP allowlists and certificates.
- `--availability-type=REGIONAL` provisions a synchronously replicated standby in a second zone with automatic failover — the standard choice for production.
- Read replicas use asynchronous replication and can live in a different region, useful both for scaling reads and disaster recovery.
- Connection and storage problems are usually configuration issues (`autoresize` disabled, Auth Proxy not in use) rather than deeper database issues.

Chapter 14: Cloud Spanner & Firestore: Globally Distributed Databases

Cloud SQL, covered in Chapter 13, is the right default for most relational workloads, but it has limits. It scales vertically — a bigger machine handles more load — and even with read replicas there's a practical ceiling on strong consistency across regions at very large scale. This chapter covers two other answers: Cloud Spanner, for relational workloads needing horizontal, cross-region scale, and Firestore, a serverless NoSQL document database for backends needing real-time updates.

Cloud Spanner: SQL at Global Scale

Cloud Spanner is Google's globally distributed, horizontally scalable, strongly consistent relational database — an unusual combination. Most databases force a choice between SQL's familiar guarantees (strict consistency, joins, a relational schema) and the horizontal scalability of distributed NoSQL systems. Spanner offers both, which is why it commands a premium price and is reserved for workloads that genuinely need it.

Horizontal scalability means Spanner adds more machines and automatically shards your data across them as load grows, rather than moving to one bigger machine as Cloud SQL does.

Strong consistency across regions means every client sees the same, up-to-date data regardless of which replica answers a query — no window where one region shows stale data. Spanner achieves this with a Paxos-based consensus protocol: for each piece of data, a group of replicas votes on write order, and a write is acknowledged only once a majority durably agrees — synchronous agreement that makes global consistency possible, rather than the eventual consistency common elsewhere.

Instance Configurations, Nodes, and Processing Units

An **instance configuration** determines where replicas physically live: a **regional configuration** keeps replicas within one region, spread across zones for fault tolerance without cross-region latency; a **multi-region configuration** spreads replicas across regions or continents for maximum availability and low-latency reads worldwide, at the cost of higher write latency (distant replicas must agree) and significantly higher price.

Compute and storage capacity is measured in **nodes** (or, more finely, **processing units**, where 1,000 processing units equal one node). Each node adds both throughput and storage capacity, unlike Cloud SQL, where compute and storage are configured somewhat independently.

A Complete Runnable Example

```
gcloud spanner instances create my-app-db \
  --config=regional-us-central1 \
  --description="App instance" \
  --nodes=1

gcloud spanner databases create appdb --instance=my-app-db
```

- my-app-db names the instance — the container holding one or more databases and determining capacity and replica locations.
- --config=regional-us-central1 selects a regional configuration in us-central1. Multi-region configs use names like nam3 or eur5.
- --nodes=1 allocates one node's capacity — the minimum for a regional configuration, and a meaningful ongoing cost even when nearly idle.
- gcloud spanner databases create appdb --instance=my-app-db creates a database inside the instance; one instance can host several.

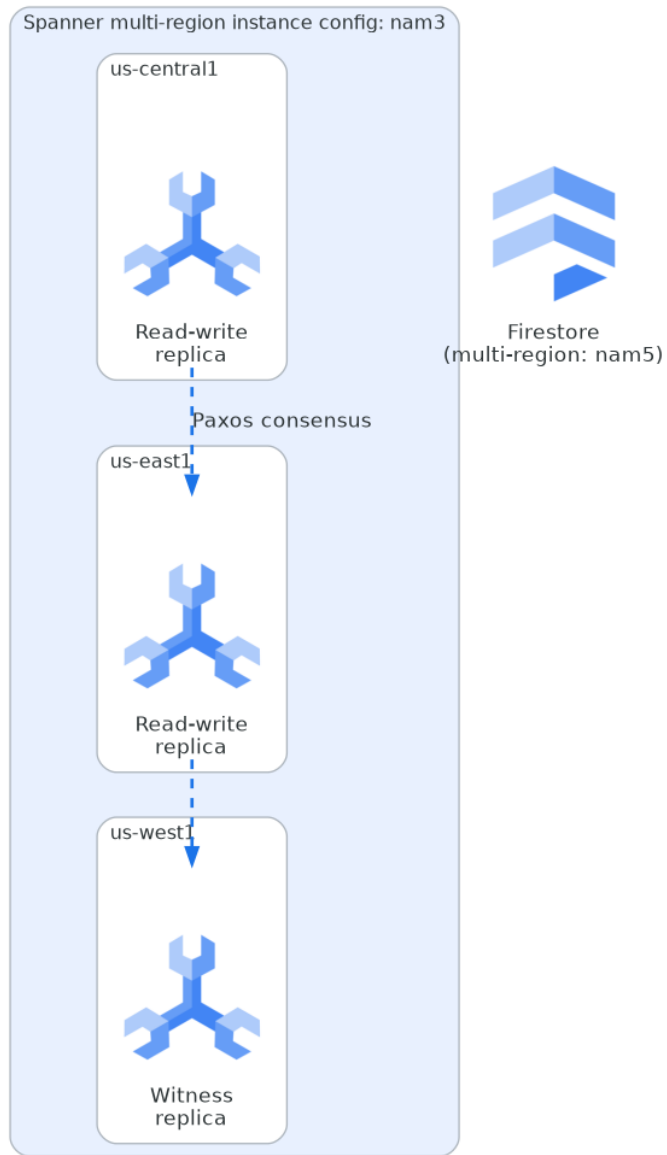


Figure 17: A multi-region Cloud Spanner instance replicates synchronously across zones using Paxos consensus for strong global consistency.

NOTE

Spanner capacity is expressed in nodes or processing units rather than a machine tier, and directly determines both query throughput and storage volume — Spanner’s equivalent of scaling up and out at once.

When Spanner Is Worth It — and When It Is Not

Spanner fits workloads with a documented need for multi-region strong consistency at scale: a global financial ledger, an inventory system that must never oversell stock, or write throughput that has genuinely outgrown a single Cloud SQL instance with read replicas.

For most applications, Cloud SQL is preferable. Spanner’s minimum footprint costs meaningfully more than a generously sized Cloud SQL instance, and its data model — while relational — has differences (how primary keys and interleaved tables affect performance) that require deliberate schema design.

WARNING

Choosing Cloud Spanner for a workload with no real multi-region consistency requirement is one of the most common and expensive architecture mistakes on Google Cloud. If your application runs in one region without cross-region consistency needs, Cloud SQL will very likely serve you better at a fraction of the cost.

Firestore: Serverless NoSQL for Application Backends

Firestore is a fully managed, serverless NoSQL document database, designed primarily as a backend for mobile and web applications. “Serverless” means there are no instances or nodes to size — you create a database and start reading and writing, and Google scales capacity behind the scenes.

Firestore has two modes: **Native mode**, the modern document-database experience with real-time listeners and strong mobile/web client support, and **Datastore mode**, a compatibility mode for applications built against the older Cloud Datastore API. New applications should use Native mode.

Collections, Documents, and Real-Time Listeners

Firestore organizes data into **collections** of **documents**. A document is a set of key-value pairs, similar to a JSON object; a collection is a named group of documents. A users collection might contain one document per user, keyed by user ID, with fields like name, email, and createdAt. Documents can contain nested sub-collections — users/{userId}/orders, for example — letting related data nest hierarchically without the joins a relational database would need.

One of Firestore’s most distinctive features is attaching a real-time listener to a document or query, so a client is automatically notified whenever the underlying data changes — no polling required. This powers the “live updating” feel of many apps: a chat message appearing instantly on every device, or a dashboard reflecting new data the moment it’s written. On a relational database, the same behavior typically requires a separate messaging layer; in Firestore it’s built in. Firestore also replicates data across multiple regions automatically, with no configuration equivalent to Spanner’s instance configurations.

Troubleshooting and Cost Considerations

The most common costly mistake is reaching for Cloud Spanner by default because it sounds like the “bigger, more capable” option, without a concrete requirement justifying it. Since Spanner’s minimum cost substantially exceeds a small or medium Cloud SQL instance, this often means paying a large ongoing premium for consistency guarantees the application never exercises. Before provisioning Spanner, be able to articulate why Cloud SQL, with read replicas if needed, would not meet the requirement.

A second, less costly mistake is choosing Firestore for a workload that is fundamentally relational — heavy joins across many entity types and multi-table transactions — where a relational database is

a more natural fit.

Best Practices

- **Default to Cloud SQL** for relational workloads, and move to Cloud Spanner only with a documented need for horizontal scale or multi-region consistency.
- **Use Firestore for mobile and web backends** that benefit from real-time listeners and a flexible document model, especially when data is naturally hierarchical rather than deeply relational.
- **Be mindful of Spanner's minimum node cost when prototyping** — a single-node regional instance is still a meaningful ongoing expense.
- **Design Spanner schemas deliberately**, considering primary key choice and interleaved tables, rather than assuming a PostgreSQL-style schema will perform equally well unmodified.

Key Takeaways

- Cloud Spanner combines relational SQL semantics with horizontal, multi-region scalability and strong consistency, enabled by a Paxos-based consensus protocol across replicas.
- Spanner capacity is expressed in nodes or processing units, and instance configurations determine whether replicas stay regional or spread across multiple regions.
- Most applications are well served by Cloud SQL; Spanner suits documented multi-region consistency or extreme-scale needs, not a default upgrade path.
- Firestore is a serverless NoSQL document database organized around collections and documents, with real-time listeners as a signature feature for live-updating applications.
- Choosing Spanner without a genuine multi-region consistency need is a common and expensive architectural misstep — evaluate Cloud SQL first.

Chapter 15: Memorystore & Caching Strategies

The previous two chapters covered storing data durably — Cloud SQL, Cloud Spanner, and Firestore. This chapter covers a related problem: making reads fast and reducing load on the primary database using a cache.

What Memorystore Is, and Why Applications Cache Data

Memorystore is Google Cloud's fully managed in-memory data store service, offering managed Redis and managed Memcached. As with Cloud SQL, Google handles patching, monitoring, and (for the configuration below) failover.

The core reason applications cache data is straightforward: not everything needs reading from the primary database every time. A product catalog page or a user's profile changes rarely but is read constantly. An in-memory cache stores such data in memory — far faster than disk-backed storage — so repeated reads are served in a fraction of the time, without touching the primary database.

Creating a Memorystore for Redis Instance

```
gcloud redis instances create my-cache \
  --size=1 \
  --region=us-central1 \
  --redis-version=redis_7_0 \
  --tier=standard
```

- `my-cache` is the instance name.
- `--size=1` requests 1 gibibyte (GiB) of capacity — Redis capacity is specified directly in GiB.
- `--region=us-central1` places the instance in that region; as covered below, Memorystore is reachable only from within the same VPC network, so this should match where your application runs.
- `--redis-version=redis_7_0` selects the Redis engine version.
- `--tier=standard` selects the Standard tier, discussed next.

Basic Tier vs. Standard Tier

- **Basic tier** runs a single node with no replica. Cheaper, but if that node fails or needs maintenance, the cache becomes unavailable — no automatic failover.
- **Standard tier** provisions a replica and automatic failover: if the primary becomes unhealthy, Memorystore promotes the replica with minimal disruption — mirroring Cloud SQL's primary-plus-standby pattern, applied to a cache.

BEST PRACTICE

Use Standard tier for any production workload. The cost difference is modest relative to the disruption a Basic-tier outage causes — a cache outage often has a knock-on effect, since requests that would have hit the cache instead fall through to the primary database.

Common Caching Patterns

Cache-aside (lazy loading) is the most common pattern: the application checks the cache first. On a **cache hit**, it returns the data immediately. On a **cache miss**, it reads from the database, returns the result, and writes it into the cache so the next request is a hit. Frequently requested data accumulates in the cache over time, while unrequested data never gets cached.

Write-through takes writes to the cache and database together, keeping the cache always current, at the cost of added latency on every write rather than only on misses. Most general-purpose caching uses cache-aside, since it only pays the cost of populating the cache for data actually requested.

Cache Invalidation and TTL

Whatever the population strategy, the cache needs a way to deal with data that changes after being cached — otherwise it keeps serving stale values. The simplest tool is a **time-to-live (TTL)**: an expiration set on each cached entry, after which it's discarded, forcing the next request back to the database to refresh the value.

Choosing the right TTL is a real trade-off. There's a well-known, tongue-in-cheek observation, often attributed to Phil Karlton, that “there are only two hard things in computer science: cache invalidation and naming things.” Too long a TTL and users see stale data; too short, and the cache stops providing much benefit since data expires almost as soon as it's cached.

TIP

Set TTLs based on how frequently the data actually changes, not one default applied everywhere. Rarely changing data (a category list, a feature flag) can tolerate hours; frequently changing data (a live inventory count) needs seconds, or an explicit invalidation step on update.

Connecting from Within the VPC

Memorystore instances have no public IP address, by design — reachable only from resources inside the same Virtual Private Cloud (VPC) network, typically the same region. This is deliberate: a cache generally lacks strong built-in authentication, so keeping it off the public internet removes a category of exposure risk. This ties back to the VPC networking concepts covered earlier in this guide — a VM, GKE node, or serverless workload connecting to Memorystore must be attached to, peered with, or connected via a serverless VPC connector to the same network.

Troubleshooting Common Issues

“Could not connect to Redis” or timeouts are almost always networking problems, not application bugs: the client isn't in the same VPC network as the instance, or is in a different region. Confirm the client's VPC and region match before investigating further.

Lower cache hit rate than expected is frequently a symptom of TTLs set too short for the access pattern, so entries expire before being re-requested. Reviewing the read pattern and adjusting TTL is usually more effective than adding cache capacity.

Best Practices

- **Use Standard tier for production instances**, so a single node failure doesn't become a full outage that floods the primary database with traffic.
- **Set TTLs deliberately, based on how often the underlying data changes**, rather than one blanket value for everything.
- **Treat cache hit rate as a first-class metric**, monitored alongside database load — a dropping hit rate is often the earliest warning of trouble that will soon show up as increased database load.

Key Takeaways

- Memorystore provides fully managed Redis and Memcached, used to reduce primary database load and cut latency for frequently read, rarely changed data.
- Standard tier adds a replica and automatic failover; Basic tier is cheaper but has no failover, making Standard the right default for production.
- Cache-aside (check cache, fall back to the database on a miss, populate the cache) is the most common pattern; write-through keeps the cache and database in sync on every write, at extra write cost.
- TTLs are the primary defense against stale data, set based on how often the underlying data actually changes.
- Memorystore has no public IP and is reachable only within the same VPC network, so connec-

tivity issues are usually networking problems, not caching-logic problems.

Chapter 16: Service Accounts & Workload Identity

Every previous chapter has assumed that *something* is allowed to create a VM, read a bucket, or deploy a service. That “something” has an identity behind it, and so far most of the examples in this book have quietly relied on your own user account’s permissions. Production workloads should not. This chapter covers **service accounts** — the identity mechanism GCP (Google Cloud Platform) provides for applications and infrastructure rather than humans — and the modern, key-free ways of using them safely.

Service Accounts vs. User Accounts

A **user account** represents a human being: you sign in with your email address and password (and typically multi-factor authentication), and Google Cloud’s Identity and Access Management (IAM) system grants that human identity certain roles. A **service account** represents a workload — a VM, a Cloud Run service, a CI/CD (continuous integration/continuous delivery) pipeline, or any other piece of software that needs to call GCP APIs on its own behalf, without a person typing in credentials.

Functionally, a service account behaves like any other IAM principal: you can grant it roles, add it as a member on a policy binding, and audit its actions in Cloud Logging. The key differences are practical:

- A service account has no password and does not use multi-factor authentication. Instead, it authenticates using cryptographic keys or short-lived tokens, described below.
- A service account is identified by an email-like address that GCP generates for you, in the form `<name>@<project-id>.iam.gserviceaccount.com` — for example, `app-sa@my-gcp-project-123456.iam`
- A service account is meant to be attached to infrastructure (a VM, a container, a function) or impersonated by another principal, not logged into interactively.

The Default Compute Engine Service Account

Every GCP project automatically has a **default Compute Engine service account**, and unless you specify otherwise, any new VM you create is attached to it. This is convenient for a first experiment, but it is a poor long-term choice for two reasons. First, the default service account historically accumulates the broad `roles/editor` role on the project, meaning any VM using it can modify nearly any resource in the project — far more than almost any single workload actually needs. Second, because it is shared across every VM that does not specify its own service account, a compromise of one workload effectively grants the same broad access to whatever the attacker does with that credential, and it becomes hard to tell from an audit log which specific workload performed a given action.

BEST PRACTICE

Create a dedicated service account per workload or application, scoped to only the roles that workload actually needs, rather than relying on the default Compute Engine service account. This follows the **principle of least privilege** — granting the minimum access necessary — and makes audit logs meaningful, since every action is traceable to a specific, narrowly-scoped identity.

A Complete Example: Dedicated Service Account for a Workload

Suppose `web-server-1` runs an application that only needs to read objects from Cloud Storage. Start by creating a purpose-specific service account:

```
gcloud iam service-accounts create app-sa \
  --display-name="Application service account"
```

- `app-sa` is the account ID; combined with the project, GCP generates the full email `app-sa@my-gcp-project-123456.iam.gserviceaccount.com`.

- `--display-name` is a human-readable label shown in the console, useful when a project accumulates many service accounts over time.

Next, grant it only the role it needs, at the narrowest scope that makes sense — here, project-level, since the exact bucket may not exist yet or may need to be added later:

```
gcloud projects add-iam-policy-binding my-gcp-project-123456 \
  --member="serviceAccount:app-sa@my-gcp-project-123456.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

- `--member="serviceAccount:..."` — the `serviceAccount:` prefix tells IAM this binding applies to a service account, not a user or group.
- `--role="roles/storage.objectViewer"` — a narrow, read-only role limited to viewing and reading object contents and metadata, deliberately far short of `roles/editor`.

Finally, attach the service account to the VM at creation time:

```
gcloud compute instances create web-server-1 \
  --zone=us-central1-a \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --service-account=app-sa@my-gcp-project-123456.iam.gserviceaccount.com \
  --scopes=cloud-platform
```

- `--service-account` — attaches `app-sa` as the VM's identity, instead of the default Compute Engine service account.
- `--scopes=cloud-platform` — grants the broadest possible OAuth scope, deliberately, because access is actually enforced by the IAM roles granted to `app-sa` (just `roles/storage.objectViewer` above), not by the scope. This is the modern recommended pattern: use the broad `cloud-platform` scope on the VM, and do the real access control entirely through IAM role bindings.

NOTE

Scopes and IAM roles are two different, overlapping layers. A scope limits what OAuth (Open Authorization) token the VM's metadata server will ever hand out; the IAM role limits what that token can actually do once presented to an API. Modern practice avoids narrow scopes entirely and instead controls access purely through IAM, which is easier to reason about and audit.

Why Downloaded JSON Keys Are Risky

Historically, the way to authenticate as a service account from outside GCP (a laptop, an on-premises server, another cloud) was to download a **JSON key file** — a long-lived credential you could pass to client libraries or the `gcloud` CLI. This remains possible, but it should be treated as a last resort, for three concrete reasons:

- **They do not expire on their own.** A downloaded key remains valid indefinitely unless someone remembers to rotate or revoke it, unlike the short-lived tokens described next.
- **They are easy to leak.** A JSON key is just a file, and files get committed to source control, copied into container images, or left in home directories far more often than anyone would like to admit.
- **They are hard to audit.** Once a key is downloaded, GCP has no visibility into where it lives or who holds a copy — only that a request came in authenticated as that service account.

WARNING

If you find a downloaded JSON key checked into a Git repository, treat it as compromised immediately. Disable and delete the key (`gcloud iam service-accounts keys delete`), rotate any secrets the workload had access to, and only then remove it from the repository — removing it from the latest commit does not remove it from Git history.

Service Account Impersonation

The modern, key-free alternative for a principal that needs to *act as* a service account temporarily is **impersonation**. If a user or another service account is granted the `roles/iam.serviceAccountTokenCreator` role on a target service account, it can request a short-lived access token (typically valid for one hour) that lets it act as that service account, without ever downloading or possessing a long-lived key.

```
gcloud projects add-iam-policy-binding my-gcp-project-123456 \
  --member="user:ana@example.com" \
  --role="roles/iam.serviceAccountTokenCreator" \
  --condition=None
```

- This grants the named user permission to impersonate *any* service account with a suitably scoped binding; in practice you would scope this binding to the specific service account resource (`gcloud iam service-accounts add-iam-policy-binding app-sa@... --role=roles/iam.serviceAccountTokenCreator --member=user:...`) rather than project-wide, so the user can only impersonate the one account they need.

Once granted, running any `gcloud` command with `--impersonate-service-account=app-sa@my-gcp-project` executes as that service account using a token that expires automatically, with no key file ever touching disk.



Figure 18: Impersonation lets a caller obtain short-lived credentials as a service account without ever downloading a long-lived JSON key.

Workload Identity Federation

Impersonation solves the problem for principals that already exist inside GCP's IAM system. But what about a GitHub Actions CI/CD pipeline, or a workload running in another cloud provider, that needs to deploy to GCP without ever holding a GCP credential at all? **Workload Identity Federation** solves exactly this: it lets an external identity provider's tokens (an OIDC — OpenID Connect — token issued by GitHub, AWS, or another trusted identity source) be exchanged for permission to impersonate a GCP service account, entirely without a downloaded key.

The shape of the setup is a **workload identity pool** (a container for external identities) and a **provider** within it (the specific external issuer, such as GitHub's OIDC endpoint):

```
gcloud iam workload-identity-pools create github-pool \
  --location="global" \
  --display-name="GitHub Actions pool"

gcloud iam workload-identity-pools providers create-oidc github-provider \
  --location="global" \
  --workload-identity-pool=github-pool \
  --issuer-uri="https://token.actions.githubusercontent.com" \
```

```
--attribute-mapping="google.subject=assertion.sub"
```

- `workload-identity-pools create` — defines a namespace that will hold trusted external identities.
- `providers create-oidc` — registers GitHub Actions specifically as a trusted OIDC token issuer for that pool.
- `--attribute-mapping` — maps a claim from GitHub’s token (`assertion.sub`, the subject) into a Google-recognized attribute, which is then used to scope exactly which repository or branch is allowed to authenticate.

Once configured, a GitHub Actions workflow presents its own short-lived OIDC token, GCP verifies it against the trusted provider, and exchanges it for a short-lived token to impersonate a designated service account — closing the loop without any long-lived credential existing anywhere.

Troubleshooting

- **“Permission denied” even though I granted the role.** Double-check *who* you granted the role to. It is common to grant a role to your own user account while testing, then wonder why the VM still fails — the VM authenticates as its attached service account, not as the user who created it. The role must be bound to the service account itself.
- **Attached `--scopes=cloud-platform` but access still denied.** This is expected and correct: the broad scope only determines what the metadata server is *willing to hand out*; actual permissions still come entirely from the IAM roles granted to the service account. Adding a scope never substitutes for granting a role.
- **Impersonation fails with an authorization error.** Confirm the `roles/iam.serviceAccountTokenCreator` binding was granted on the service account resource itself (or a resource containing it), not just generically in the project, and that the calling identity matches exactly what was granted.

Best Practices

- Create one service account per workload or purpose, rather than reusing a single broadly-permissioned account across many applications.
- Avoid downloading JSON keys whenever an alternative — impersonation or Workload Identity Federation — is available.
- Prefer short-lived, automatically expiring credentials over long-lived ones in every case where GCP offers the choice.
- Periodically audit service accounts and keys with `gcloud iam service-accounts list` and `gcloud iam service-accounts keys list`, removing unused accounts and any keys that no longer need to exist.

Key Takeaways

- A service account is an identity for workloads, not humans, authenticated via keys or short-lived tokens rather than a password.
- Avoid the default Compute Engine service account for production workloads; create a dedicated, narrowly-scoped service account per application instead.
- Downloaded JSON keys are long-lived, easy to leak, and hard to audit — treat them as a last resort.
- Service account impersonation (via `roles/iam.serviceAccountTokenCreator`) and Workload Identity Federation both provide short-lived access without ever storing a long-lived credential.
- Use `--scopes=cloud-platform` on VMs and enforce actual access purely through IAM role bindings, since scopes alone do not grant permissions.

Chapter 17: Cloud KMS, Secret Manager & Data Encryption

Data protection on GCP splits into two related but distinct concerns: encrypting the data itself, and safely storing the sensitive configuration values (passwords, API keys, tokens) your applications need at runtime. This chapter covers **Cloud KMS (Key Management Service)** for the first concern and **Secret Manager** for the second, and is deliberately explicit about where the line between them falls, since the two are often confused.

Encryption at Rest: The Default, and When You Need More

Every piece of data GCP stores for you — every object in Cloud Storage, every persistent disk, every row in a managed database — is encrypted at rest by default, automatically, using keys that Google generates and manages entirely on your behalf. **Google-managed encryption** requires no configuration and no key management on your part; it is simply always on.

For most organizations, this default is sufficient. But some organizations — often for regulatory or compliance reasons — need demonstrable control over the encryption keys themselves: the ability to rotate them on their own schedule, restrict who can use them, and revoke access to the keys (which effectively makes the encrypted data unreadable) independently of the underlying storage system. This is what **customer-managed encryption keys (CMEK)**, provisioned through Cloud KMS, provide.

NOTE

CMEK does not mean *you* handle the raw key material or the cryptographic operations — Google's infrastructure still performs the actual encryption and decryption. What changes is that the key used lives in a Cloud KMS resource you create, control access to, and can rotate or disable independently of the storage service using it.

Cloud KMS Core Concepts

Cloud KMS organizes keys into two levels:

- **Key ring** — a regional grouping of keys, existing purely as a container with no cryptographic properties of its own. A key ring is created once and lives in a specific location (for example, `us-central1`), which then constrains where any keys inside it can be used.
- **Crypto key** — the actual key resource used to encrypt and decrypt data, created inside a key ring. A crypto key does not expose its raw key material directly; instead, you reference it by resource path when configuring another service (such as Cloud Storage) to use it.

Cloud KMS also supports **key rotation** — automatically generating a new underlying key version on a schedule (for example, every 90 days) while keeping older versions available to decrypt data that was encrypted before the rotation. This happens transparently to anything using the key; you do not need to re-encrypt existing data every time the key rotates.

A Complete Example: CMEK-Protected Cloud Storage Bucket

First, create a key ring in the same region as the resources that will use it:

```
gcloud kms keyrings create my-keyring \
  --location=us-central1
```

- `my-keyring` — the name of the key ring; key ring names cannot be changed or deleted once created (though the keys inside can be disabled), so choose a durable, descriptive name.
- `--location=us-central1` — key rings are regional; a key inside this ring can only protect resources in the same location.

Next, create a crypto key inside that key ring:

```
gcloud kms keys create my-key \  
  --keyring=my-keyring \  
  --location=us-central1 \  
  --purpose=encryption
```

- `--purpose=encryption` — specifies this key is for symmetric encrypt/decrypt operations, as opposed to asymmetric signing or other purposes Cloud KMS also supports.
- By default, a newly created key rotates automatically every 90 days; this can be adjusted with `--rotation-period` and `--next-rotation-time` at creation time.

Finally, create a Cloud Storage bucket that uses this key as its default encryption key for every object written to it:

```
gcloud storage buckets create gs://my-app-assets \  
  --location=us-central1 \  
  --default-encryption-key= \  
    ↪ projects/my-gcp-project-123456/locations/us-central1/keyRings/my-keyring/cryptoKeys/my-key
```

- `--default-encryption-key` — takes the full resource path of the crypto key; every object subsequently written to `my-app-assets` is encrypted with this key rather than a Google-managed one.
- The bucket's location (`us-central1`) must match the key ring's location; Cloud KMS does not allow a bucket to reference a key in a different region.

WARNING

If the Cloud Storage service agent's access to the crypto key is later revoked, or the key itself is disabled or destroyed, objects protected by it become permanently unreadable. Disabling a key is a powerful, effectively irreversible operation for any data encrypted with it — treat key deletion with the same caution as deleting the data itself.

Secret Manager: A Different Problem

Cloud KMS manages *encryption keys*. It is not where you should put your database password, a third-party API key, or an OAuth client secret — those are **secret values**, and GCP provides a separate, purpose-built service for them: **Secret Manager**.

A **secret** in Secret Manager is a named container for one or more **versions** of a sensitive value. Every version you add is immutable once created and automatically encrypted at rest (using Google-managed keys by default, or CMEK if configured); access to read a version is governed entirely through IAM roles, and every access is recorded in Cloud Logging's audit logs — giving you a complete, queryable history of exactly which identity read which secret and when.

A Complete Example: Storing and Retrieving a Secret

Create a secret container first:

```
gcloud secrets create db-password \  
  --replication-policy=automatic
```

- `db-password` — the secret's name; the value itself is added separately as a version, never inline in this command.
- `--replication-policy=automatic` — lets Google choose where to replicate the secret's encrypted data for durability and availability, appropriate for most use cases; a user-managed policy exists for pinning replication to specific regions when compliance requires it.

Add the actual sensitive value as a version:

```
echo -n "S3cr3tPass!" | gcloud secrets versions add db-password \
--data-file=-
```

- `echo -n` — prints the password without a trailing newline, so the newline does not become part of the stored secret value.
- `--data-file=-` — reads the secret payload from standard input (the piped output of `echo`) rather than from a file on disk, which avoids ever writing the plaintext password to a file.

Retrieve the value at runtime, typically from within an application or a deployment script rather than interactively:

```
gcloud secrets versions access latest --secret=db-password
```

- `latest` — a special version alias that always resolves to the most recently added, enabled version, so callers do not need to track version numbers as the secret is rotated.

TIP

In application code, use the Secret Manager client library to fetch a secret at startup rather than shelling out to `gcloud`. Combine this with caching the value in memory for the life of the process, so you are not calling Secret Manager on every single request.

Why Not Environment Variables or Config Files?

It is tempting to just put a password in an environment variable or a configuration file baked into a deployment. Secret Manager is preferred for production workloads for several concrete reasons:

- **Centralized rotation.** Updating a secret means adding a new version in one place; every consumer that reads `latest` (or is updated to a new specific version) picks up the change without redeploying configuration files across many machines.
- **Audit logging of every access.** Cloud Logging records exactly which identity accessed which secret version and when — visibility a plain environment variable or file simply cannot provide.
- **Fine-grained IAM.** Access can be granted per secret, so a workload that only needs the database password is never able to read an unrelated API key stored in a different secret.

Troubleshooting

- **“Permission denied” accessing a secret.** The calling identity — typically a service account — needs the `roles/secretmanager.secretAccessor` role, granted either project-wide or, preferably, on the specific secret resource. Check with `gcloud secrets get-iam-policy db-password`.
- **Confusing Cloud KMS with Secret Manager.** A common early mistake is trying to store an API key directly in Cloud KMS, or trying to use Secret Manager to manage encryption keys for a storage bucket. Cloud KMS manages the *keys that encrypt data*; Secret Manager manages *the sensitive values themselves*. If you are storing a password, token, or credential, use Secret Manager. If you need control over the encryption key protecting a storage resource, use Cloud KMS.
- **CMEK-protected bucket creation fails with a permission error.** The Cloud Storage service agent (a Google-managed service account distinct from your own) needs the `roles/cloudkms.cryptoKeyEncrypterDecrypter` role on the crypto key itself; this binding is usually created automatically the first time you configure CMEK through the console, but can require a manual grant when scripting the setup end-to-end.

Best Practices

- Use CMEK when a compliance requirement demands demonstrable control over encryption keys; otherwise, Google-managed encryption at rest is secure and requires no extra operational burden.

- Never commit secret values to source control, even temporarily — use Secret Manager (or, for CI/CD pipelines, its equivalent) from the very first commit of a project.
- Grant `roles/secretmanager.secretAccessor` on individual secrets rather than project-wide, so a compromised workload cannot read secrets it has no business accessing.
- Leave automatic key rotation enabled on Cloud KMS keys unless you have a specific reason to manage rotation manually.

Key Takeaways

- GCP encrypts data at rest by default using Google-managed keys; Cloud KMS with CMEK adds customer control over the encryption keys themselves, for organizations that need it.
- Cloud KMS organizes keys into regional key rings containing crypto keys, with automatic rotation available on a configurable schedule.
- Secret Manager is a distinct service for storing sensitive configuration values, organized as versioned secrets, with access controlled by IAM and every read logged for audit.
- Prefer Secret Manager over environment variables or configuration files for anything sensitive, for centralized rotation, audit visibility, and fine-grained access control.
- Keep the concepts separate: Cloud KMS protects encryption keys used by other services; Secret Manager stores the sensitive values your applications actually need at runtime.

Chapter 18: Cloud Monitoring & Cloud Logging

Building and deploying a workload is only half the job — you also need to know whether it is healthy right now, and be able to reconstruct exactly what happened when something goes wrong. GCP's **Operations Suite** (formerly branded Stackdriver) provides both halves of that story: **Cloud Monitoring** for metrics, dashboards, and alerting, and **Cloud Logging** for capturing and searching detailed log records. This chapter covers both, and how they complement each other in a production system.

The Operations Suite: On By Default

A genuinely useful property of GCP's Operations Suite is that most of it is switched on automatically the moment you create a resource — you do not install an agent or configure a pipeline just to get basic visibility. A Compute Engine VM, for example, automatically reports infrastructure-level metrics (CPU utilization, disk I/O, network traffic) to Cloud Monitoring, and its system logs (boot messages, kernel events) flow into Cloud Logging, with no setup required.

For more detailed, application-aware metrics and logs — memory usage from inside the guest operating system, logs from a specific application process, and so on — Google provides the **Ops Agent**, an optional piece of software installed on the VM that collects and forwards this richer data. Basic infrastructure health is free and automatic; deeper visibility into what is happening inside the VM requires this small extra step.

Cloud Logging: Structure and Retention

Every log entry Cloud Logging receives is routed into a **log bucket** — a storage container for logs, distinct from a Cloud Storage bucket, that determines how long entries are retained and where they can be queried from. Every project automatically has two log buckets from the moment it is created:

- **_Required** — holds admin activity, system event, and access transparency logs that GCP retains for a fixed 400 days and that cannot be modified or deleted, ensuring a baseline audit trail always exists.
- **_Default** — holds essentially everything else (application logs, data access logs if enabled, and so on), with a default retention of 30 days, which you can extend or shorten by editing the bucket's retention setting.

You can also create additional custom log buckets with their own retention periods and route specific logs into them using **log sinks**, useful when certain logs need much longer retention (compliance logs kept for years) while routine debug logs are fine being discarded after a week.

To search logs from the command line rather than the console, use `gcloud logging read` with a filter expression:

```
gcloud logging read "resource.type=gce_instance AND severity>=WARNING" \
  --limit=20
```

- `resource.type=gce_instance` — restricts results to log entries produced by Compute Engine VMs, as opposed to Cloud Run services, Cloud Functions, or other resource types that also write to Cloud Logging.
- `severity>=WARNING` — Cloud Logging severities are ordered (DEBUG, INFO, NOTICE, WARNING, ERROR, CRITICAL, ALERT, EMERGENCY); this filter returns WARNING and anything more severe, excluding routine informational noise.
- `--limit=20` — caps the number of entries returned, since a broad query against a busy project can otherwise return an unmanageable volume of output.

TIP

The filter syntax used by `gcloud logging read` is the same one used in the Logs Explorer in the Cloud Console, so a filter you build and refine interactively in the console can be copied directly into a script or alerting query.

Log-Based Metrics

Sometimes the thing you actually want to monitor is not a raw infrastructure number but a *pattern in your logs* — for example, the rate at which your application logs an “authentication failed” message. A **log-based metric** turns exactly this kind of pattern into a proper time series that Cloud Monitoring can graph, alert on, and treat like any other metric. You define a log-based metric with a filter (similar in shape to the one above) and, optionally, a way to extract a numeric value from matching entries; from that point on, it behaves like any built-in metric for dashboards and alerting purposes.

Cloud Monitoring: Structure

Cloud Monitoring is built around four core concepts:

- **Metrics** — time series data: a numeric value recorded over time, such as CPU utilization sampled every minute. Metrics are the raw material everything else in Cloud Monitoring is built from.
- **Dashboards** — visualizations that plot one or more metrics over a chosen time range, letting you see trends and anomalies at a glance rather than reading raw numbers.
- **Uptime checks** — synthetic, external monitoring that periodically sends a request (HTTP, HTTPS, or TCP) to an endpoint from outside GCP’s own network and records whether it succeeded and how long it took, catching outages that internal metrics might miss entirely (for example, a DNS misconfiguration that makes your service unreachable even though the VM itself looks perfectly healthy).
- **Alerting policies** — rules that watch a metric (or a log-based metric) and notify one or more **notification channels** (email, Slack, PagerDuty, and others) when a defined condition is met, such as a threshold being crossed for a sustained period.

A typical dashboard for `web-server-1` might show its uptime check status, CPU utilization over the last 24 hours, and network throughput side by side, giving an on-call engineer a single place to check the instance’s overall health before diving into logs for detail.

A Complete Example: Alerting on CPU Utilization

Alerting policies are most reliably created from a JSON (or YAML) definition, since the condition logic is more detailed than fits comfortably as command-line flags. First, define a notification channel so the policy has somewhere to send alerts:

```
gcloud alpha monitoring channels create \
  --display-name="Ops Email" \
  --type=email \
  --channel-labels=email_address=ops@example.com
```

- `--type=email` — the simplest channel type; Cloud Monitoring also supports Slack, PagerDuty, SMS, and webhook-based channels for integrating with other on-call tooling.
- `--channel-labels=email_address=...` — the channel-type-specific configuration, here the address that receives the alert email.

Next, define the alerting policy itself in a JSON file, for example `cpu-alert-policy.json`:

```
{
  "displayName": "High CPU utilization",
  "conditions": [
    {
```

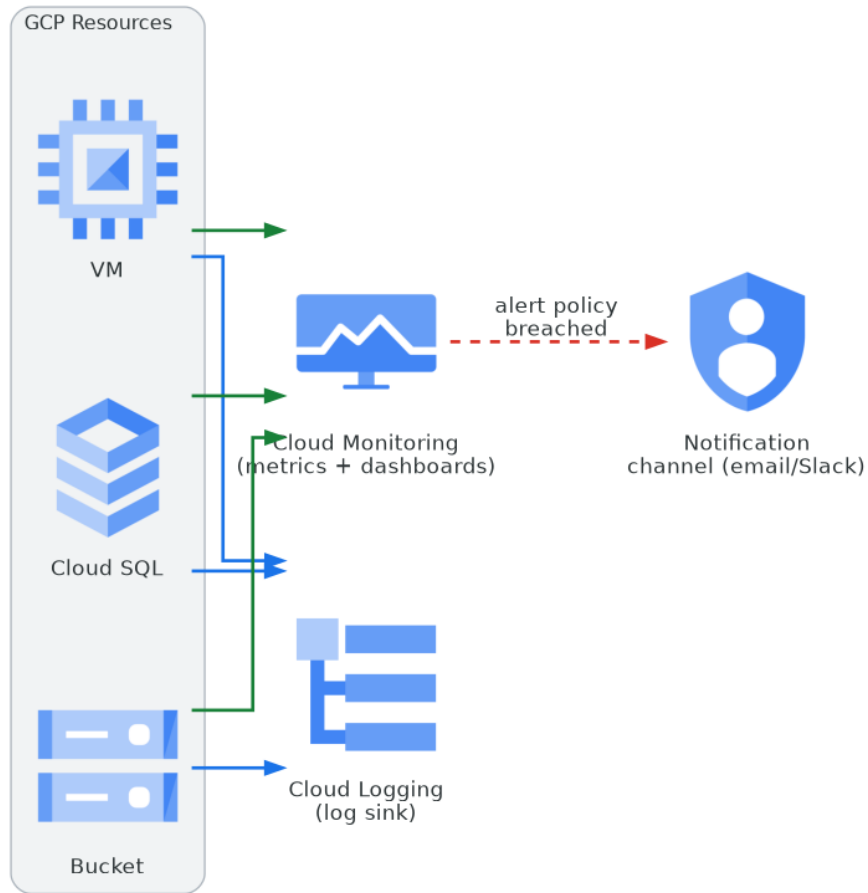


Figure 19: GCP resources feed both Cloud Logging and Cloud Monitoring; monitoring alert policies notify a channel when a threshold is breached.

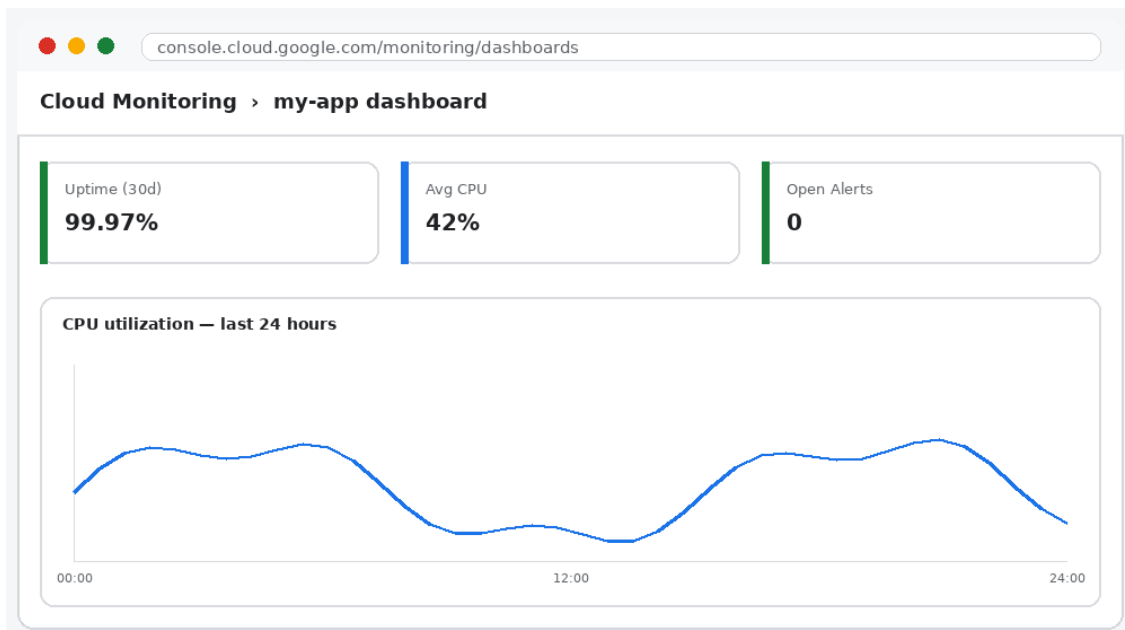


Figure 20: Illustrative a Cloud Monitoring dashboard showing uptime, CPU, and a utilization chart.

```

    "displayName": "CPU above 80% for 5 minutes",
    "conditionThreshold": {
      "filter": "resource.type=\"gce_instance\" AND
        ↪ metric.type=\"compute.googleapis.com/instance/cpu/utilization\"",
      "comparison": "COMPARISON_GT",
      "thresholdValue": 0.8,
      "duration": "300s",
      "aggregations": [
        {
          "alignmentPeriod": "60s",
          "perSeriesAligner": "ALIGN_MEAN"
        }
      ]
    }
  ],
  "combiner": "OR",
  "notificationChannels": [
    "projects/my-gcp-project-123456/notificationChannels/CHANNEL_ID"
  ]
}

```

Then create the policy from that file:

```

gcloud alpha monitoring policies create \
  --policy-from-file=cpu-alert-policy.json

```

- "thresholdValue": 0.8 with "comparison": "COMPARISON_GT" — fires the condition when CPU utilization exceeds 80 percent (Cloud Monitoring expresses utilization as a fraction between 0 and 1, so 0.8 is 80 percent).
- "duration": "300s" — requires the condition to remain true for a sustained 5 minutes before the policy fires, preventing a brief, harmless spike from triggering an alert.
- notificationChannels — references the channel created above by its resource ID (returned when the channel was created, or found with `gcloud alpha monitoring channels list`), so the alert actually reaches someone.

NOTE

The CHANNEL_ID placeholder above must be replaced with the actual ID returned by the channels create command; the alerting policy does nothing useful without a valid, existing notification channel attached.

Monitoring vs. Logging: Two Different Questions

It is worth being explicit about why you need both services rather than just one. **Monitoring answers “is it healthy right now, and will it be soon”** — a single number (CPU at 92 percent, an uptime check failing) that tells you something needs attention, fast, without requiring you to read anything. **Logging answers “what exactly happened”** — the detailed, ordered record of events that lets you understand *why* CPU spiked or *why* the uptime check failed, often long after the fact. A production incident typically starts with a monitoring alert (something is wrong) and is diagnosed by reading logs (here is why), which is why mature systems invest in both rather than treating one as a substitute for the other.

Troubleshooting

- **Alert fires too often (“alert fatigue”).** A threshold set too aggressively — for example, alerting the instant CPU crosses 50 percent, or with no sustained duration requirement — trains

on-call engineers to ignore or silence notifications, which defeats the entire purpose of alerting. Raise the threshold, add or lengthen the duration requirement, or both.

- **Alert never fires, even during an outage.** The opposite problem: a threshold set too leniently, a filter that does not actually match the resource you intended, or a metric that is not, in fact, the right signal for the failure mode you are trying to catch. Test alerting policies deliberately (for example, temporarily lowering the threshold) rather than assuming they work simply because they were created without error.
- **Uptime check reports failures but the service seems fine internally.** Confirm the check's target is reachable from the public internet if the service is meant to be public, and check whether a firewall rule or load balancer configuration is blocking the uptime checker's source IP ranges specifically, as distinct from ordinary user traffic.

Best Practices

- Start with a small number of high-signal alerting policies — a handful of conditions that genuinely require action — rather than alerting on every metric available, which produces noise that engineers learn to ignore.
- Use uptime checks for anything user-facing; they catch entire classes of failure (DNS, routing, load balancer misconfiguration) that internal VM-level metrics cannot see.
- Configure log bucket retention deliberately: long enough that you can investigate an incident discovered days after it began, but not so long that you are paying to store logs nobody will ever read. Route anything that needs long-term retention for compliance into a dedicated custom log bucket rather than relying on `_Default`.
- Treat monitoring and logging as complementary, not redundant — alerts tell you something is wrong; logs tell you why.

Key Takeaways

- Cloud Monitoring and Cloud Logging together form GCP's Operations Suite, and both are largely active automatically the moment you create supported resources, with the Ops Agent adding deeper, application-level detail.
- Cloud Logging routes entries into log buckets; `_Required` and `_Default` exist automatically, and `gcloud logging read` with a filter expression lets you search from the command line.
- Cloud Monitoring is built from metrics, dashboards, uptime checks, and alerting policies that notify a channel (email, Slack, PagerDuty) when a condition is met.
- Alerting policies need a carefully chosen threshold and duration: too aggressive causes alert fatigue, too lenient means real incidents go unnoticed.
- Monitoring tells you something is wrong right now; logging tells you exactly what happened — production systems need both, working together.

Chapter 19: Cloud Audit Logs & Security Command Center

Every action taken against a Google Cloud Platform (GCP) project leaves a trace. Someone creates a virtual machine, changes an IAM (Identity and Access Management) policy, or reads a row out of a database — all of it is recorded automatically. That record is Cloud Audit Logs, and it answers the single most important question in any security investigation: who did what, where, and when.

This chapter covers Cloud Audit Logs, then introduces Security Command Center (SCC), GCP's centralized dashboard for surfacing misconfigurations, vulnerabilities, and active threats. Audit logs tell you what already happened; SCC helps you catch problems before they become incidents.

The Three Types of Cloud Audit Logs

GCP writes audit log entries into three log streams, each with different defaults and purposes.

Admin Activity logs record changes to a resource's configuration or metadata — creating a VM, deleting a bucket, modifying an IAM policy, changing a firewall rule. They are always on, cannot be disabled, and are free, running from the moment a project is created.

Data Access logs record operations that read or write the data inside a resource, as opposed to changes to its configuration — reading an object from Cloud Storage, querying a BigQuery table, fetching a secret's value. Because data access volume is far higher than configuration changes, these logs are off by default for most services and can be costly if enabled broadly, so you must deliberately turn them on for what you care about.

System Event logs are generated by Google Cloud itself, not a human or service account — for example, a Compute Engine VM being live-migrated to different hardware for maintenance.

NOTE

"Audit log" in GCP refers to this specific, structured category — distinct from the general-purpose application logs of Chapter 15. Audit logs follow a fixed schema (protoPayload) built for accountability and compliance.

Viewing Admin Activity Logs

Because Admin Activity logs are always on, you can query them immediately, with no setup. This command searches for every `SetIamPolicy` call — every time someone changed who has access to what:

```
gcloud logging read "logName:activity AND protoPayload.methodName:SetIamPolicy" \
  --limit=10 \
  --format=json
```

- `logName:activity` filters to the Admin Activity stream, as opposed to Data Access or System Event.
- `protoPayload.methodName:SetIamPolicy` filters to the API method that changes an IAM policy, one of the most security-sensitive actions in any project.
- `--limit=10` caps the results to the ten most recent matching entries.
- `--format=json` renders full structured output, including the calling principal (`protoPayload.authentication`), timestamp, and policy delta applied.

Enabling Data Access Logs

Data Access logging is controlled through an IAM audit configuration attached to a project's IAM policy:

```
{
  "auditConfigs": [
    {
      "service": "storage.googleapis.com",
```

```

    "auditLogConfigs": [
      { "logType": "DATA_READ" },
      { "logType": "DATA_WRITE" }
    ]
  }
}

```

- service names the specific API whose data access should be logged — here, Cloud Storage.
- DATA_READ and DATA_WRITE are the two Data Access log types; enable either or both independently.
- This block merges into the project's IAM policy (via `gcloud projects get-iam-policy / set-iam-policy`, or the Console's Audit Logs page) — per-service enablement means you only pay for the visibility you actually need.

BEST PRACTICE

Enable Data Access logs proactively for your most sensitive services — Secret Manager, Cloud KMS, and any Cloud Storage buckets holding regulated data are good starting points — before you need them for an investigation, not after.

Why Immutability and 400-Day Retention Matter

Admin Activity logs are retained for 400 days regardless of other retention settings, and cannot be edited or deleted by any project member, including owners. This is deliberate: even an attacker with full IAM control could not erase the record of the IAM changes they made to get that control — a guarantee compliance auditors often rely on first.

Security Command Center

Security Command Center is GCP's centralized security and risk dashboard. Where audit logs give a detailed, per-event historical record, SCC gives a continuously updated view of your current security posture — what's wrong right now, ranked by how much it matters.

SCC surfaces three categories of findings:

- **Misconfigurations** — a Cloud Storage bucket with public read access, or a firewall rule allowing ingress from `0.0.0.0/0` (any IP address on the internet) on a sensitive port like 22 or 3389.
- **Vulnerabilities** — known software weaknesses, such as an outdated OS image or a container image with a known CVE (Common Vulnerabilities and Exposures) entry.
- **Threats** — active malicious activity, such as a compromised VM communicating with a known command-and-control server.

SCC has two tiers. **Standard** is free and covers basic misconfiguration detection. **Premium** adds active threat detection, deeper vulnerability scanning, and compliance-mapping reports, at additional cost. Most organizations treat Standard as baseline hygiene and adopt Premium once they need continuous threat monitoring at scale.

A Worked Example

Suppose SCC surfaces a finding titled “Public bucket ACL” against `my-app-assets`, or “Firewall rule allows all IPs” against a rule permitting port 22 traffic from anywhere. Fix the bucket finding by correcting its IAM policy and legacy ACLs (Chapter 8); fix the firewall finding by tightening the source range or using Identity-Aware Proxy (IAP) tunneling instead of open SSH (Chapter 12). SCC does not fix these for you — it tells you they exist and gives enough context to remediate them.

WARNING

A common, painful surprise during a security incident is discovering Data Access logs were never enabled for the service in question — so there is no record of what data was actually read or exfiltrated, only that someone touched the resource. Admin Activity logs cannot tell you what a compromised credential read; only Data Access logs can, and only if turned on beforehand.

Best Practices

- Enable Data Access logs proactively for sensitive services rather than waiting until an incident forces the question.
- Review SCC findings on a regular cadence rather than only reactively.
- If compliance requires retention beyond 400 days, export audit logs to a long-retention sink, as covered in Chapter 15.

Key Takeaways

- Cloud Audit Logs split into Admin Activity (always on, free), Data Access (off by default), and System Event (Google-generated) log types.
- Admin Activity logs are immutable and retained for 400 days regardless of other settings, so they remain trustworthy even after a compromise.
- Data Access logs must be enabled before an incident occurs — their absence is one of the most common gaps discovered during a security investigation.
- Security Command Center aggregates misconfigurations, vulnerabilities, and threats into one dashboard, with a free Standard tier and a paid Premium tier adding threat detection.
- Remediating an SCC finding means applying IAM, Cloud Storage, or networking controls covered elsewhere in this book — SCC's job is detection, not automatic remediation.

Chapter 20: Billing, Budgets & Cost Management

Every resource in Google Cloud Platform (GCP) costs money, and eventually someone has to pay the bill. This chapter covers how GCP structures billing, where to see spend, how to set up alerts before spending gets out of hand, and the related-but-distinct concept of quotas, which limit how much of a resource you can consume regardless of budget.

The Billing Account

A billing account is the payment entity in GCP — it holds a payment method (credit card or invoicing arrangement) and is linked to one or more projects. Recall from Chapter 2 that a project is the basic unit of resource organization in GCP’s hierarchy; a billing account sits alongside that hierarchy as the thing that actually gets charged. One billing account can fund many projects, a common pattern for centralized financial control with separate technical boundaries between, say, production and staging. Conversely, a project must be linked to exactly one billing account to create billable resources; without one, most resource creation calls fail immediately.

Every billing account has a unique ID, formatted like 012345-6789AB-CDEF01, used whenever you interact with billing through `gcloud`.

Finding Current Spend

The Billing Reports page in the GCP Console is the primary place to see where money is going. It lets you filter spend by project, by service (Compute Engine, Cloud Storage, BigQuery, and so on), by SKU (Stock Keeping Unit — the finest-grained billable line item, such as “N2 Instance Core running in Americas”), and by label, covered below. This view answers the question every finance team eventually asks: why did the bill go up, and which team caused it?

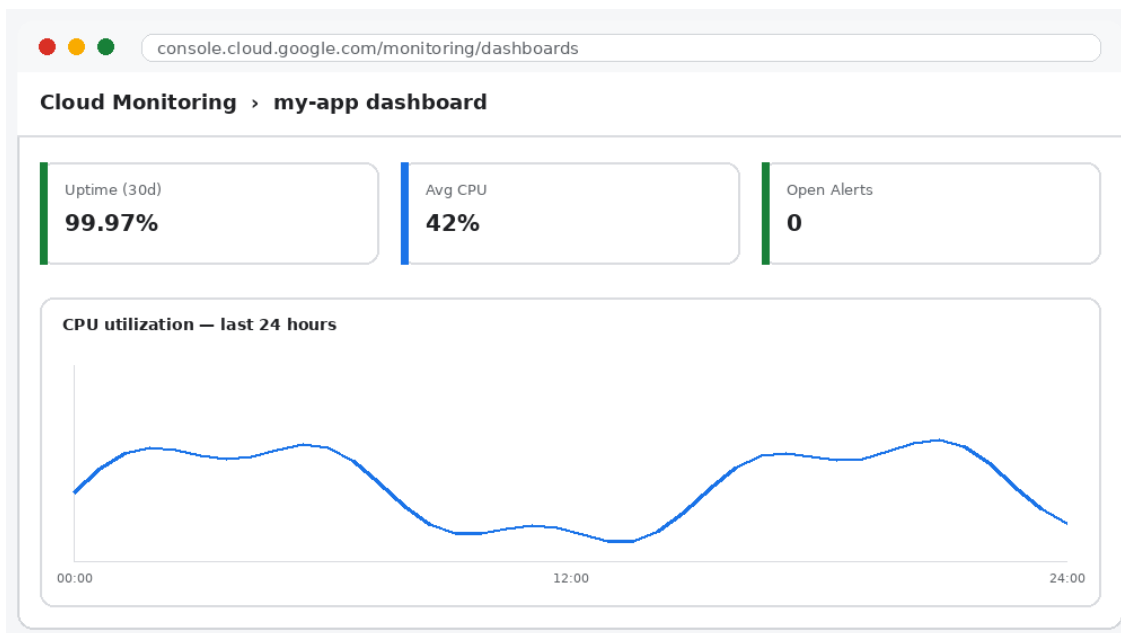


Figure 21: Illustrative a budget dashboard showing spend and forecast against the configured budget.

Budgets and Budget Alerts

A budget is a target spending amount tied to a billing account (or scoped to specific projects), paired with one or more alert thresholds. When actual or forecasted spend crosses a threshold, GCP sends a notification — but, critically, it does not stop anything from running.

```
gcloud billing budgets create \
  --billing-account=012345-6789AB-CDEF01 \
  --display-name="Monthly Budget" \
  --budget-amount=1000 \
  --threshold-rule=percent=0.5 \
  --threshold-rule=percent=0.9 \
  --threshold-rule=percent=1.0
```

- `--billing-account` identifies the billing account this budget applies to, using its ID.
- `--display-name` gives the budget a human-readable label, useful when one billing account has several budgets.
- `--budget-amount=1000` sets the target monthly spend, in the account's currency — here, \$1,000.
- `--threshold-rule=percent=0.5` (and the two after it) define alert points at 50%, 90%, and 100% of budget; each crossing triggers a notification, typically by email and, if configured, a Pub/Sub message.

WARNING

A budget alert is a notification, not an enforcement mechanism. Reaching 100% does not pause billing or stop VMs — it only sends an email or Pub/Sub message. For actual spending caps, build automation on top of that notification yourself, such as a Cloud Function that disables billing or stops non-critical VMs when a threshold fires.

Labels for Cost Attribution

Labels are key-value pairs attached to resources that let you slice billing reports by team, environment, cost center, or any dimension your organization cares about. Unlike free-form Compute Engine tags used for networking rules (Chapter 12), labels are designed to flow through into billing data.

```
gcloud compute instances create web-server-1 \
  --project=my-gcp-project-123456 \
  --zone=us-central1-a \
  --machine-type=e2-medium \
  --labels=team=platform,env=production,cost-center=cc-4021
```

- `--labels=team=platform,env=production,cost-center=cc-4021` attaches three labels at creation time, each a key-value pair with no predefined schema — agree on a labeling convention before resources accumulate.
- These labels appear as filterable, groupable dimensions in the Billing Reports page, letting you answer “how much did the platform team spend on production compute last month?” directly.

BEST PRACTICE

Apply a consistent labeling scheme from the first resource you create. Retrofitting labels onto hundreds of existing resources later is tedious and error-prone; enforcing labels at creation time (via an organization policy or a Terraform module, as covered in the companion “Terraform on GCP” guide) costs almost nothing.

Discounts: Sustained Use and Committed Use

GCP offers two distinct mechanisms that reduce compute costs.

Sustained use discounts are automatic. If a VM runs for a significant portion of a billing month, GCP applies an increasing discount to that usage — no commitment, no configuration, nothing to opt into. It simply rewards workloads that run continuously rather than briefly.

Committed use discounts require committing to a specific amount of compute usage for a fixed

1-year or 3-year term, in exchange for a substantially larger discount. The trade-off: a lower price for giving up flexibility, since the commitment applies whether or not you use that capacity. These make sense once you have a stable baseline of usage — a production fleet that stays roughly the same size month over month — rather than for workloads still growing or shrinking unpredictably.

TIP

Review actual usage patterns in the Billing Reports page for a few months before purchasing a committed use discount. Committing based on a guess, rather than observed steady-state usage, is a common way to end up paying for capacity you don't use.

Quotas: A Related but Distinct Concept

Quotas are resource limits — caps on how many of a resource type you can create, or how much of an operation you can perform — independent of budget. A project might have plenty of budget remaining and still fail to create a new VM because it hit its regional CPU quota, or its API-requests-per-minute quota. Quotas protect shared infrastructure from runaway usage and give Google predictable capacity signals; they do not control cost.

You can view many of a project's quotas directly:

```
gcloud compute project-info describe --project=my-gcp-project-123456
```

- This returns current Compute Engine quota usage and limits, such as CPUS, IN_USE_ADDRESSES, and INSTANCES, broken out by region where applicable.
- For a filterable view across all services, use the Quotas page in the GCP Console, which also lets you submit an increase request.

NOTE

Quota increase requests are not instantaneous — some approve automatically within minutes, while others (large increases, or scarce hardware like GPUs) require manual review that can take days. Treat quota increases as proactive planning, not a same-day fix.

Troubleshooting

- **A budget alert fired, but spend keeps climbing.** Expected — budget alerts notify only. For automatic enforcement, build it yourself on top of the budget's Pub/Sub notifications.
- **A “stopped” VM still generates charges.** As covered in Chapter 4, stopping a VM halts compute charges but its attached persistent disk keeps accruing storage charges until deleted or detached — a common source of a lingering, unexplained charge on an idle project.
- **A deploy fails with a quota-exceeded error despite plenty of budget remaining.** Check the relevant quota with `gcloud compute project-info describe` or the Quotas page — this is a resource limit issue, not a billing one, requiring a quota increase rather than any billing change.

Best Practices

- Set budget alerts on every billing account from day one — a rough estimate beats no signal at all.
- Use labels consistently from the start, since retrofitting cost attribution onto an unlabeled environment later is far more painful.
- Once usage patterns stabilize, regularly review committed use discount opportunities rather than leaving steady-state workloads on pure on-demand pricing indefinitely.
- Treat quota increases as a proactive request made well ahead of when you'll actually need the headroom, not a same-day fix.

Key Takeaways

- A billing account is the payment entity linked to one or more projects; a project must be linked to one to create billable resources.
- Budgets and budget alerts notify stakeholders when spend crosses configured thresholds, but never automatically stop spending on their own.
- Labels attached to resources at creation time make cost attribution by team, environment, or cost center possible in the Billing Reports page.
- Sustained use discounts apply automatically to continuously running compute usage, while committed use discounts require a 1- or 3-year commitment for a larger discount.
- Quotas are resource limits, not cost controls, and can unexpectedly block scaling even when budget is not a constraint — check and request increases proactively.

Chapter 21: Cloud Build & CI/CD Fundamentals on GCP

Writing code is only part of shipping software — it also needs to be tested, packaged, and moved toward production reliably and repeatably. This chapter introduces Cloud Build, GCP’s managed service for automating that process, and the broader CI/CD (Continuous Integration / Continuous Delivery) concepts it supports.

What Cloud Build Does

Cloud Build is a managed CI/CD execution service: you give it a sequence of build steps, and it runs each one inside its own container, in order, on infrastructure Google manages so you never have to provision or patch a build server yourself. The sequence of steps is defined in a file named `cloudbuild.yaml`, which lives alongside your source code.

Here is a complete, runnable `cloudbuild.yaml` that runs a test suite and then packages the result into a tarball artifact:

```
steps:
- name: 'python:3.12-slim'
  id: 'install-dependencies'
  entrypoint: 'pip'
  args: ['install', '-r', 'requirements.txt', '--target', '/workspace/deps']

- name: 'python:3.12-slim'
  id: 'run-tests'
  entrypoint: 'python'
  args: ['-m', 'pytest', 'tests/']
  env:
    - 'PYTHONPATH=/workspace/deps'

- name: 'ubuntu'
  id: 'package-artifact'
  entrypoint: 'bash'
  args:
    - '-c'
    - 'tar -czf app-$SHORT_SHA.tar.gz ./src ./deps'

artifacts:
  objects:
    location: 'gs://my-app-assets/builds/'
    paths: ['app-$SHORT_SHA.tar.gz']

timeout: '1200s'
```

- `steps` is an ordered list; each step runs in its own container image (name), so `install-dependencies` uses a Python image while `package-artifact` uses a plain Ubuntu image just to run `tar`.
- `id` labels each step so its logs and status are easy to find individually.
- `entrypoint` and `args` override the container’s default command, letting a generic image (like `python:3.12-slim`) run whatever tool you need — here, `pip` and then `pytest`.
- `env` passes environment variables into a step; here it makes the freshly installed dependencies importable during the test run.
- `$SHORT_SHA` is a Cloud Build substitution, automatically populated with the short Git commit hash that triggered the build, used here to name the artifact uniquely per commit.
- `artifacts` tells Cloud Build to upload the resulting tarball to a Cloud Storage bucket (`my-app-assets`) once all steps succeed.
- `timeout` caps the whole build at 1,200 seconds (20 minutes); a longer build is automatically canceled, preventing a hung step from consuming build minutes indefinitely.

NOTE

This chapter keeps the example generic. Building and pushing container images to Artifact Registry, and deploying to Google Kubernetes Engine (GKE), is covered in full in the companion “Docker, Kubernetes, GKE & GCP Artifact Registry” guide — Cloud Build works the same way there, just with `docker build` and `docker push` steps in place of the `tarball` step shown here.

Build Triggers

A build trigger connects Cloud Build to a source repository — Cloud Source Repositories, GitHub, or GitLab — so that a push or a pull request automatically starts a build, without anyone running `gcloud builds submit` by hand.

```
gcloud builds triggers create github \
  --repo-name=my-app \
  --repo-owner=my-org \
  --branch-pattern="^main$" \
  --build-config=cloudbuild.yaml
```

- `github` specifies the source of truth for this trigger; `gcloud builds triggers create` also supports Cloud Source Repositories and, via a connected host, GitLab.
- `--repo-name` and `--repo-owner` identify the GitHub repository to watch.
- `--branch-pattern="^main$"` is a regular expression matched against the branch name; here it fires only on pushes to `main`, deliberately excluding feature and pull-request branches.
- `--build-config=cloudbuild.yaml` points at the build definition file to run whenever the trigger fires.

BEST PRACTICE

Set branch patterns deliberately rather than leaving a trigger to match everything. A trigger firing on every branch push burns build minutes on work-in-progress branches and surfaces confusing, half-finished failures to the whole team; scope each trigger to the branches that actually warrant an automatic build.

Substitutions and Build Variables

Substitutions let you parameterize a build so the same `cloudbuild.yaml` behaves correctly across many commits and branches, without editing the file itself. Cloud Build provides a set of built-in substitutions automatically, including:

- `$COMMIT_SHA` — the full Git commit hash that triggered the build.
- `$SHORT_SHA` — the first several characters of `$COMMIT_SHA`, often more convenient for naming artifacts or image tags.
- `$BUILD_ID` — the unique identifier Cloud Build assigns to this build run, useful for looking up logs later.
- `$BRANCH_NAME` — the branch that triggered the build, useful when one trigger matches multiple branches.

You can also define custom substitutions (prefixed with an underscore, such as `_ENVIRONMENT`) and pass different values per trigger, letting one `cloudbuild.yaml` serve multiple environments.

Viewing Build Logs

Every build produces logs for each step, viewable through the Console or the command line:

```
gcloud builds log BUILD_ID
```

- Replace `BUILD_ID` with the identifier Cloud Build assigned when the build started; find it in the output of `gcloud builds submit`, the trigger’s build history in the Console, or `gcloud builds`

list.

- The log output concatenates each step's container output in order, making it easy to see exactly which step failed and why.

CI Versus CD, and Where Cloud Build Fits

Continuous Integration (CI) is the practice of automatically building and testing every change as soon as it is pushed, so integration problems are caught within minutes rather than discovered weeks later when several people's changes collide. Cloud Build, as described in this chapter, is fundamentally a CI engine: it runs steps, in containers, triggered by a source change.

Continuous Delivery / Deployment (CD) is the practice of automatically promoting a build that has passed CI toward production — deploying it to a staging environment, running further checks, and eventually rolling out to production, either automatically (continuous deployment) or with a manual approval gate (continuous delivery). Cloud Build can include deployment steps directly in its `cloudbuild.yaml` — for example, a final step running `kubectl apply` or a Helm upgrade against a GKE cluster, as covered in the companion Kubernetes guide — but for more structured, multi-stage rollout pipelines, GCP also offers Cloud Deploy, a purpose-built CD service that manages promotion through a defined sequence of target environments with approval gates and rollback support.

In short: Cloud Build is the CI engine that runs on every change, while either simple deployment steps in `cloudbuild.yaml` or a dedicated service like Cloud Deploy handles the CD half of the pipeline.

Troubleshooting

- **A build fails with a permission-denied error when writing to a bucket or registry.** Cloud Build runs each build as a service account — by default, the project's Cloud Build service account — subject to the exact same IAM rules as any other principal covered throughout this book. If a step writes to `gs://my-app-assets/` or pushes to a registry, that service account needs an appropriate role (such as `roles/storage.objectCreator`) granted explicitly; it gets no broad access just for being Google-managed.
- **A trigger never fires after a push.** Confirm the branch pattern actually matches the branch pushed to, and that the repository connection (GitHub App installation, or Cloud Source Repositories mirroring) is still authorized — a revoked or expired connection silently stops triggers.
- **A build times out on a step that normally finishes quickly.** This is often a step waiting on a resource it lacks permission to reach, or a now-unreachable network call, rather than a real performance regression; check that step's log output for what it was doing when the timeout hit.

Best Practices

- Keep build steps small, single-purpose, and cacheable, so a failure isolates cleanly to one step and unrelated steps don't need to rerun unnecessarily.
- Never embed secrets directly in `cloudbuild.yaml` — instead, reference Secret Manager (covered in Chapter 17) so credentials are fetched securely at build time and never committed to source control.
- Use branch patterns deliberately on every trigger so builds fire only where they are actually meant to run.

Key Takeaways

- Cloud Build runs a sequence of containerized steps defined in `cloudbuild.yaml`, giving you a managed, serverless CI execution environment with no build server to maintain.
- Build triggers connect a source repository (Cloud Source Repositories, GitHub, or GitLab) so a push or pull request starts a build automatically, scoped by branch pattern.
- Built-in substitutions like `$COMMIT_SHA` and `$BUILD_ID`, plus custom `_VARIABLE` substitutions, let one build definition serve many commits, branches, and environments.
- Cloud Build itself runs as a service account bound by ordinary IAM rules — a permission-denied

- build failure almost always means that service account needs an explicit role grant.
- Cloud Build is the CI engine; the CD half of a pipeline is handled either by deployment steps within the same build or by a dedicated service like Cloud Deploy for structured, multi-stage promotion.

Chapter 22: High Availability, Disaster Recovery & Production Best Practices

Every chapter up to this point has focused on a single service in isolation: how to configure Compute Engine, how to wire up a VPC, how to make Cloud SQL highly available. Production systems, however, are judged as a whole, under failure — not service by service. A perfectly configured load balancer does no good if the database behind it has no standby, and a database with automatic failover does no good if the application layer cannot survive a single zone going dark. This chapter zooms out and designs for failure rather than the happy path, tying the services from Chapters 2 through 21 into one coherent production architecture.

Two metrics drive almost every architecture decision in this space:

- **RTO (Recovery Time Objective)** answers “how long can this system be down before the outage is unacceptable to the business?” An RTO of four hours means a four-hour outage is tolerable, but anything longer is not.
- **RPO (Recovery Point Objective)** answers “how much data can we afford to lose, measured in time?” An RPO of fifteen minutes means that, in the worst case, the last fifteen minutes of writes before a failure might be gone, and that has been judged acceptable.

These numbers directly determine which architecture is affordable and which is overkill. An RPO of zero (no data loss under any failure) effectively requires synchronous replication, where a write is not acknowledged until durably recorded in a second location — exactly what Cloud SQL’s REGIONAL availability type does across zones. Synchronous replication is expensive in both cost and write latency, since every write waits on a round trip to a second copy. An RPO of a few hours, by contrast, allows much cheaper asynchronous backups or replicas, where a small amount of recent data could theoretically be lost, but that risk has been judged acceptable given the savings. Neither answer is universally “correct” — which is exactly why RTO and RPO are defined before the architecture is designed, not after.

Reference Architecture Walkthrough

To make these ideas concrete, walk through a single user request as it flows through a realistic production architecture built entirely from services already covered in this book.

1. **Cloud DNS (Chapter 10)** resolves the application’s public domain name to the load balancer’s anycast IP. As a globally distributed, Google-managed name service, this step is itself highly available with no extra configuration.
2. **Google Cloud Armor** sits at the network edge, in front of the load balancer, inspecting requests before they reach application infrastructure. It provides Web Application Firewall (WAF) rules against common exploits (SQL injection, cross-site scripting) and absorbs distributed denial-of-service (DDoS) traffic, dropping malicious requests at Google’s edge.
3. **A global external Application Load Balancer (Chapter 9)** receives traffic passing Cloud Armor’s inspection and, using its single anycast IP, routes each request to the nearest healthy backend, factoring in geographic proximity and current load.
4. **A regional Managed Instance Group (Chapter 6)** provides the compute layer behind that backend service, running instances spread automatically across multiple zones. If a zone fails, the MIG’s remaining zones keep serving traffic while GCP replaces the lost capacity — no manual intervention required.
5. **A Cloud SQL instance in REGIONAL (HA) mode (Chapter 13)** serves as the primary datastore, with a synchronously replicated standby in a second zone ready to take over if the primary becomes unhealthy.
6. **Cloud Storage (Chapters 11-12)** serves static assets directly, offloading that traffic from compute, and separately holds automated database and configuration backups in a versioned bucket.
7. **Cloud KMS (Chapter 17)** manages the encryption keys protecting data at rest across the stack — Cloud SQL storage, Cloud Storage buckets, and application secrets — so one auditable key layer sits underneath every stateful component.

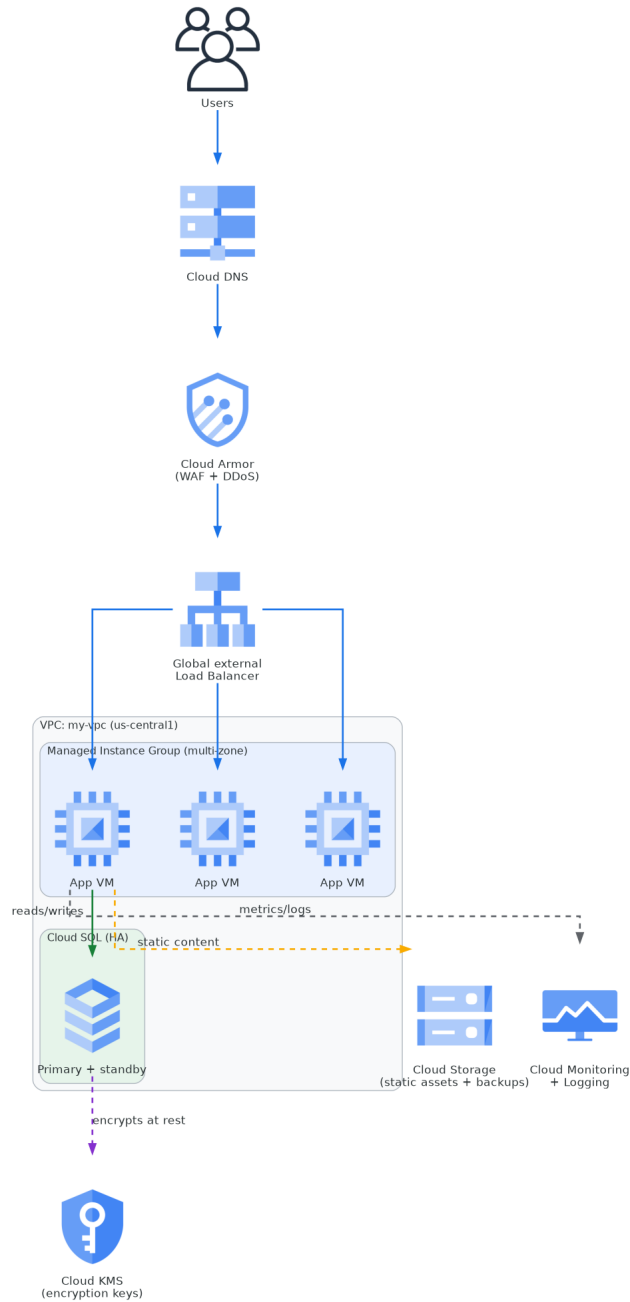


Figure 22: A production reference architecture: DNS, Cloud Armor, and a global load balancer in front of a multi-zone instance group backed by an HA Cloud SQL instance, with storage, KMS, and monitoring layered in.

8. **Cloud Monitoring and Cloud Logging (Chapter 18)** collect metrics and logs from every layer above, so a zone failure, a database failover, or a spike in Cloud Armor blocks is visible on a dashboard and can trigger an alert, rather than being discovered when a user complains.

No single piece of this architecture is exotic. What makes it a production architecture rather than a collection of services is that each layer's failure mode has been considered and addressed by the layer around it.

High Availability Patterns

High availability (HA) is about surviving the failure of infrastructure you do not control — a host, a rack, or an entire zone — without an outage. GCP gives you two broad postures to choose between:

- **Multi-zone** protects against the failure of a single zone by spreading redundant capacity across two or more zones within one region. It is the cheaper option, adds relatively little complexity, and is the default recommendation for the overwhelming majority of production workloads.
- **Multi-region** protects against the failure of an entire region — a much rarer event, but not an impossible one. It is meaningfully more expensive (duplicate infrastructure in a second geography) and more complex (cross-region networking, consistency across greater distances), and should be reserved for workloads with a genuine, articulated business requirement for it.

BEST PRACTICE

Default to multi-zone HA for production workloads unless a specific, documented business requirement justifies the added cost and complexity of multi-region.

A regional MIG and a REGIONAL Cloud SQL instance both sit at the multi-zone end of this spectrum, each keeping standby or extra capacity in a second zone for low-latency synchronous coordination. Cloud SQL has no built-in multi-region HA mode, so multi-region resilience for relational data relies instead on asynchronous cross-region read replicas, trading a small replication lag for broader reach. Cloud Spanner (Chapter 14), by contrast, was built for multi-region operation from the ground up: its regional and multi-regional configurations use synchronous, globally consistent replication as a first-class feature — exactly why Spanner costs more and is reserved for workloads that need global, strongly consistent scale.

Disaster Recovery Strategies

Disaster recovery (DR) is the discipline of recovering when HA was not enough — an event large enough to take down the primary environment entirely. Four standard patterns exist, in increasing order of cost and complexity, and correspondingly decreasing RTO and RPO:

- **Backup and restore**, the cheapest pattern with the highest RTO: no standby runs at all, and recovery means provisioning new infrastructure and restoring from backups. Cloud SQL's automated backups and point-in-time recovery, with Cloud Storage as the destination, underpin this pattern.
- **Pilot light** keeps a minimal, low-cost environment running at all times — typically just the data layer — with compute switched off until needed. A cross-region Cloud SQL read replica kept warm and ready to promote is a textbook example.
- **Warm standby** runs a smaller-scale but fully functional replica of the whole application continuously, so recovery means scaling it up rather than building from scratch. A low-instance-count regional MIG paired with a standby replica in a second region fits this pattern.
- **Multi-site active-active**, the most expensive pattern, runs full-scale infrastructure in more than one region simultaneously, all actively serving traffic. It delivers the lowest achievable RTO and RPO, near zero, and is the use case Spanner's multi-region configurations and a global Application Load Balancer support together.

Production Checklist

Before calling any architecture production-ready, work through a checklist drawing on best practices from across this book:

- IAM roles follow least privilege using predefined or custom roles, never the broad Owner or Editor basic roles.
- Every stateful service that supports it (Cloud SQL, Spanner) runs in a regional or multi-region HA configuration, not single-zone.
- Firewall rules restrict source ranges to known networks, instead of defaulting to 0.0.0.0/0.
- Service accounts use workload identity federation or short-lived credentials rather than downloaded JSON keys.
- Data at rest uses customer-managed encryption keys (CMEK) in Cloud KMS wherever compliance requires controlling the keys.
- Secrets (database passwords, API keys) live in Secret Manager, not environment variables or source code.
- Budgets and budget alerts are configured on the billing account so cost spikes are caught early, not at month end.
- Audit logging, including Data Access logs, is explicitly enabled for sensitive services, since it is not on by default.
- Automated backups exist for every stateful service and have actually been test-restored at least once.
- Cloud Monitoring dashboards and alerting policies exist for every production service, covering infrastructure health and business metrics.

WARNING

A backup that has never been restored is not a verified backup — schedule periodic test restores, not just backup creation, for any service your DR plan depends on.

Architecture is always a trade-off between cost, complexity, and resilience; the reference design here is not a mandate to build the most resilient system for every workload. A low-traffic internal tool with a generous RTO and RPO may be well served by a single-zone Compute Engine instance and nightly backups; a payments system with an RPO near zero may genuinely justify Spanner and multi-region active-active. The lesson is not “always build the most resilient thing” — it is knowing your actual RTO and RPO, and choosing the cheapest architecture that meets them.

Chapter 23: Interview Questions

This chapter collects interview-style questions spanning this book, from resource hierarchy fundamentals through disaster recovery architecture, doubling as a quick-reference review.

Q1. What is the GCP resource hierarchy, and why does it matter?

Organization, then Folders, then Projects, then individual resources. IAM policies set at the Organization or Folder level are inherited by everything beneath, so a binding placed high up can have broader reach than expected. Structuring folders around departments or environments lets policies apply once at the right level instead of repeating per project.

Q2. What is the difference between IAM roles, permissions, and policies?

A permission is a fine-grained capability, like `compute.instances.start`, that cannot be granted directly. A role is a named bundle of permissions, such as `roles/compute.instanceAdmin`. A policy binding connects a role to a member on a specific resource; you grant roles, never raw permissions.

Q3. What is the difference between basic, predefined, and custom IAM roles?

Basic roles (Owner, Editor, Viewer) are broad legacy roles to avoid in production. Predefined roles are curated by Google for a specific service, like `roles/storage.objectViewer`, and are the recommended default. Custom roles assemble a bespoke permission set when nothing predefined fits, at the cost of maintaining it yourself.

Q4. What is a service account, and how does it differ from a user account?

A service account is an identity for a workload rather than a human — a VM or CI/CD pipeline authenticating without a person's credentials. It has no password and typically authenticates via short-lived tokens issued to the resource it is attached to. Because a leaked key is a serious risk, best practice avoids downloading JSON keys wherever possible.

Q5. What is workload identity, and why is it preferred over downloaded keys?

Workload identity lets a workload present its own native credentials, mapped automatically to a service account's permissions, without ever handling a downloaded key. A leaked JSON key works from anywhere indefinitely until revoked, while workload identity credentials are short-lived — the recommended pattern for GKE and external CI/CD alike.

Q6. What identifiers does every GCP project have?

A project ID (globally unique, human-chosen, immutable, like `my-gcp-project-123456`), a project number (assigned automatically), and a display name (not unique, changeable anytime) — every resource and every billed dollar is scoped to a project.

Q7. What is the difference between a zone, a region, and a multi-region?

A zone is a single isolated deployment area within a region; a VM lives in one zone. A region groups several zones with low-latency links between them; a REGIONAL Cloud SQL instance spans zones within one region. A multi-region spans several distant regions for maximum resilience, at greater cost.

Q8. How do you choose a Compute Engine machine family?

General-purpose families (E2, N2, N4) suit most workloads, compute-optimized (C2, C3) suit CPU-intensive work, memory-optimized (M-series) suit in-memory databases, and accelerator-optimized (A-series) suit GPU workloads. Identify the actual bottleneck first, then pick the smallest matching type and adjust based on monitoring data.

Q9. What does a Managed Instance Group (MIG) add beyond individual VMs?

A MIG manages identical instances from one template and adds autoscaling, autohealing (recreating instances that fail their health check), and rolling updates. A regional MIG also spreads instances

across multiple zones automatically — the standard way to get multi-zone HA.

Q10. How does a VPC differ from a traditional on-premises network?

A VPC is a global, software-defined network whose subnets can span multiple regions worldwide while routing between them privately by default. Subnets, however, are regional: each subnet's IP range lives in one specific region even though the VPC itself is global.

Q11. How do GCP firewall rules work, and what is the default-deny behavior?

Rules are stateful, evaluated by direction, priority, and criteria like source ranges, protocols, and ports. Every VPC has implied lowest-priority rules that deny all ingress and allow all egress, so nothing reaches an instance unless an explicit ALLOW rule permits it. Keep rules narrow — specific source ranges and target tags rather than 0.0.0.0/0.

Q12. What is Cloud NAT, and when do you need it?

Cloud NAT lets instances with only private IPs make outbound internet connections — for patches or API calls — without ever having a public IP. It is used for zero inbound exposure while still allowing outbound access, standard for backend and database tiers.

Q13. What are the main options for connecting on-premises networks to a VPC?

Cloud VPN gives encrypted IPsec connectivity over the public internet — quick to set up, subject to internet variability. Cloud Interconnect gives a direct private (Dedicated) or partner-mediated (Partner) connection with higher, more predictable bandwidth at greater cost. Choose based on bandwidth and budget.

Q14. Global external Application Load Balancer versus regional external Network Load Balancer?

The Application Load Balancer operates at Layer 7 (HTTP-aware) behind one global anycast IP, routing users to their nearest healthy backend region. The Network Load Balancer operates at Layer 4 (TCP/UDP only) and is scoped to one region. Use the former for public HTTP(S) services needing global reach and CDN; use the latter for simpler, non-HTTP traffic.

Q15. Every backend shows UNHEALTHY even though the app works fine directly — why?

Almost always a firewall issue: health-check probes originate from 130.211.0.0/22 and 35.191.0.0/16, and a rule allowing normal user traffic does not automatically allow those ranges through. The fix is a dedicated rule with --source-ranges=130.211.0.0/22,35.191.0.0/16 on the same target tags. Also confirm the health check's port matches what the application listens on.

Q16. How does Cloud DNS achieve high availability, and what is a private zone for?

Cloud DNS is globally distributed and Google-managed, so its availability needs no configuration from the user. A public zone answers internet queries for a domain you control; a private zone answers queries only within specified VPCs, resolving internal hostnames privately.

Q17. What are Cloud Storage's main storage classes?

STANDARD for frequent access, NEARLINE for roughly monthly access, COLDLINE for a few times a year, and ARCHIVE for less than once a year. Each colder tier trades lower storage cost for higher retrieval cost and a longer minimum duration.

Q18. Uniform bucket-level IAM versus fine-grained object ACLs?

Uniform bucket-level access applies one IAM policy to every object, so permissions stay consistent — the recommended default. Fine-grained access (legacy ACLs) lets individual objects carry their own rules on top of the bucket policy, offering finer control but making auditing far harder.

Q19. What is a signed URL, and when would you use one?

A signed URL grants time-limited, credential-free access to a specific object, generated with a service account's private key encoding the permission and expiration. It hands a user a temporary download

or upload link without making the object or bucket public.

Q20. When would you choose Cloud SQL over Cloud Spanner?

Cloud SQL suits relational workloads that fit on a single primary node (plus read replicas for read scaling) and need standard MySQL, PostgreSQL, or SQL Server compatibility. Spanner is reserved for a demonstrated need for horizontal write scaling with strong consistency across a very large or global dataset — its added cost is only justified when a relational instance genuinely cannot fit.

Q21. What does `--availability-type=REGIONAL` do for a Cloud SQL instance?

It provisions a synchronously replicated standby in a second zone, so a write is not complete until durably recorded on both. If the primary's zone fails, Cloud SQL fails over automatically within a minute or two with no application change beyond reconnecting. ZONAL, the alternative, runs a single instance with no standby.

Q22. Cloud SQL read replica versus HA standby — what's the difference?

An HA standby exists purely for failover, is synchronously in sync, and cannot serve reads during normal operation. A read replica is asynchronously replicated, can live in a different region, and actively serves read-only queries, at the cost of a small replication lag. HA addresses failure; replicas address read scaling and disaster recovery.

Q23. What is the recommended way to connect an application to Cloud SQL?

The Cloud SQL Auth Proxy, authenticating via IAM and establishing an encrypted tunnel automatically. Authorized networks (an IP allowlist) is simpler but brittle in dynamic environments and requires manual TLS management. Private IP over the VPC is another strong alternative, avoiding a public IP entirely.

Q24. What is Firestore, and how does it differ from Cloud SQL?

Firestore is a fully managed, serverless NoSQL document database storing data as documents in collections, scaling horizontally without capacity planning. It suits mobile and web backends with flexible data and real-time sync needs, whereas Cloud SQL suits workloads needing SQL joins, transactions, and a fixed schema.

Q25. What is Memorystore, and what problem does it solve?

Memorystore is GCP's fully managed in-memory data store (Redis or Memcached), caching frequently accessed data to reduce latency and load on a backend database. It removes the operational burden of patching and failover for the caching layer, typically placed in front of Cloud SQL so repeated reads hit memory, not the database.

Q26. What is Cloud KMS, and what is CMEK?

Cloud KMS is GCP's managed service for creating, storing, rotating, and controlling access to cryptographic keys without exposing key material directly. CMEK means using a Cloud KMS key you control, instead of Google's default, to encrypt data at rest — giving you control over rotation and the ability to revoke the key, instantly rendering data unreadable.

Q27. What is Secret Manager, and why not just use environment variables?

Secret Manager stores sensitive values with fine-grained IAM control, automatic versioning, and audit logging. Environment variables and source code leak easily into logs, version control, or image layers, and offer no built-in access control or audit trail.

Q28. Cloud Monitoring versus Cloud Logging — what's the difference?

Cloud Monitoring collects numeric time-series metrics (CPU, latency, error rates) and hosts dashboards and alerting policies. Cloud Logging collects log entries and is where you find the specific event explaining why a metric moved — Monitoring tells you something is wrong, Logging tells you why.

Q29. What are Cloud Audit Logs, and why do Data Access logs need manual enabling?

Audit Logs split into Admin Activity (configuration changes, always on), Data Access (reads and writes of data itself), and System Event logs. Data Access logs are off by default due to volume and cost, so workloads handling sensitive data must explicitly enable them for a full audit trail.

Q30. What is Security Command Center (SCC)?

SCC is GCP's centralized security dashboard, aggregating findings — misconfigurations, vulnerabilities, active threats — across an organization's projects. Built-in detectors surface issues like publicly exposed buckets or overly permissive IAM bindings, giving security teams a single pane of glass.

Q31. How do budgets and budget alerts work on a billing account?

A budget is configured on a billing account, or scoped to projects, with a defined amount; alerts trigger notifications when actual or forecasted spend crosses thresholds like 50, 90, and 100 percent. Budgets are notifications only — they do not stop spending, so a hard cap requires added automation, such as a Cloud Function triggered by a budget alert.

Q32. What is Cloud Build, and how does it fit a GCP CI/CD pipeline?

Cloud Build runs steps from a `cloudbuild.yaml` file in ephemeral containers, commonly triggered by a push to a connected repository. A typical pipeline runs tests, builds a container image, pushes it to Artifact Registry, and deploys it to Compute Engine or GKE, without maintaining a build server.

Q33. What is the difference between RTO and RPO?

RTO is how long a system can be down before that is unacceptable; RPO is how much data, measured in time, can be lost. A near-zero RPO requires synchronous replication (a REGIONAL Cloud SQL instance), costing more in latency and infrastructure, while a looser RPO of hours permits cheaper asynchronous backups. Both should be defined before the architecture is designed.

Q34. Multi-zone versus multi-region high availability?

Multi-zone HA protects against a single zone's failure by spreading capacity across zones in one region — cheaper, simpler, the default for most production workloads. Multi-region HA protects against losing an entire region, at higher cost and complexity, reserved for workloads with a genuine business need, such as those built on Spanner's multi-region configurations.

Q35. Name the four standard disaster recovery patterns, cheapest to most expensive.

Backup and restore (no standby, restore from backups on disaster), pilot light (minimal standby with just the data layer kept warm), warm standby (a smaller but fully functional replica running continuously, scaled up on failover), and multi-site active-active (full-scale infrastructure in multiple regions simultaneously serving traffic, for the lowest RTO and RPO).

Q36. How would you decide between Cloud SQL and Spanner for a new project?

Start with actual scale and consistency needs, not assumptions: if the dataset and write throughput fit on a single well-sized primary, Cloud SQL is simpler and cheaper. Reach for Spanner only with a demonstrated need for horizontal write scaling with strong consistency, or a multi-region consistency requirement async replicas cannot satisfy.

Q37. A team's GCP bill spiked three times overnight — how would you investigate?

Start in Cloud Billing reports, broken down by project, then service and SKU, to identify what got more expensive. Cross-reference the timing against Cloud Logging and Admin Activity audit logs for a recent deployment, an oversized resource, or a runaway autoscaling group.

Q38. A production Cloud SQL instance is completely unreachable — what is your triage process?

First confirm the instance's state with `gcloud sql instances describe` — `RUNNABLE` versus a failure or maintenance state shows whether this is an instance or connectivity problem. If healthy, check

the connection path: whether the Auth Proxy is running, or the client's IP is in authorized networks, since a missing entry or a proxy that never started are the usual causes. If mid-failover, a REGIONAL instance should recover within a minute or two.

Q39. How would you design the compute and data tier for a growing public-facing web app?

Start with a regional MIG behind a global Application Load Balancer, with Cloud Armor for WAF and DDoS protection — inexpensive, and covers the most likely failure mode, a single zone outage. Pair it with a REGIONAL Cloud SQL instance as the primary datastore, adding a read replica only once monitoring shows it is needed, and use Cloud Storage for static assets with CMEK if compliance requires it. This gives solid multi-zone resilience at modest cost, with a clear path to Spanner or multi-region active-active if growth ever justifies it.

Chapter 24: Commands Cheat Sheet

Every chapter in this book has introduced `gcloud` commands one at a time, in context, alongside the concepts they configure. This chapter collects the ones you will reach for most often into a single, scannable reference — organized by service area rather than by chapter, so you can find the syntax you need without paging back through the whole book. Nothing here is new; every command below was covered in depth earlier in this guide. Treat this chapter as a lookup table, not a tutorial.

All examples continue to use the same illustrative values used throughout the book: `project my-gcp-project-123456`, `region us-central1`, `zone us-central1-a`, `VPC my-vpc`.

Project & IAM Commands

Command	Purpose	Example
<code>gcloud config set project</code>	Sets the active project so it does not need to be passed with <code>--project</code> on every subsequent command	<code>gcloud config set project my-gcp-project-123456</code>
<code>gcloud projects describe</code>	Shows metadata for a project, including its project number and lifecycle state	<code>gcloud projects describe my-gcp-project-123456</code>
<code>gcloud projects add-iam-policy-binding</code>	Grants a role to a user, group, or service account at the project level	<code>gcloud projects add-iam-policy-binding my-gcp-project-123456 --member="user:ana@example.com" --role="roles/viewer"</code>
<code>gcloud projects remove-iam-policy-binding</code>	Revokes a previously granted project-level role binding	<code>gcloud projects remove-iam-policy-binding my-gcp-project-123456 --member="user:ana@example.com" --role="roles/viewer"</code>
<code>gcloud projects get-iam-policy</code>	Prints the full IAM policy attached to a project, useful for auditing who has access	<code>gcloud projects get-iam-policy my-gcp-project-123456 --format=json</code>
<code>gcloud iam service-accounts create</code>	Creates a new service account identity	<code>gcloud iam service-accounts create app-sa --display-name="Application service account"</code>
<code>gcloud iam service-accounts list</code>	Lists every service account in the current project	<code>gcloud iam service-accounts list</code>
<code>gcloud iam roles describe</code>	Shows the exact permissions bundled into a predefined or custom role	<code>gcloud iam roles describe roles/storage.objectViewer</code>
<code>gcloud organizations list</code>	Lists the organization resources the caller has visibility into	<code>gcloud organizations list</code>
<code>gcloud resource-manager folders create</code>	Creates a folder in the resource hierarchy under an organization or another folder	<code>gcloud resource-manager folders create --display-name="Production" --organization=123456789012</code>

Compute Engine Commands

Command	Purpose	Example
<code>gcloud compute instances create</code>	Creates a new virtual machine	<code>gcloud compute instances create web-server-1 --zone=us-central1-a --machine-type=e2-medium --image-family=debian-12 --image-project=debian-cloud</code>
<code>gcloud compute instances list</code>	Lists all VMs in the project, across zones unless filtered	<code>gcloud compute instances list --filter="zone:us-central1-a"</code>
<code>gcloud compute instances describe</code>	Shows full configuration detail for one instance	<code>gcloud compute instances describe web-server-1 --zone=us-central1-a</code>
<code>gcloud compute instances stop</code>	Stops a running instance without deleting it, halting compute billing	<code>gcloud compute instances stop web-server-1 --zone=us-central1-a</code>
<code>gcloud compute instances start</code>	Starts a previously stopped instance	<code>gcloud compute instances start web-server-1 --zone=us-central1-a</code>
<code>gcloud compute instances delete</code>	Permanently deletes an instance and, by default, its boot disk	<code>gcloud compute instances delete web-server-2 --zone=us-central1-a --quiet</code>
<code>gcloud compute instances add-metadata</code>	Adds or updates instance metadata, such as a startup script	<code>gcloud compute instances add-metadata web-server-1 --zone=us-central1-a --metadata=env=production</code>
<code>gcloud compute instance-templates create</code>	Creates a reusable instance blueprint for a managed instance group	<code>gcloud compute instance-templates create web-template-v1 --machine-type=e2-medium --image-family=debian-12 --image-project=debian-cloud</code>
<code>gcloud compute instance-groups managed create</code>	Creates a managed instance group (MIG) from a template	<code>gcloud compute instance-groups managed create web-mig --template=web-template-v1 --size=2 --zone=us-central1-a</code>
<code>gcloud compute instance-groups managed set-autoscaling</code>	Attaches an autoscaling policy to an existing MIG	<code>gcloud compute instance-groups managed set-autoscaling web-mig --zone=us-central1-a --max-num-replicas=10 --min-num-replicas=2 --target-cpu-utilization=0.6</code>

Networking Commands

Command	Purpose	Example
<code>gcloud compute networks create</code>	Creates a custom-mode VPC network	<code>gcloud compute networks create my-vpc --subnet-mode=custom</code>
<code>gcloud compute networks subnets create</code>	Creates a regional subnet inside a VPC	<code>gcloud compute networks subnets create my-subnet-central --network=my-vpc --region=us-central1 --range=10.0.0.0/24</code>
<code>gcloud compute firewall-rules create</code>	Creates a firewall rule governing ingress or egress traffic	<code>gcloud compute firewall-rules create allow-http --network=my-vpc --direction=INGRESS --action=ALLOW --rules=tcp:80,tcp:443 --source-ranges=0.0.0.0/0</code>
<code>gcloud compute firewall-rules list</code>	Lists every firewall rule attached to a network	<code>gcloud compute firewall-rules list --filter="network:my-vpc"</code>
<code>gcloud compute routers create</code>	Creates a Cloud Router, required infrastructure for Cloud NAT and dynamic routing	<code>gcloud compute routers create my-router --network=my-vpc --region=us-central1</code>
<code>gcloud compute routers nats create</code>	Creates a Cloud NAT gateway attached to a Cloud Router	<code>gcloud compute routers nats create my-nat --router=my-router --region=us-central1 --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges</code>
<code>gcloud compute backend-services create</code>	Creates a backend service that a load balancer routes traffic to	<code>gcloud compute backend-services create web-backend-service --protocol=HTTP --port-name=http --global</code>
<code>gcloud compute addresses create</code>	Reserves a static internal or external IP address	<code>gcloud compute addresses create my-lb-ip --global</code>
<code>gcloud compute vpn-tunnels create</code>	Creates an IPsec VPN tunnel to an on-premises peer gateway	<code>gcloud compute vpn-tunnels create tunnel-to-onprem --peer-address=198.51.100.20 --target-vpn-gateway=my-vpn-gateway</code>
<code>gcloud dns managed-zones create</code>	Creates a public or private Cloud DNS zone	<code>gcloud dns managed-zones create internal-zone --dns-name=internal. --visibility=private --networks=my-vpc</code>

Storage & Database Commands

Command	Purpose	Example
<code>gcloud storage buckets create</code>	Creates a new Cloud Storage bucket	<code>gcloud storage buckets create gs://my-app-assets --location=us-central1 --uniform-bucket-level-access</code>
<code>gcloud storage cp</code>	Copies files to, from, or between buckets	<code>gcloud storage cp ./site/index.html gs://my-app-assets/index.html</code>
<code>gcloud storage ls</code>	Lists buckets or objects	<code>gcloud storage ls gs://my-app-assets/</code>
<code>gcloud storage buckets update</code>	Updates bucket configuration, such as a lifecycle rule or retention policy	<code>gcloud storage buckets update gs://my-app-assets --lifecycle-file=lifecycle.json</code>
<code>gcloud sql instances create</code>	Creates a Cloud SQL database instance	<code>gcloud sql instances create my-sql-instance --database-version=POSTGRES_15 --tier=db-custom-2-8192 --region=us-central1</code>
<code>gcloud sql databases create</code>	Creates a logical database inside a Cloud SQL instance	<code>gcloud sql databases create appdb --instance=my-sql-instance</code>
<code>gcloud sql backups create</code>	Triggers an on-demand backup of a Cloud SQL instance	<code>gcloud sql backups create --instance=my-sql-instance</code>
<code>gcloud spanner instances create</code>	Creates a Cloud Spanner instance with a given compute capacity	<code>gcloud spanner instances create my-spanner-instance --config=regional-us-central1 --nodes=1 --description="my-app-db instance"</code>
<code>gcloud spanner databases create</code>	Creates a database within a Spanner instance	<code>gcloud spanner databases create my-app-db --instance=my-spanner-instance</code>
<code>gcloud redis instances create</code>	Creates a Memorystore for Redis instance	<code>gcloud redis instances create my-cache --size=1 --region=us-central1 --tier=standard</code>

Security Commands

Command	Purpose	Example
<code>gcloud kms keyrings create</code>	Creates a key ring to organize encryption keys	<code>gcloud kms keyrings create my-keyring --location=us-central1</code>
<code>gcloud kms keys create</code>	Creates an encryption key inside a key ring	<code>gcloud kms keys create my-key --keyring=my-keyring --location=us-central1 --purpose=encryption</code>

Command	Purpose	Example
<code>gcloud kms keys list</code>	Lists keys in a given key ring	<code>gcloud kms keys list --keyring=my-keyring --location=us-central1</code>
<code>gcloud secrets create</code>	Creates a new Secret Manager secret container	<code>gcloud secrets create db-password --replication-policy=automatic</code>
<code>gcloud secrets versions add</code>	Adds a new secret value as a version under an existing secret	<code>gcloud secrets versions add db-password --data-file=./password.txt</code>
<code>gcloud secrets versions access</code>	Retrieves the value of a specific (or latest) secret version	<code>gcloud secrets versions access latest --secret=db-password</code>
<code>gcloud secrets add-iam-policy-binding</code>	Grants a principal permission to read or manage a specific secret	<code>gcloud secrets add-iam-policy-binding db-password --member="serviceAccount:app-sa@my-g --role="roles/secretmanager.secretAc</code>

Monitoring & Billing Commands

Command	Purpose	Example
<code>gcloud logging read</code>	Queries Cloud Logging entries with a filter expression	<code>gcloud logging read "logName:activity AND protoPayload.methodName:SetIamPolicy --limit=10 --format=json</code>
<code>gcloud logging sinks create</code>	Routes matching log entries to a long-term destination such as BigQuery or Cloud Storage	<code>gcloud logging sinks create audit-export bigquery.googleapis.com/projects/my- --log-filter="logName:activity"</code>
<code>gcloud monitoring dashboards list</code>	Lists Cloud Monitoring dashboards configured in the project	<code>gcloud monitoring dashboards list</code>
<code>gcloud billing accounts list</code>	Lists billing accounts the caller has access to	<code>gcloud billing accounts list</code>
<code>gcloud billing projects link</code>	Links a project to a billing account so its usage can be charged	<code>gcloud billing projects link my-gcp-project-123456 --billing-account=012345-6789AB-CDEF</code>
<code>gcloud billing budgets create</code>	Creates a budget with alert thresholds against a billing account	<code>gcloud billing budgets create --billing-account=012345-6789AB-CDEF --display-name="Monthly budget" --budget-amount=1000USD</code>

Quick Reference: Useful Flags

- `--project=PROJECT_ID` — overrides the active project for a single command, without changing your persistent `gcloud` config default.

- `--format=json` (or `=yaml`, `=table(...)`) — controls how output is rendered; `json` is the most useful format when piping results into a script or another tool.
- `--filter="EXPRESSION"` — narrows a list or `read` command's results server-side, far more efficient than listing everything and filtering locally.
- `--quiet` — suppresses interactive confirmation prompts, essential for scripts and CI/CD pipelines where nothing is present to answer a y/N prompt.
- `--async` — returns immediately after submitting a long-running operation instead of blocking the terminal until it completes.
- `--impersonate-service-account=EMAIL` — runs the command as the named service account rather than your own user identity, using short-lived credentials.

TIP

Run `gcloud config set project my-gcp-project-123456` once at the start of a session instead of repeating `--project=my-gcp-project-123456` on every single command. It is easy to forget the flag on one command in a long session and accidentally operate against the wrong project — setting the default once removes that risk entirely.

BEST PRACTICE

Before running a command that creates, modifies, or deletes billed infrastructure, add `--format=json` or the equivalent read-only variant (`describe`, `list`) first to confirm you are targeting the resource you think you are. Several `gcloud compute` and `gcloud iam` commands also support a preview of the request without executing it — check `--help` on the specific command, since dry-run support varies by resource type rather than being universal across the CLI.

Chapter 25: Summary

This book set out to take you from a blank Google Cloud project to a working understanding of how production infrastructure actually gets built on GCP. Twenty-three chapters and one cheat sheet later, it is worth stepping back and looking at the shape of the journey as a whole, rather than at any one service in isolation.

The book began with identity and structure. Chapters 2 and 3 established the resource hierarchy — organizations, folders, and projects — and the IAM (Identity and Access Management) model that governs who can do what, at which level of that hierarchy. Nothing else in GCP makes sense without this foundation: every VM you create, every bucket you provision, every database you spin up lives inside a project, and every action against it is checked against an IAM policy. It is deliberately the first real technical content in the book, because every later chapter assumes it.

From there, the book moved into compute. Chapters 4 through 6 covered Compute Engine from a single VM to instance templates, managed instance groups, and autoscaling — the progression from “one server” to “a fleet of servers that scales and heals itself.” This is the layer most readers arrive at GCP already thinking about, and it sets up the vocabulary (instance templates, health checks, rolling updates) that resurfaces throughout the rest of the book.

Networking followed in Chapters 7 through 10: VPCs and subnets, firewall rules and Cloud NAT, load balancing, and finally DNS and hybrid connectivity. This section is where isolated compute resources become an actual system — instances that can reach each other, be reached from the internet through a controlled boundary, and be found by name rather than by hardcoded IP address. It is also, not coincidentally, where the most consequential production misconfigurations tend to happen, which is why firewall hygiene and NAT design got as much attention as they did.

Chapters 11 through 15 turned to storage and databases — Cloud Storage and its lifecycle and security controls, Cloud SQL, Spanner and Firestore, and Memorystore for caching. Here the theme was choosing the right tool for a given data shape and consistency requirement, rather than defaulting to whichever database is most familiar. A relational workload with clear transactional boundaries fits Cloud SQL; a globally distributed workload with strict consistency needs at scale points toward Spanner; a flexible document model favors Firestore; and a cache in front of any of them, via Memorystore, absorbs read pressure that would otherwise hit the database directly.

The final thematic group, Chapters 16 through 21, was security, operations, and cost: service accounts and workload identity, encryption and secret management, monitoring and logging, audit logs and Security Command Center, billing and budget controls, and Cloud Build for CI/CD. These are the chapters that separate a system that merely works from a system that is safe to operate, observable when something goes wrong, and affordable to keep running. It is easy to treat this material as an afterthought bolted onto “the real infrastructure” — this book deliberately did not, because in practice these concerns determine whether a production system survives its first security review, its first incident, and its first unexpectedly large invoice.

Chapter 22 then pulled every one of those threads together into a single capstone architecture — a highly available, disaster-recovery-aware design that used compute, networking, storage, and security controls from every earlier chapter as parts of one coherent system, rather than as isolated examples. If any single chapter demonstrates why the ordering of this book matters, it is that one: none of its decisions would make sense without the twenty-one chapters that came before it.

Where to Go Next

This guide focused on GCP’s core services and how to reason about them — what each one does, when to use it, and how the pieces fit together. Two companion guides from the same series pick up from here in different directions.

Terraform on GCP covers how to provision everything described in this book as code — the same projects, VPCs, instances, load balancers, and databases, defined declaratively, versioned, reviewed, and applied through a repeatable pipeline instead of one-off `gcloud` commands typed into a terminal.

If you found yourself wishing the commands in Chapter 24 could be captured once and reapplied reliably, that guide is the direct next step.

Docker, Kubernetes, GKE & GCP Artifact Registry covers how to package and run containerized workloads on top of the infrastructure this book teaches you to build — moving from VMs running a single application directly to containers orchestrated by Kubernetes, with images stored and scanned in Artifact Registry.

Together, the three guides form one learning path: this book establishes the GCP services and concepts you are provisioning and operating; the Terraform guide shows how to provision them reliably and repeatably; and the Kubernetes guide shows how to run modern, containerized applications on top of the result. Read in that order, they take you from a GCP beginner to someone who can design, provision, and operate a real production system end to end.

Certifications Worth Pursuing

If you want an external, structured way to validate what you have learned, three Google Cloud certifications map closely to the material in this book:

- **Associate Cloud Engineer** validates hands-on ability to deploy, monitor, and manage GCP resources day to day — the closest match to this book’s Chapters 1 through 21, and the natural first certification to pursue.
- **Professional Cloud Architect** validates the ability to design complete, resilient, cost-aware cloud solutions from business requirements — the skill Chapter 22’s capstone architecture was written to exercise.
- **Professional Cloud Network Engineer** validates deep expertise specifically in GCP networking — VPC design, hybrid connectivity, load balancing, and DNS — for readers who found Chapters 7 through 10 the most interesting part of this book and want to specialize further.

Closing Thoughts

The single best way to retain any of this is to keep building with it. Spin up a small project, wire together a VPC, a managed instance group, a Cloud SQL instance, and a load balancer, and let it run long enough to watch a health check fail, an autoscaler react, or a budget alert fire. Reading about these behaviors is useful; watching them happen to something you built yourself is what makes the concepts stick.

WARNING

As Chapter 20 covered, GCP resources bill for as long as they exist, not just while you are actively using them — a Cloud SQL instance, a Compute Engine VM, or a Memorystore instance left running after a learning exercise will keep generating charges quietly in the background. Get in the habit of tearing down anything you spin up for practice as soon as you are done with it, and double-check your billing dashboard afterward rather than assuming a delete command caught everything.

Thank you for working through this guide. Whichever direction you take next — deeper into Terraform, into Kubernetes, or straight into a certification exam — you now have a solid, practical foundation in the core services that the rest of Google Cloud is built on.